A sparsity-independent near-linear-space sketch for expander decomposition in dynamic streams

Yu Chen, Michael Kapralov, Mikhail Makarov, Davide Mazzali

arxiv:2404.16701v2; CC BY 4.0

Editorial note. Only cor:main is this task counted target. Complete author arguments and references retained. Native EPS figures are preserved and converted with installed Ghostscript to PDF for XeLaTeX, without changing drawing content; unused tikz-network load omitted.

Editorial note. Algorithmicx pseudocode uses installed algpseudocode in place of algpseudocodex indentation decoration; every algorithm instruction is retained.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

In this paper we study the problem of finding (ϵ, ϕ)-expander decompositions of a graph in the streaming model, in particular for dynamic streams of edge insertions and deletions. The goal is to partition the vertex set so that every component induces a ϕ-expander, while the number of inter-cluster edges is only an ϵ fraction of the total volume. It was recently shown that there exists a simple algorithm to construct a $(O(\phi \log n), \phi)$-expander decomposition of an n-vertex graph using $\widetilde{O}(n/\phi^2)$ bits of space [Filtser, Kapralov, Makarov, ITCS'23]. This result calls for understanding the extent to which a dependence in space on the sparsity parameter ϕ is inherent. We move towards answering this question on two fronts.

We prove that a $(O(\phi \log n), \phi)$-expander decomposition can be found using $\widetilde{O}(n)$ space, for every ϕ. At the core of our result is the first streaming algorithm for computing boundary-linked expander decompositions, a recently introduced strengthening of the classical notion [Goranci et al., SODA'21]. The key advantage is that a classical sparsifier [Fung et al., STOC'11], with size independent of ϕ, preserves the cuts inside the clusters of a boundary-linked expander decomposition within a multiplicative error.

Notable algorithmic applications use sequences of expander decompositions, in particular one often repeatedly computes a decomposition of the subgraph induced by the inter-cluster edges (e.g., the seminal work of Spielman and Teng on spectral sparsifiers [Spielman, Teng, SIAM Journal of Computing 40(4)], or the recent maximum flow breakthrough [Chen et al., FOCS'22], among others). We prove that any streaming algorithm that computes a sequence of $(O(\phi \log n), \phi)$-expander decompositions requires $\widetilde{\Omega}(n/\phi)$ bits of space, even in insertion only streams.

Contents

1 Introduction 1
- 1.1 Previous work . . . 1
- 1.2 Our contribution . . . 2
- 1.3 Basic notation . . . 3

2 Technical overview 4
- 2.1 Sparsity-independent one-level expander decomposition . . . 4
- 2.2 Two-level expander decomposition incurs a sparsity dependence . . . 11

3 Preliminaries 15

4 Sparsification for vertex-induced subgraphs 17
- 4.1 Sparsification primer . . . 18
- 4.2 Sampling algorithm for dynamic streams . . . 19
- 4.3 Cut counting and proving the sparsification lemma . . . 23

5 Testing expansion and finding sparse cuts in sparsifiers 26
- 5.1 Characterizing boundary-linked cuts . . . 27
- 5.2 Preserving volumes and sparsities . . . 28
- 5.3 Proving the proxying lemma . . . 31

6 Space efficient recursive partitioning 33
- 6.1 Offline analysis . . . 36
  - 6.1.1 Properties of nested cuts . . . 37
  - 6.1.2 Relations between volume estimates . . . 41
  - 6.1.3 Bounding the number of iterations . . . 41
  - 6.1.4 Bounding the recursion depth . . . 46
  - 6.1.5 Correctness . . . 49
- 6.2 Dynamic stream implementation . . . 53

7 Lower bound for two-level expander decomposition 55
- 7.1 Hard distribution . . . 55
- 7.2 Reduction to a communication problem . . . 60
- 7.3 Hardness of the communication problem . . . 65
- 7.4 Proving the lower bound . . . 68

References 70

A Technical facts 74
- A.1 Polynomial time balanced sparse cut algorithm . . . 74
- A.2 Expansion and regularity of random graphs . . . 79
- A.3 A useful inequality . . . 80

1 Introduction

Expander graphs are known to represent a class of easier instances for many problems. Therefore, breaking down the input into disjoint expanders can allow to conveniently solve the task on each of them separately, before combining the partial results into a global solution. This approach is enabled by (ϵ, ϕ)-expander decompositions (for short, (ϵ, ϕ)-ED). For an undirected graph $G = (V, E)$, this is a partition $\mathcal{U}$ of the vertex set V, such that there are at most $\epsilon|E|$ inter-cluster edges, while every cluster $U \in \mathcal{U}$ induces a ϕ-expander. The list of successful applications of this framework is long, including Laplacian system solvers [ST04], deterministic algorithms for minimum cut [Li21], graph and hyper-graph sparsification [ST11, CGP$^+$18, KKTY21, APP$^+$23], dynamic algorithms for cut and connectivity problems [GRST21, GHN$^+$23], fast max flow algorithms [CKL$^+$22a], distributed triangle enumeration [CS19], polynomial time algorithms for semirandom planted CSPs [GHKM23], and many more.

We refer to ϵ and ϕ as the sparsity parameters: the former controls how sparsely connected the clusters need to be, and the latter determines how expanding (i.e. non-sparse) the cuts within clusters are. The reason for using the same term for both is that they are in fact very closely related. One can show that any n-vertex graph has an (ϵ, ϕ)-ED with $\epsilon = O(\phi \log n)$. To see this, consider the following constructive argument: if the graph has no ϕ-sparse cut then $\{V\}$ is a valid ED of G, otherwise recurse on the two sides $(S, V \setminus S)$ of a ϕ-sparse cut and union the results to get an ED for G. Every cluster in this decomposition is then an expander, and a charging argument allows to bound the number of inter-cluster edges by $O(\phi|E| \log n)$ [SW19]. One can also observe that no better asymptotic trade-off between ϵ and ϕ is possible in general [AALG18]. Therefore, this sets the benchmark for algorithmic constructions of EDs.

At a high level, many ED algorithms follow the approach suggested by the existential argument. As a naive implementation would take exponential time, the crux often lies in efficient algorithms that either certify that a large portion of the input is an expander, or find a balanced sparse cut. This would result in a small depth recursion, thanks to balancedness, where each level requires little computational resources. There are several successful examples of this approach. In the sequential setting, a recent algorithm constructs a $(O(\phi \log^3 n), \phi)$-ED in $\tilde{O}(|E|)$ time [LNPS23], based on the previous best algorithm which runs in $\tilde{O}(|E|/\phi)$ time [SW19]. There is also a deterministic counterpart, which outputs a $(\phi \cdot n^{o(1)}, \phi)$-ED in almost linear time [CGL$^+$20]. For the CONGEST model of distributed computing, it is possible to obtain, for instance, a $(\phi^{1/\sqrt{\log n}} \cdot n^{o(1)}, \phi)$-ED in $n^{o(1)}/\phi$ rounds [CS19]. It is also possible to maintain a $(\phi \cdot n^{o(1)}, \phi)$-ED for a graph undergoing edge updates in $n^{o(1)}/\phi^2$ amortized update time [GRST21, HKGW23].

1.1 Previous work

In the streaming setting, the problem of finding EDs was open until the recent work of [FKM23]. They obtain a dynamic stream algorithm which outputs a $(O(\phi \log n), \phi)$-ED and takes $\tilde{O}(n/\phi^2)$ space. While being optimal in the quality of the decomposition, decoding the sketch to actually output an ED takes exponential time. The authors also give a polynomial time version: one can produce a $(\phi \cdot n^{o(1)}, \phi)$-ED using space $\tilde{O}(n/\phi^2) + n^{1+o(1)}/\phi^{1-o(1)}$, with a post-processing that takes poly(n) time (where the $o(1)$'s can be tuned, allowing for a quality-space trade-off).

These streaming algorithms also adopt the recursive approach based on finding balanced ϕ-sparse cuts, but the streaming model poses a challenge that we now illustrate. Sparsification for graph streams has been extensively studied [AG09, AGM12a, AGM12b, KLM$^+$14, KMM$^+$20], so a natural attempt would consist of maintaining a cut sparsifier as the stream comes and later run the recursive partitioning on it. However, it must be noted that these cuts are to be found in the

subgraphs induced by the two sides of a previously made cut. This is not a problem in a classical computational setting, but it actually constitutes the main obstacle for the streaming model: at sketching time, the algorithm does not know which subgraphs it will need to access. Unfortunately, it is impossible to preserve cut sizes in arbitrary subgraphs with multiplicative precision. The natural work-around is to introduce an additive error term: in [FKM23], the authors introduce the concept of power-cut sparsifiers. For any given partition $\mathcal{U}$ of V, these sparsifiers preserve the cuts $|E(S, U \setminus S)|$ of each cluster $U \in \mathcal{U}$ in the partition up to an error of $\delta \cdot |E(S, U \setminus S)| + \psi \cdot \mathrm{vol}(S)$. They further show that maintaining $\tilde{O}(n/\delta\psi)$ random linear measurements of the incidence matrix of the input graph is enough to obtain such sparsifiers. One can see that setting $\delta \ll 1$ and $\psi \approx \phi$ preserves the sparsity of cuts to within an additive error of roughly ϕ, using $\tilde{O}(n/\phi)$ linear measurements. This is enough to find a balanced ϕ-sparse cut in every subgraph induced by a given partition of the vertex set. There is another caveat, though: power-cut sparsifiers give a high probability guarantee for any fixed partition, but we cannot expect them to work for all partitions simultaneously. Therefore, in order not to use the sparsifiers adaptively, we need one power-cut sparsifier for every recursion level. The authors show that the depth of the procedure cannot exceed $\tilde{O}(1/\epsilon) = \tilde{O}(1/\phi)$, thus obtaining the space complexity stated before. A few more details are involved in the polynomial time algorithm, but the underlying framework is the same.

1.2 Our contribution

The work of [FKM23] initiated the study of expander decompositions in the streaming setting, and consequently raised the question of whether a dependence in space on the sparsity parameter ϕ is inherent. In this paper, we move towards settling the streaming complexity of expander decompositions by attacking the problem on two fronts: (1) we give a nearly optimal algorithm for "one-level" expander decomposition that avoids the sparsity dependence, and (2) we show that computing a "repeated" expander decomposition, commonly used in applications, cannot avoid such dependence.

Upper bound

We give an $\tilde{O}(n)$ space algorithm for computing a $(O(\phi \log n), \phi)$-ED in dynamic streams. Specifically, we show that a "universal" sketch consisting of $\tilde{O}(n)$ random linear measurements of the incidence matrix can be decoded into a $(O(\phi \log n), \phi)$-ED for any ϕ: the sketch is independent of the sparsity ϕ.

Theorem 1 (ED algorithm – exponential time decoding). Let $G = (V, E)$ be a graph given in a dynamic stream. Then, there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n)$ space. For any $\phi \in (0, 1)$, the algorithm decodes the sketch to compute a $(O(\phi \log n), \phi)$-ED of G with high probability, in $\tilde{O}(n)$ space and $2^{O(n)}$ time.

We note that at least $\Omega(n \log n)$ space is needed for any small enough ϕ: for example, if the input graph is a matching of size $n/10$, say, its ED gives a $1 - O(\phi \log n)$ fraction of the matching edges.

The decoding time of our sketch can be made polynomial, at the expense of some loss in the quality of the expander decomposition (similarly to [FKM23]), but keeping the space independent of the sparsity ϕ.

Theorem 2 (ED algorithm – polynomial time decoding). Let $G = (V, E)$ be a graph given in a dynamic stream. Then, there is an algorithm that maintains a linear sketch of G in $n^{1+o(1)}$ space. For any $\phi \in (0, 1)$, the algorithm decodes the sketch to compute a $(\phi \cdot n^{o(1)}, \phi)$-ED of G with high probability, in $n^{1+o(1)}$ space and $\mathrm{poly}(n)$ time.

In this case we are off by subpolynomial factors in both quality and space complexity as compared to the optimal ones. The actual theorem that we prove allows one to trade-off the loss in quality and increase in space.

Lower bound

Most algorithmic applications of EDs, including the ones mentioned above, do not use just one ED of the input graph. Rather, they use an ED sequence obtained by repeatedly computing an ED of the inter-cluster edges from the previous level. This can be done in two natural ways: by contracting the clusters of an ED (we call this variant CED, for "contraction"), or by removing the intra-cluster edges without changing the vertex set (we call this variant RED, for "removal"). These approaches lead to different results and are used for different applications (e.g., [GRST21, Li21] contract the clusters, and [ST11, CKL$^+$22a] recurse on inter-cluster edges). In the sequential setting, both CEDs and REDs can be obtained straightforwardly given an ED algorithm. However, this is not so obvious in the streaming model.

One the one hand, one should be able get a sparsity-independent algorithm for computing CEDs in dynamic streams via Theorem 1 or Theorem 2: observe that contracting the vertices of a sparsifier of the input graph G gives a sparsifier of the graph obtained by contracting vertices in G, so the idea would be to maintain an independent copy of our algorithm for each of the $O(\log n)$ levels and contracting vertices in the sparsifier based on the decomposition of the previous level. On the other hand, we show a space lower bound for computing REDs, even in insertion only streams, showing that a dependence on $1/\phi$ is necessary.

Theorem 3 (RED lower bound). Let $\epsilon, \phi \in (0,1)$ such that $1/n \ll \phi \ll 1/\operatorname{polylog} n$ and $\epsilon = \tilde{O}(\phi)$. Any streaming algorithm that with constant probability computes at least two levels of an (ϵ, ϕ)-RED requires $\widetilde{\Omega}(n/\phi)$ bits of space.

The result seems to challenge the intuition that EDs become weaker as ϕ and ϵ decrease (note that when ϕ is, say, $1/n^2$, an ED can simply consist of connected components). The questions of (1) whether this bound can be improved, (2) how it scales with the number of levels of RED we compute, and (3) whether there are algorithms matching such bounds, remain open.

1.3 Basic notation

Graph streaming. In this paper, we will be mostly working in the dynamic graph streaming model, where we know the vertex set $V = [n]$, and we receive a stream of insertions and deletions for undirected edges over V. In insertion-only graph streams, the only difference is that previously inserted edges cannot be deleted. At the end of the stream, the graph $G = (V, E)$ consists of the edges that have been inserted and not deleted, and we say that G is given in a (dynamic) stream. When we consider a graph given in a (dynamic) stream, we are implicitly assuming it to have n vertices, without introducing the parameter n explicitly. Also, when we refer to G and n without reintroducing them we are implicitly considering the graph resulting from the input stream and its number of vertices.

A powerful tool for dynamic graph streams is linear sketching, introduced in the seminal work of Ahn, Guha, and McGregor [AGM12a]. The idea is to left-multiply the $\binom{n}{2} \times n$ incidence matrix of the graph by a random $k \times \binom{n}{2}$ matrix for $k \ll \binom{n}{2}$. Since the sketch consists of linear measurements, it automatically handles the case of dynamic streams. We will not be using sketching techniques directly, but rather employ existing algorithms that do. In this paper we restrain ourselves to unweighted edge streams. One may study the same problem in general turnstile graph streams [CKL22b], but we do not do this here.

Cuts, volumes, expanders. Given an unweighted graph $G = (V, E)$, possibly with multiple self-loops on the vertices, we will operate with weighted graphs that approximate G in an appropriate sense. We write $G' = (V', E', w)$ to denote a weighted graph, possibly with multiple weighted self-loops on the vertices. We describe next some notation for such weighted graph G'. The same notation carries over to the unweighted graph G by implicitly setting w to assign a weight of 1 to all edges and self-loops.

For any $A, B \subseteq V'$ we denote by $E'(A, B)$ the edges in E' with one endpoint in A and one in B, and by $w(A, B)$ the total weight of edges in $E'(A, B)$. The volume of a cut $S \subseteq V'$ is the sum of the (weighted) degrees, including self-loops, of its vertices. We denote it by $\mathrm{vol}_{G'}(S)$. The sparsity of a cut $\emptyset \neq S \subsetneq V'$ is defined as

$$\Phi_{G'}(S) = \frac{w(S, V' \setminus S)}{\min\{\mathrm{vol}_{G'}(S), \mathrm{vol}_{G'}(V' \setminus S)\}}\,.$$

For $\psi \in (0, 1)$, we make a distinction between cuts having sparsity less than ψ, which we call ψ-sparse, and cuts having sparsity at least ψ, which we call ψ-expanding. The sparsity of G' is the defined as

$$\Phi_{G'} = \min_{\emptyset \neq S \subsetneq V'} \Phi_{G'}(S)\,,$$

and we call G' a ψ-expander if all its cuts are ψ-expanding, i.e. $\Phi_{G'} \geq \psi$.

For a pair of vertices $e = \{u, v\} \in \binom{V'}{2}$, we denote by $\lambda_e(G')$ the edge connectivity of e in G', i.e.

$$\lambda_e(G') = \min_{\emptyset \neq S \subsetneq V' : u \in S, v \in V' \setminus S} w(S, V' \setminus S)\,.$$

Expander decomposition. As we will treat expander decompositions for the input graph G only, we conveniently use the following additional notation. For a cluster $U \subseteq V$, i.e. a subset of the vertices, and a cut $S \subseteq U$, which is also a subset of the vertices, we denote the number of edges crossing S in U by $\partial_U S$, i.e. $\partial_U S = |E(S, U \setminus S)|$. This is the local cut of S in U. If $U = V$, we use a shorthand notation $\partial S \coloneqq \partial_V S$. We call such a cut the global cut of S, since for $S \subseteq U$ we will be interested in both $\partial_U S$ and ∂S. Moreover, we drop the subscript from the volume and simply write $\mathrm{vol}(\cdot)$ instead of $\mathrm{vol}_G(\cdot)$. Then, EDs can be defined as follow.

Definition 1.1 (Expander decomposition). Let $G = (V, E)$ and let $\epsilon, \phi \in (0, 1)$. A partition $\mathcal{U}$ of V is an (ϵ, ϕ)-expander decomposition (for short, (ϵ, ϕ)-ED) of G if

1. $\frac{1}{2} \sum_{U \in \mathcal{U}} \partial U \leq \epsilon |E|$, and
2. for every $U \in \mathcal{U}$, one has that $G[U]$ is a ϕ-expander.

2 Technical overview

In this section, we outline our techniques, and give intuition for how to prove our results.

2.1 Sparsity-independent one-level expander decomposition

Given a graph $G = (V, E)$ in a dynamic stream, and a parameter $\phi \in (0, 1)$, we consider the problem of computing an (ϵ, ϕ)-ED of G for $\epsilon = O(\phi \log n)$. We show that one can do this in $\tilde{O}(n)$ bits of space, without any dependence on ϕ. In this section, we sketch our approach.

Let us begin by recalling the standard recursive framework for constructing expander decompositions [KVV00, Tre08, ST11], concisely summarized in Algorithm 1. For any parameter $\phi \leq 10^{-1}/\log n$, this algorithm produces a $(O(\phi \log n), \phi)$-ED by recursively partitioning G along ϕ-sparse cuts until no more such cuts are found.

Algorithm 1 Decompose: recursive procedure for computing a $(O(\phi \log n), \phi)$-ED of G

```
▷ φ ∈ (0, 1) is the sparsity parameter
procedure Decompose(G)                          ▷ G = (V, E) is the input graph
    if G is a φ-expander then return {V}
    else
        S ← a φ-sparse cut of G
        return Decompose(G[S]) ∪ Decompose(G[V \ S])
    end if
end procedure
```

Many algorithmic constructions of EDs are essentially efficient implementations of Algorithm 1. Adapting Algorithm 1 to dynamic streams comes with its own set of challenges. When the input is given in a dynamic stream, one can only afford to store a limited amount of information about the input graph. Since Algorithm 1 only needs to measure the sparsity of cuts, it seems enough to have access to cut sizes and volumes. Both these quantities are preserved by cut sparsifiers. According to the classical definition [BK15], a δ-cut sparsifier is a weighted subgraph $H = (V, E', w)$ of the input $G = (V, E)$, where for every cut $S \subseteq V$ one has

$$(1 - \delta) \cdot \partial S \leq \partial^w S \leq (1 + \delta) \cdot \partial S \,.$$

It is known that such a sparsifier can be constructed in dynamic streams using $\tilde{O}(n/\delta^2)$ bits of space [AGM12b]. With the same space requirement we can also measure the sparsity of cuts up to a $(1 \pm \delta)$ multiplicative error. Having this in mind, it is natural to consider the following algorithm: first, read the stream and construct a cut sparsifier, then run Algorithm 1 on it and output the resulting clustering as the expander decomposition. However, this approach does not work as is. As it turns out, more information is needed about the graph than what is captured by the regular notion of a cut sparsifier.

The problem with this approach becomes immediately apparent when one considers the second recursion level. As the process recurses on the two sides of a sparse cut S, it will repeat the procedure on the subgraphs $G[S]$ and $G[V \setminus S]$. Unfortunately, the cut preservation property of the sparsifier does not carry over to those subgraphs, making it inadequate for estimating the sparsity of the cuts in those graphs. In fact, the notion of expander decomposition itself already operates on the subgraphs, as it is required that each cut in each cluster of the decomposition is expanding.

Testing expansion of subgraphs

The reasoning from above gives rise to the following sketching problem: produce a sparsifier that can be used to check that any subgraph is a ϕ-expander. To solve it, the authors of [FKM23] introduced the concept of a (δ, ψ)-power-cut sparsifier. Its property is that, for any cluster $U \subseteq V$, with high probability all cuts $S \subseteq U$ are preserved within an additive-multiplicative error. Recall that $\partial_U S$ is equal to the size of the cut S inside the induced subgraph $G[U]$. When talking about sparsifiers in this section, we will slightly abuse notation by assuming that there is some instance $H = (V, E', w)$ of it and denote by $\partial_U^w S$ the size of the cut S in the subgraph $H[U]$ of this sparsifier H. Then we

can write the guarantee of a (δ, ψ)-power-cut sparsifier as follows[1]:

$$\forall\, S \subseteq U, \quad (1-\delta)\cdot \partial_U S - \psi \cdot \mathrm{vol}(S) \leq \partial_U^w S \leq (1+\delta)\cdot \partial_U S + \psi \cdot \mathrm{vol}(S)\,.$$

The authors also give a dynamic stream construction, which uses $\tilde{O}(n/\delta\psi)$ bits of space by sampling edges proportionally to the degrees of their endpoints.

To check the ϕ-expansion of subgraphs up to a small constant multiplicative error, it is enough to set δ to be a small constant. However, the multiplicative error parameter ψ must be $\lesssim \phi$. This is a significant downside of this construction, as it was shown in [FKM23] that a (δ, ϕ)-power-cut sparsifier must have at least $\Omega(n/\phi)$ edges. In fact, their lower bound is more general, and holds for general subgraphs[2]. In other words, $\Omega(n/\phi)$ space is necessary in order to test ϕ-expansion of general subgraphs. Consequently, new tools must be used to have any hope of getting an algorithm independent of $1/\phi$.

Our contribution: sparsification of boundary-linked subgraphs As it turns out, solving the expansion testing problem, as it was stated, is not necessary. Recently, in the breakthrough work of [GRST21], it was shown that one could demand additional properties from the expander decomposition, and it will still exist at the price of increasing the number of inter-cluster edges by a small multiplicative factor.

More formally, for a set of vertices $U \subseteq V$ and $\tau > 0$, we denote by $G[U]^\tau$ an induced subgraph of G on U, where additionally for each edge $e = \{u, v\}$ that connects a vertex u inside U with a vertex v outside of U, τ self-loops are added to u. The graph $G[U]^\tau$ is called a τ-boundary-linked subgraph. Note that if $G[U]^\tau$ is a ϕ-expander, then so necessarily is $G[U]$, but not the other way around.

Then it is possible, for a given ϕ, and $\epsilon = \tilde{O}(\phi)$, $\tau \approx 1/\phi$, to construct an (ϵ, ϕ)-expander decomposition $\mathcal{U}$ where for each cluster $U \in \mathcal{U}$, $G[U]^\tau$ is a $\widetilde{\Omega}(\phi)$-expander [GRST21]. Such a decomposition is called a boundary-linked expander decomposition.

A key observation is that if in our algorithm we were aiming to construct a boundary-linked expander decomposition, the testing problem would only involve checking that any given boundary-linked subgraph is ϕ-expanding. Indeed, this problem is much easier than the original one and can be solved in space $\tilde{O}(n)$. We will show how to do it in two steps: first, we will discuss how to strengthen the power-cut sparsifier, and then prove that this strengthening is enough to resolve the problem.

Achieving additive error in the global cut As was noted in [FKM23], the idea behind constructing a power-cut sparsifier was to reanalyse the guarantee given by the construction of [ST11] for sparsifying expanders. In other words, a power-cut sparsifier results from a more rigorous analysis of an existing sparsifier. The problem with this approach is that the sparsifier of [ST11] is relatively weak to begin with: it only preserves cuts in expanders, while other constructions can preserve them in all graphs [BK15, FHHP11]. To strengthen the guarantee, we give the same treatment to the construction of [FHHP11] in Section 4. For the sake of simplicity and to gain an intuition for why this kind of sparsification is at all possible, we discuss here how to do that with the classical construction of [Kar94] by closely following the original proof.

[1] A similar sparsifier construction was proposed by [AKLP22]. Their construction has an additive error of $\psi|S|$, so the dependence is on the number of vertices instead of the volume

[2] The lower bound instance is a $1/\phi$-regular graph. There, each edge by itself forms a ϕ-expander, while any pair of vertices without an edge between them is not. Being able to test the ϕ-expansion of the majority of those small subgraphs would imply being able to recover the majority of edges in the graph.

We show that, given a graph $G = (V, E)$ with a minimum cut of size k, it is possible to construct a sparsifier H of G such that every cut inside any given subgraph $G[U]$ of G is preserved with high probability with the following guarantee:

$$\forall\, S \subseteq U, \quad \partial_U S - \delta \cdot \partial S \leq \partial_U^w S \leq \partial_U S + \delta \cdot \partial S\,. \tag{2.1}$$

In this paper, a sparsifier with property (2.1) is called a cluster sparsifier (see Definition 4.1). To achieve (2.1), consider using the same process as in [Kar94]: sample each edge with the same probability $p \approx \delta^{-2}/k$.

To see why this works, fix a subgraph $U \subseteq V$, and consider any cut S in U. We wish to show that the size of the cut S inside U concentrates well after sampling. In the original proof, this is done by simply applying a Chernoff bound. In our case, this bound would look like this:

$$\Pr[|\partial_U^w S - \partial_U S| \geq \delta \cdot \partial_U S] \leq \exp\left(-\frac{1}{3}\delta^2 p \cdot \partial_U S\right)\,.$$

However, this is insufficient, as the probability depends on the cut size inside $G[U]$. As we have no lower bound on its size, unlike with the sizes of global cuts, the second part of the argument of [Kar94] cannot be applied. Instead, we apply an additive-multiplicative version of the Chernoff bound (see for example [FKM23]), that allows us to compare the approximation error with a bigger value than its expectation. This gives us

$$\Pr[|\partial_U^w S - \partial_U S| \geq \delta \cdot \partial S] \leq 2\exp\left(-\frac{1}{100}\delta^2 p \cdot \partial S\right)\,.$$

Expressing the probability in terms of the global cut allows us to use the cut counting lemma [Kar93], which bounds the number of global cuts of size at most αk by $n^{2\alpha}$, for $\alpha \geq 1$. The proof is concluded by associating each global cut with a local cut in $G[U]$ and taking the union bound over them.

In order to get the guarantee of (2.1) for all graphs, not only those with a minimum cut of size k, one can sample edges proportionally to the inverse of their connectivity (as in the work of [FHHP11]) as opposed to uniformly. Moreover, one can implement such sampling scheme in dynamic streams, using the approach of Ahn, Guha, and McGregor [AGM12b]. We defer the details of these reanalyses to Section 4, where we show how to construct cluster sparsifiers with property (2.1) in dynamic streams.

Benefits of boundary-linked graphs To see why the cluster sparsifier is enough to solve the boundary-linked ϕ-expansion testing problem, consider the following reasoning. Set the self-loop parameter τ equal to b/ϕ, for some $b \gg \phi$ and $b \ll 1$. Fix a cluster U, for which $G[U]^{b/\phi}$ is an $\widetilde{\Omega}(\phi)$-expander. The crucial fact is that the size of any cut inside $G[U]^{b/\phi}$ is lower bounded by its size in the global graph up to a small polylogarithmic factor in the following way:

$$\partial_U S \geq \widetilde{\Omega}(b) \cdot \partial S\,. \tag{2.2}$$

We will now explain the derivation of the above equation in detail. For a cut $\emptyset \neq S \subsetneq U$, we denote by $\mathrm{bnd}_U(S) = \partial S - \partial_U S$ the number of edges going from S to $V \setminus U$. Note that by the definition of $G[U]^{b/\phi}$, the volume of any cut S inside of it is equal to $\mathrm{vol}_{G[U]^{b/\phi}}(S) = \mathrm{vol}(S) + (\frac{b}{\phi} - 1)\,\mathrm{bnd}_U(S)$. First, because we assume that $G[U]^{b/\phi}$ is an $\widetilde{\Omega}(\phi)$-expander, we have

$$\partial_U S \geq \widetilde{\Omega}(\phi) \cdot \mathrm{vol}_{G[U]^{b/\phi}}(S)\,.$$

Then, applying the aforementioned formula for $\mathrm{vol}_{G[U]^{b/\phi}}(S)$, we have

$$\partial_U S \geq \widetilde{\Omega}(\phi)\,\mathrm{vol}(S) + \widetilde{\Omega}(\phi)\left(\frac{b}{\phi} - 1\right)\mathrm{bnd}_U(S)\,.$$

Since $b \gg \phi$ and dropping the first summand, the above simplifies to

$$\partial_U S \geq \widetilde{\Omega}(b)\,\mathrm{bnd}_U(S)\,.$$

Finally, applying $\partial_U S + \mathrm{bnd}_U(S) = \partial S$ and $b \ll 1$, we arrive back at equation (2.2).

A consequence of equation (2.2) is that setting $\delta \approx 1/b$ in the cluster sparsifier produces the following guarantee for $\widetilde{\Omega}(\phi)$-expander subgraphs $G[U]^{b/\phi}$:

$$\forall\, S \subseteq U, \quad \partial_U S - O(1)\cdot\partial_U S \leq \partial_U^w S \leq \partial_U S + O(1)\cdot\partial_U S\,,$$

where the $O(1)$ can be made arbitrarily small. In other words, we achieve a multiplicative approximation guarantee on subgraphs of interest, which is enough to solve the testing problem. Since b can be set to be $1/\operatorname{polylog} n$, the final sparsifier size does not depend on $1/\phi$. In this sense, such sparsifier can be thought of as a "universal" sketch of the graph that allows to solve the testing problem for all ϕ simultaneously.

Even though we now know how to solve the testing problem, an implementation of Algorithm 1 would need to actually find a sparse cut when the test fails (and in particular, it needs to find a balanced sparse cut, see the next section). In Section 5, we show that cluster sparsifiers enable to solve this harder task too: consider an offline algorithm that either correctly determines a graph to be a boundary-linked expander or finds a (balanced) sparse cut; we show that one can run such algorithm on a subgraph of the sparsifier as a black box and obtain essentially the same result as if the algorithm was run on the corresponding subgraph of the original graph. In this sense, cluster sparsifiers serve as small error proxies to the original graph for expander-vs-sparse-cut type of queries on vertex-induced subgraphs.

Low depth recursion

Another problem that arises when using the recursive approach is that the subgraph sparsification guarantee of both power-cut sparsifiers and cluster sparsifiers is only probabilistic, and it can be shown that it cannot be made deterministic [FKM23]. This means that after finding a sparse cut S in a subgraph $G[U]^{b/\phi}$ of a sparsifier, we cannot claim that the same sparsifier would preserve the cuts inside the new graph $G[S]^{b/\phi}$ with high probability, as it is dependent on another cut that we have already found inside the sparsifier. This means we cannot use the same sparsifier for two different calls to Algorithm 1 inside the same execution path. However, we can share a sparsifier among all the calls at the same recursion level since these will operate on independent portions of G. Therefore, we want to have a separate sparsifier for every recursion level. This means that in order to minimize space requirements, it is crucial to have a small recursion depth.

To have small recursion depth, the algorithmic approach of [FKM23, CS19, CPSZ21] enforces the sparse cut S from line (5) to be balanced, i.e., none of S and $V \setminus S$ is much larger than the other. In particular, they only recurse on the two sides of a cut if $\mathrm{vol}(S) \gtrsim \epsilon\,\mathrm{vol}(V \setminus S)$. When there is no balanced sparse cut and yet the input is not an expander, a lemma of Spielman and Teng [ST11] suggests there should be an $\Omega(\phi)$-expander $G[S']$ that accounts for a $(1-O(\epsilon))$-fraction of the total volume. An algorithmic version of this structural result allows us to iteratively trim off a small piece of the graph until such S' is found. At this point, the algorithm of [FKM23] can simply return $\{S'\} \cup (\cup_{u \in V\setminus S'}\{u\})$ as an ED, with at most $O(\epsilon|E|)$ inter-cluster edges between singletons. As the volume of the cluster multiplicatively decreases by $1 - O(\epsilon)$ after each call, this gives recursion depth at most $\tilde{O}(1/\epsilon) \approx \tilde{O}(1/\phi)$.

Our contribution: adaptation of trimming. We show that a simple refinement of this approach allows us to adapt the framework of [SW19], which leads to an algorithm with recursion depth independent of ϕ. We run the same algorithm, but instead of separating each vertex in $V \setminus S'$ into its own singleton cluster, we recurse with Algorithm 1 on the whole set $V \setminus S'$. Conceptually, this implements an analogue of the trimming step of [SW19].

This means that at the end, fewer edges in $V \setminus S$ become inter-cluster edges. Because of that, we can strengthen the balancedness requirement: we recurse on the two sides of a sparse cut only if $\mathrm{vol}(S) \gtrsim \frac{1}{C}\mathrm{vol}(V \setminus S)$ for a large constant C. This allows us to trim more vertices each time, resulting in $\tilde{O}(1)$ depth of the trimming step. On the other hand, because in each call to Algorithm 1 the volume of clusters passed to recursive calls is decreased by at least a constant factor, the total recursion depth becomes at most $\tilde{O}(1)$.

Other than the refinement discussed above, our space efficient implementation of Algorithm 1, as well as the iterative procedure to find the large expander S', are almost the same as the ones of [FKM23] (which in turn are inspired by the one of [CS19]). The details of the algorithms are given in Section 6.

Putting it all together

Combining the ideas illustrated in the two sections above, neither the sparsifier's size nor the recursion depth depend on ϕ, thus giving a sparsity-independent space algorithm for boundary-linked expander decomposition (BLD for short). This result is stated in terms of parameters $b, \epsilon, \phi \in (0,1)$, $\gamma \geq 1$: a $(b, \epsilon, \phi, \gamma)$-BLD is a partition $\mathcal{U}$ with at most an ϵ fraction of crossing edges and every $U \in \mathcal{U}$ induces a ϕ/γ-expander $G[U]^{b/\phi}$ (see Section 3 and Definition 3.1). Using this terminology, we obtain the following result.

Theorem 2.1 (Exponential time decoding BLD). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $b \in (0,1)$ be a parameter such that $b \leq 1/\log^2 n$. Then, there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n/b^3)$ space. For any $\epsilon \in [n^{-2}, b\log n]$, the algorithm decodes the sketch to compute, with high probability and in $\tilde{O}(n/b^3)$ space and $2^{O(n)}$ time, a $(b, \epsilon, \phi, \gamma)$-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\log n}\right) \quad \text{and} \quad \gamma = O(1)\,.$$

From this, one can easily conclude our main result, restated here for convenience of the reader.

Theorem 1 (ED algorithm – exponential time decoding). Let $G = (V, E)$ be a graph given in a dynamic stream. Then, there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n)$ space. For any $\phi \in (0,1)$ such that $\phi \leq c/\log^2 n$ for a small enough constant $c > 0$, the algorithm decodes the sketch to compute a $(O(\phi \log n), \phi)$-ED of G with high probability, in $\tilde{O}(n)$ space and $2^{O(n)}$ time.

We remark that for $\phi \geq \Omega(1/\log^2 n)$, one can use the algorithm of [FKM23] to still have an $\tilde{O}(n)$ space construction of a $(O(\phi \log n), \phi)$-ED.

Proof. First note that without loss of generality we can assume $\phi \geq 1/n^2$, otherwise an ED can simply consists of the connected components of G (which can be computed in dynamic streams in $\tilde{O}(n)$ space [AGM12a]). Then, we note that since c is small enough and $\phi \leq c/\log^2 n$, one can always define $\epsilon = C \cdot \phi \cdot \log n$ for an appropriate constant $C > 0$ while ensuring $1/n^2 \leq \epsilon \leq 1/\log n$. We can thus prove the theorem by equivalently showing that there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n)$ space, and that for all $\epsilon \in [1/n^2, 1/\log n]$ decodes the sketch to compute, with high probability, an $(\epsilon, \Omega(\epsilon/\log n)))$-ED of G in $\tilde{O}(n)$ space and $2^{O(n)}$ time.

Let then $\epsilon \in [1/n^2, 1/\log n]$. We use the algorithm from Theorem 2.1 with parameter ϵ and a parameter b of our choice. We need to meet two preconditions: $b \leq 1/\log^2 n$ and $\epsilon \leq b \log n$. Since we assume $\epsilon \leq 1/\log n$, we can set $b = 1/\log^2 n$, and all the prerequisites are fulfilled. Then the algorithm from Theorem 2.1 runs in $2^{O(n)}$ time and takes $\tilde{O}(n/b^3) = \tilde{O}(n)$ bits of space. The output $\mathcal{U}$ is a $(b, \epsilon, \phi, \gamma)$-BLD of G with high probability, where $\phi = \Omega(\epsilon/\log n)$ and $\gamma = O(1)$. Since a $(b, \epsilon, \phi, \gamma)$-BLD of G is an $(\epsilon, \phi/\gamma)$-ED of G, we have obtained an $(\epsilon, \Omega(\epsilon/\log n))$-ED of G with high probability. □

The exponential time in the decoding is due to the subtask of finding a balanced sparse cut. As we show, one can make the decoding time polynomial by resorting to known offline approximation algorithms [SW19, LNPS23]. However, we only have $\log^{\Omega(1)} n$-approximations for finding a balanced sparse cut, and in particular, we do not expect (under NP-hardness and the Unique Games Conjecture) there to be a polynomial time $O(1)$-approximation [CKK+06]. Such super-constant factor error incurs some loss in the quality of decomposition and space requirement, which, nevertheless, remains independent of the sparsity.

Theorem 2.2 (Polynomial time decoding BLD). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $b \in (0, 1)$ be a parameter such that $b \leq 1/\log^5 n$. Then, there is an algorithm that maintains a linear sketch of G in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space. For any $\epsilon \in [n^{-2}, b \log n]$, the algorithm decodes the sketch to compute, with high probability and in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space and poly(n) time, a $(b, \epsilon, \phi, \gamma)$-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\log^4 n}\right) \quad \text{and} \quad \gamma = \log^{O\left(\frac{\log n}{\log 1/b}\right)} n\,.$$

Theorem 2, restated here for convenience, then follows from Theorem 2.2.

Theorem 2 (ED algorithm – polynomial time decoding). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $b \in (0, 1)$ be a parameter such that $b \leq 1/\log^5 n$. Then, there is an algorithm that maintains a linear sketch of G in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space. For any $\phi \in (0, 1)$ such that $\phi \leq b/\log^{C \cdot \log n/\log \frac{1}{b}} n$ for a large enough constant $C > 0$, the algorithm decodes the sketch to compute a $(\phi \cdot \log^{O(\log n/\log \frac{1}{b})} n, \phi)$-ED of G with high probability, in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space and poly(n) time.

We remark that setting, say, $b = 2^{-\sqrt{\log n}}$ in the above result gives a $n^{1+o(1)}$ space algorithm for computing a $(\phi \cdot n^{o(1)}, \phi)$-ED for any $\phi \leq 2^{-2C \log\log n \sqrt{\log n}}$. For larger values of ϕ, one can use the polynomial time algorithm of [FKM23] to still get a $n^{1+o(1)}$ space construction for a $(\phi \cdot n^{o(1)}, \phi)$-ED.

Proof. As in the proof of Theorem 1, we can assume $\phi \geq 1/n^2$. Also, by virtue of C being a large enough constant and $\phi \leq b/\log^{C \cdot \log n/\log \frac{1}{b}} n$, one can always define $\epsilon = \phi \cdot \log^{C \cdot \log n/\log \frac{1}{b}} n$ while ensuring $n^{-2} \leq \epsilon \leq b \log n$. Then, we equivalently prove that for any $b \in (0, 1)$ with $b \leq 1/\log^5 n$ there is an algorithm that maintains a linear sketch of G in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space, and that for any $\epsilon \in (0, 1)$ such that $n^{-2} \leq \epsilon \leq b \log n$ decodes the sketch to compute, with high probability, an $(\epsilon, \epsilon/\log^{O(\log n/\log \frac{1}{b})} n)$-ED of G in $n/b^3 \cdot \log^{O(\log n/\log \frac{1}{b})} n$ space and poly(n) time.

Let then $b, \epsilon \in (0, 1)$ with $b \leq 1/\log^5 n$ and $n^{-2} \leq \epsilon \leq b \log n$. We use the algorithm from Theorem 2.2 with the same parameters b and ϵ, since every admissible pair of parameters b and ϵ fulfils the conditions of Theorem 2.2. The space complexity is also the same, and the running time is poly(n). Again, observe that a $(b, \epsilon, \phi, \gamma)$-BLD of G is an $(\epsilon, \phi/\gamma)$-ED of G. Hence the claim, since Theorem 2.2 gives

$$\frac{\phi}{\gamma} = \Omega\left(\frac{\epsilon}{\log^4 n}\right) \cdot \frac{1}{\log^{O\left(\frac{\log n}{\log 1/b}\right)} n} = \frac{\epsilon}{\log^{O\left(\frac{\log n}{\log 1/b}\right)} n}\,.$$

□

The proofs of Theorem 2.1 and Theorem 2.2 can be found in Section 6.

2.2 Two-level expander decomposition incurs a sparsity dependence

Given a graph $G = (V, E)$ in a stream and parameters $\epsilon, \phi \in (0, 1)$, we consider the problem of computing a two-level (ϵ, ϕ)-RED of G. In other words, we study the problem of computing an (ϵ, ϕ)-ED $\mathcal{U}$ of G and an (ϵ, ϕ)-ED $\mathcal{U}'$ of the graph $G' = (V, E \setminus \mathcal{U})$, where $E \setminus \mathcal{U}$ denotes the set of inter-cluster edges of $\mathcal{U}$, i.e. the edges of E that are not entirely contained in a cluster $U \in \mathcal{U}$ (see Section 3 and Definition 3.2). We remark that we wish to do so in a single pass over the stream.

A natural algorithmic approach and why it fails. A naive attempt to solve this problem would be that of sketching the graph twice, for example by using our algorithm from Theorem 1. One can use the first sketch to construct the first level ED $\mathcal{U}$. After, the hope is that one can send updates to the second sketch so as to remove the intra-cluster edges and then decode this sketch into an ED of $G' = (V, E \setminus \mathcal{U})$ using again Theorem 1. However, this hope is readily dashed. Indeed, sketching algorithms break down if we send a removal update for an edge that was not there in the first place, and we do not have knowledge of which of the pairs $\binom{U}{2}$ are in E and which are not.

Our contribution: space lower bound. We show that, in sharp contrast to our algorithm for constructing a one-level expander decomposition, this problem requires $\widetilde{\Omega}(n/\phi)$ space, i.e. a dependence on $1/\phi$ is unavoidable. Formally, we obtain the following result.

Theorem 3 (RED lower bound). Let $\ell \geq 2$ and let $\epsilon, \phi \in (0, 1)$ such that $\epsilon = 1 - \Omega(1)$, $\phi \leq \epsilon$, and $\phi \geq C \cdot \max\{\epsilon^2, 1/n\}$ for a large enough constant $C > 0$. Any streaming algorithm that with probability at least $9/10$ computes an ℓ-level (ϵ, ϕ)-RED requires $\Omega(n/\epsilon)$ bits of space.

The above theorem gives an $\widetilde{\Omega}(n/\phi)$ space lower bound for algorithms that compute a RED with near-optimal parameters, i.e. algorithms that achieve $\epsilon = \tilde{O}(\phi)$ for any $1/n \ll \phi \ll 1/\log n$.

Setup and hard instances. Throughout this section, the symbols $\ll$ and $\gg$ mean smaller or larger by a large constant factor. Let us fix the RED parameters $\epsilon, \phi \in (0, 1)$, and let us restrain ourselves to the regime $\phi \gg \epsilon^2$ (and of course $\phi \ll \epsilon$). We prove the lower bound by giving a distribution over hard instances $G = (V, E)$. This distribution is parametrised by integers d and m such that $1 \ll d \ll m \ll n$, $m \gg 1/\phi$, $m \ll 1/\epsilon^2$, and $d \ll \frac{1}{\epsilon}$. With these parameters fixed, our hard distribution $\mathcal{G}$ is defined below in Definition 2.3. An illustration is given in Figure 1.

Definition 2.3 (Distributions $\mathcal{G}$ and $\mathcal{G}'$ – Informal, see Definition 7.2). We partition V arbitrarily into two sets S and T with $n/2$ vertices each, and further partition S into n/m sets $S_1, \ldots, S_{n/m}$ with $m/2$ vertices each. The edge set of the graph $G = (V, E) \sim \mathcal{G}$ is defined as follows.

1. For each $i \in [n/m]$, the induced subgraph $G[S_i]$ is an Erdős-Rényi random graph with $m/2$ vertices and degree $\approx d$. We denote by $\mathcal{G}'$ the distribution of the subgraph $G[S]$.
2. The induced subgraph $G[T]$ is a fixed d-regular $\Omega(1)$-expander.
3. We fix for convenience an arbitrary labelling $s_{i,1}, \ldots, s_{i,m/2}$ of the vertices in each S_i, and we sample an index K uniformly from $[m/2]$. Then, for every $i \in [n/m]$, we add $dm/2$ edges from $s_{i,K}$ to T so that each $t \in T$ has d incident edges connecting to S.

Roughly speaking, our hard instances should be composed of n/m regular expanders that are densely connected to T, which is also an expander, through a selection of "special" vertices. We will show that the hardness arises from recovering information about certain important vertices and edges, defined below and also illustrated in Figure 1.

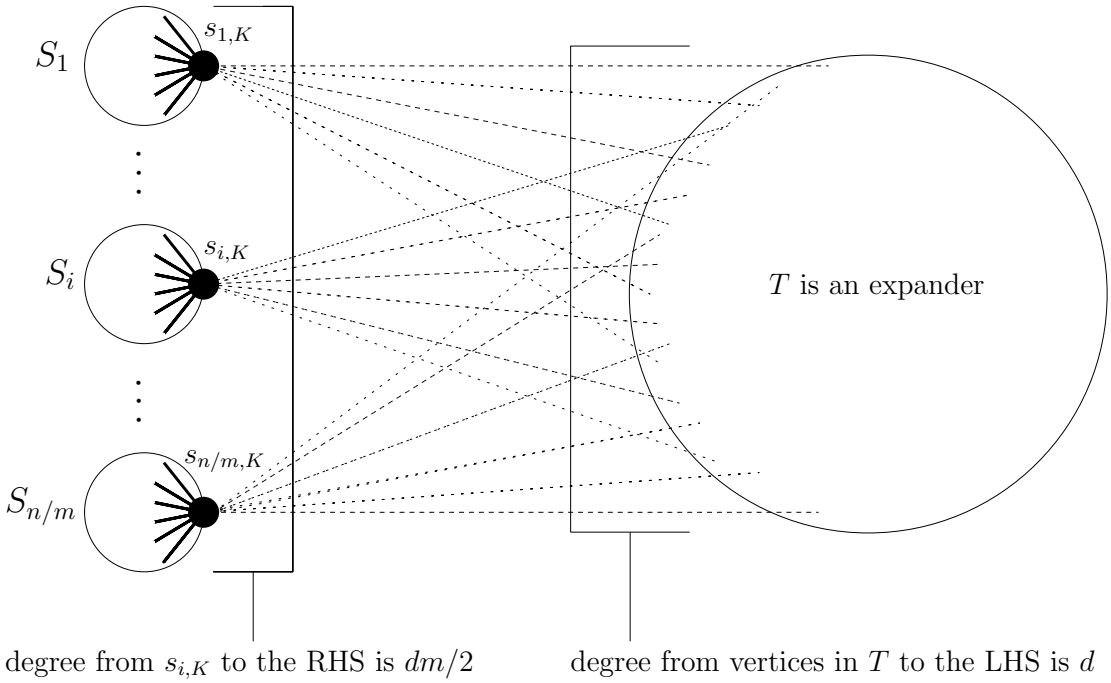


Figure 1: Illustration of the graph we use for proving the lower bound. Thick bullets represent important vertices, thick lines represent important edges, dotted lines represent edges connecting the important vertices to T.

Definition 2.4 (Important vertices and edges – Informal, see Definition 7.3). Let $G = (V, E) \sim \mathcal{G}$. We define the set of important vertices $V^* = \{s_{i,K} : i \in [n/m]\}$ to be the set of vertices of S that are connected to T, and define the set of important edges $E^* = \{\{s_{i,K}, v\} : i \in [n/m],\, v \in S\}$ to be the set of edges in the induced subgraph $G[S]$ that are incident on V^*.

The lower bound proof has two steps: we first prove that a two-level RED leaks a non-trivial amount of information about the graph $G \sim \mathcal{G}$; then we prove that, in order to obtain such amount of information, the algorithm must use a lot of space.

Two-level expander decomposition of the hard instance

In this section, we show that any valid two-level RED reveals a lot of informations about the important edges. The following lemma shows that a non-trivial amount of important edges are inter-cluster edges in the first level decomposition. An ideal decomposition is illustrated in Figure 2.

Lemma 2.5 (Informal, see Lemma 7.4). Let $G = (V, E) \in \mathrm{supp}(\mathcal{G})$. Then, any (ϵ, ϕ)-ED $\mathcal{U}$ of G satisfies

$$|E^* \setminus \mathcal{U}| \geq \frac{4}{5} \cdot |E^*| \ .$$

Proof sketch. By definition of the graph, there are $\Theta(dn)$ edges in the graph. Since there is at most an ϵ fraction of crossing edges, there are only $O(\epsilon dn)$ crossing edges. Note that there are $\Theta(dn)$

edges in $G[T]$, so only an $O(\epsilon)$ fraction of the edges in $G[T]$ are crossing edges. Furthermore, $G[T]$ is a regular expander: this implies that there is a large cluster $U^* \in \mathcal{U}$ comprising a $1 - O(\epsilon)$ fraction of T, together with a $1 - O(\epsilon)$ fraction of important vertices. The latter is true since the edges between S and T make up a constant fraction of the total volume and only a small fraction of the edges can be crossing. We refer the reader to Claim 7.5 for more details.

The edges in the subgraph $G[S]$ also account for a constant fraction of the total volume. Together with the fact that each S_i induces a regular expander, for many of the S_i's we will have a cluster in $\mathcal{U}$ that contains most of S_i. Now consider a set S_i such that the important vertex of S_i is in U^* and most of the vertices in S_i are all in the same cluster. If most of the vertices in S_i are also inside U^*, then consider the cut from $(U^* \cap S_i) \setminus \{s_{i,K}\}$ to $(U^* \setminus S_i) \cup \{s_{i,K}\}$. The cut size is at most the number of important edges in S_i, which is $O(d)$. On the other hand, the volume of the cut is $\Theta(dm)$ since most of the vertices of S_i are in the cluster. Recalling that $m \gg \frac{1}{\phi}$, one concludes that the cut is sparse. Therefore, we ruled out the possibility of having many vertices of S_i in U^*. See Claim 7.6 for a detailed discussion.

In summary, as illustrated in Figure 2, in any valid expander decomposition, most of the vertices in T and most of the important vertices are inside a giant cluster U^*, and for most of the S_i's, there is a cluster other than U^* that contains most of the vertices in S_i. For any such S_i, most of the important edges inside it are then crossing edges. □

The second level expander decomposition, i.e. an expander decomposition of the inter-cluster edges from the first level, is also quite structured, as illustrated in the ideal RED of Figure 2.

Lemma 2.6 (Informal, see Claim 7.14). Let $G = (V, E) \in \text{supp}(\mathcal{G})$, and let $\mathcal{U}_1, \mathcal{U}_2$ be any 2-level (ϵ, ϕ)-RED of G. Then, there are at most $n/10$ vertices in S that are non-isolated vertices[3] in $\mathcal{U}_2$. Moreover, at least a $2/3$ fraction of important edges are not in $E \setminus \mathcal{U}_2$, i.e. a $2/3$ fraction of important edges are inside clusters of $\mathcal{U}_2$.

Proof sketch. The number of crossing edges in the first level decomposition is $O(\epsilon dn)$, which is much less than n since $d \ll \frac{1}{\epsilon}$. This means that most of the vertices are isolated vertices in the second level. Moreover, by Lemma 2.5, most of the important edges are crossing edges in the first level decomposition. Among these $\Theta(dn/m)$ edges, at most $O(\epsilon^2 dn)$ edges can be crossing edges in the second level decomposition. Recalling that $m \ll \frac{1}{\epsilon^2}$, we see that most of the important edges are not crossing edges in the second level decomposition. □

Lower bound via communication complexity

Our streaming lower bound will be proven in the two-player one-way communication model. In this setting, Alice gets the edges in $G[S]$ and $G[T]$, and Bob gets the edges between S and T. We prove that in order to give a two-level RED, Alice needs to send $\Omega(dn)$ bits of information to Bob.

The high level idea is the following. Note that the identity of the important vertices can be only revealed by edges given to Bob. Thus, given Alice's input, every vertex in S has the same probability to be an important vertex, which means that every edge in S has the same probability to be an important edge. Therefore, in order to make sure that Bob recovers most of the important edges (which is morally equivalent to computing a two-level RED, as suggested by Figure 2), Alice needs to send most of the edges in S to Bob, which is $\Omega(dn)$.

[3]We call a vertex v non-isolated in a decomposition $\mathcal{U}$ if $\mathcal{U}$ puts v in a cluster with other vertices, i.e. v does not constitute a singleton cluster in $\mathcal{U}$.

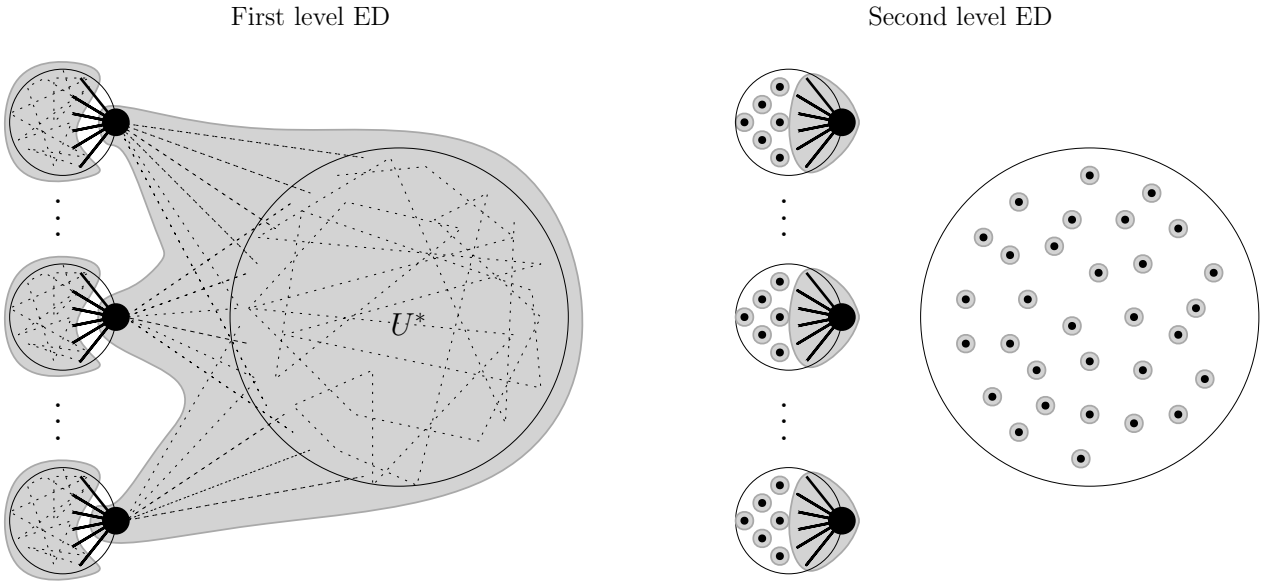


Figure 2: Illustration of the ideal expander decomposition of the graph. Thick bullets represent important vertices, smaller bullets represent ordinary vertices, thick lines represent important edges, dotted lines represent the rest of the edges. Grey areas represent the clusters in the decomposition.

To make the above idea concrete, we consider a communication problem where Alice is given a graph, and Bob is asked to output a not too large set of pairs that contains a good fraction of the edges of Alice's input. We first reduce this new problem to the two-level RED problem. Then, we will prove a communication complexity lower bound for this problem.

Definition 2.7 (Informal, see Definition 7.8). In the communication problem recover, Alice's input is a graph $G' = (S, E')$ where $|S| = n/2$, and Bob's output is a set of pairs of vertices $F \subseteq \binom{S}{2}$ that must satisfy $|F| \leq nm/10$ and $|F \cap E'| \geq \Omega(|E'|)$, i.e. at least a constant fraction of the edges in G' are in F.

Now, the idea is to plant Alice's input for recover into our instance from Definition 2.3. The input distribution for recover is sampled from the distribution $\mathcal{G}'$. In other words, the distribution of Alice's input is the same as the left-hand side part of $G \sim \mathcal{G}$ (i.e. the subgraph $G[S]$). Then, in the reduction, Alice gets the edges of $G[S]$ and $G[T]$ while Bob gets the edges between S and T. By virtue of the discussion in the previous section, we expect a RED of G to allow Bob to recover many important edges in $G[S]$. Hence, Bob could simulate the RED algorithm for all the $m/2$ possible choices of the random index $K \sim [m/2]$ that defines the important edges (see Definition 2.3): in this way, the k-th RED should reveal information about Alice's edges that are incident on the vertices $\{s_{i,k}\}_i$. Therefore, by varying k over $[m/2]$, Bob should obtain information about all the edges in Alice's graph. More precisely, the reduction is the following.

Reduction (Informal, see Reduction 7.11). Let $\mathcal{A}$ be a deterministic streaming algorithm for computing a 2-level (ϵ, ϕ)-RED. Alice, given her input graph $G' = (S, E')$ generated by $\mathcal{G}'$, feeds her edges E' to $\mathcal{A}$, together with the fixed edges of $G[T]$. Then, she sends the memory state of $\mathcal{A}$ to Bob. Upon receiving the message, Bob makes $m/2$ copies of $\mathcal{A}$ and initialises them to the memory state he received from Alice. Call these copies $\mathcal{A}_1, \ldots, \mathcal{A}_{m/2}$. Next, for each $k \in [m/2]$, call $G_k = (V, E_k)$ the graph we obtain from $\mathcal{G}$ when $K = k$ and the left-hand side $G_k[S]$ is exactly Alice's input G' (so that the vertices $\{s_{i,k}\}_i$ are the important vertices in G_k, see Definition 7.2). Then, Bob feeds

the edges $E_k(S,T)$ to $\mathcal{A}_k$. Let then $\mathcal{U}_1^k$,$\mathcal{U}_2^k$ be the RED output by $\mathcal{A}_k$. Bob finally constructs his output set F as follows: for each $k \in [m/2]$, add the pair $\{s, s_{i,k}\}$ to F for every $i \in [n/m]$ and every $s \in S_i$ that is not an isolated vertex in $\mathcal{U}_2^k$.

By Lemma 2.6, the number of non-isolated vertices in the second-level decomposition is at most $n/10$. Hence, the total number of pairs added to F is at most $nm/10$, thus satisfying the first requirement of Definition 2.7. Moreover, by Lemma 2.6, at least a $2/3$ fraction of the important edges are not crossing edges in the second level. This means that for each k, at least a $2/3$ fraction of the edges that are incident on $s_{i,k}$ is added to F. In turn, this implies that F contains at least a constant fraction of the edges in E', thus satisfying the second requirement of Definition 2.7. More precisely, one can prove the following.

Lemma 2.8 (Informal, see Lemma 7.15). If there is a deterministic L-bit space streaming algorithm $\mathcal{A}$ that computes a 2-level (ϵ, ϕ)-RED with constant probability over inputs $G \sim \mathcal{G}$, then there is a deterministic protocol $\mathcal{R}$ that solves recover with constant probability over inputs $G' \sim \mathcal{G}'$. The communication complexity of $\mathcal{R}$ is at most L.

The final component of the proof is the communication complexity lower bound for recover.

Lemma 2.9 (Informal, see Lemma 7.19). The one-way communication complexity of solving recover with constant probability over inputs sampled from $\mathcal{G}'$ is $\Omega(dn)$.

Proof sketch. Roughly speaking, we show that the posterior distribution of the input conditioned on the output is shifted away from its prior distribution.

Recall that when the input $G' = (S, E')$ is sampled from $\mathcal{G}'$, the graph is a disjoint union of n/m random graphs with $m/2$ vertices each and degree $\approx d$. There are roughly

$$\left(\binom{m/2}{d}^{m/2}\right)^{n/m} \approx \left(\frac{m}{2d}\right)^{dn/2}$$

possible inputs in total and the information complexity is $\Omega(dn)$. Conditioning on the the output F of a correct protocol for recover, the number of possible inputs is greatly decreased. In particular, to determine the input E', we need to select a constant fraction, say $2/3$ for example, of the pairs from F, and select the rest of the edges (a $1/3$ fraction, in our example) arbitrarily. Since $|F|$ is at most $nm/10$, $|E'| = \Theta(dn)$, and $\binom{S}{2} \approx nm/2$, the total number of possible inputs is then roughly

$$\binom{nm/10}{2/3 \cdot dn} \cdot \binom{nm/2}{1/3 \cdot dn} \approx \left(\frac{3m}{20d}\right)^{1/3 \cdot dn} \cdot \left(\frac{3m}{2d}\right)^{1/6 \cdot dn} < \left(\frac{m}{3d}\right)^{dn/2}.$$

This means the information complexity of the input is decreased by a constant factor, which means that the protocol needs to communicate $\Omega(dn)$ bits of information. □

Finally, one can conclude the main result (Theorem 3) combining Lemma 2.8 and Lemma 2.9. The formal proofs and definitions are deferred to Section 7.

3 Preliminaries

In this section, we introduce definitions and notation that we will use in the remainder of the paper.

Boundary-linked expander decomposition. In this paper we work heavily with boundary-linked EDs [GRST21]. A boundary-linked ED is the same as a classical ED except that Property (2) of Definition 1.1 is strengthened.

For a cut S in a cluster U, the boundary $\mathrm{bnd}_U(S)$ of S with respect to U is the number of edges that go from S to the outside of U, i.e. $\mathrm{bnd}_U(S) = |E(S, V \setminus U)|$. For $U \subseteq V$ and $\tau \geq 0$, the τ-boundary linked subgraph of G on U, denoted by $G[U]^\tau$, is the subgraph of G induced by U with additional $\tau \cdot \mathrm{bnd}_U(\{u\})$ self-loops attached to every $u \in U$. For cuts $S \subseteq U$, we adopt the following shorthand notation:

- the volume $\mathrm{vol}_{G[U]^\tau}(S)$ is denoted by $\mathrm{vol}^\tau_U(S)$;
- the sparsity $\Phi_{G[U]^\tau}(S)$ in $G[U]^\tau$ is denoted by $\Phi^\tau_U(S)$, which is equal to

$$\Phi^\tau_U(S) = \frac{\partial_U S}{\min\{\mathrm{vol}^\tau_U(S), \mathrm{vol}^\tau_U(U \setminus S)\}}\,.$$

Let $\phi \in (0,1), b \in [\phi, 1)$ be some parameters. Most of the time in this paper, the value of τ for all instances of boundary linked subgraphs will be the same and equal to the ratio b/ϕ. Hence we adopt additional shorthand notation: we use $G[U]^\circ$, $\mathrm{vol}^\circ_U(S)$, $\Phi^\circ_U(S)$, Φ°_U for referring to $G[U]^{b/\phi}$, $\mathrm{vol}^{b/\phi}_U(S)$, $\Phi^{b/\phi}_U(S)$, $\Phi^{b/\phi}_U$ respectively. One can observe that

$$\mathrm{vol}^\circ_U(S) = \mathrm{vol}^{b/\phi}_U(S) = \mathrm{vol}(S) + \left(\frac{b}{\phi} - 1\right)\mathrm{bnd}_U(S)\,.$$

Then a boundary-linked ED can be defined as follows.

Definition 3.1 (Boundary-linked expander decomposition [GRST21]). Let $G = (V, E)$, let $b, \epsilon, \phi \in (0,1)$ be parameters such that $b \geq \phi$, and let $\gamma \geq 1$ be an error parameter. A partition $\mathcal{U}$ of V is a $(b, \epsilon, \phi, \gamma)$-boundary-linked expander decomposition (for short, $(b, \epsilon, \phi, \gamma)$-BLD) of G if

1. $\frac{1}{2}\sum_{U \in \mathcal{U}} \partial U \leq \epsilon |E|$, and
2. for every $U \in \mathcal{U}$, $G[U]^\circ$ is a ϕ/γ-expander.

Expander decomposition sequence. For a partition $\mathcal{U}$ of V (think of $\mathcal{U}$ as an ED of G), we denote by $E \setminus \mathcal{U}$ the set of inter-cluster (or crossing) edges with respect to $\mathcal{U}$, i.e.

$$E \setminus \mathcal{U} = E \setminus \bigcup_{U \in \mathcal{U}} \binom{U}{2}\,.$$

Analogously, we let $G \setminus \mathcal{U} = (V, E \setminus \mathcal{U})$ be the subgraph of G obtained by removing the intra-cluster edges in $\mathcal{U}$. For a sequence of partitions $\mathcal{U}_1, \ldots, \mathcal{U}_\ell$ of V and $i \in [\ell]$, we define $G^{\mathrm{R}}_1 = G$ and denote by $G^{\mathrm{R}}_{i+1} = G^{\mathrm{R}}_i \setminus \mathcal{U}_i$ the subgraph of G obtained by removing the intra-cluster edges of the first i partitions.

Definition 3.2 (Removal-based ED sequence). Let $G = (V, E)$, let $\epsilon, \phi \in (0,1)$, let $\ell \geq 1$, and let $\mathcal{U}_1, \ldots, \mathcal{U}_\ell$ be a sequence of partitions of V. The sequence $\mathcal{U}_1, \ldots, \mathcal{U}_\ell$ is an ℓ-level removal-based (ϵ, ϕ)-ED sequence (for short, ℓ-level (ϵ, ϕ)-RED or (ϵ, ϕ, ℓ)-RED) of G if, for all $i \in [\ell]$, $\mathcal{U}_i$ is an (ϵ, ϕ)-ED of the graph G^{R}_i.

Sparsifier-specific notation. Since our algorithms will be working on sparsifiers of the input graph, it is convenient to have short-hand notation for the above quantities in the sparsifier. For a weighted subgraph $H = (V, E', w)$ of G, a cluster $U \subseteq V$ and a cut $S \subseteq U$, we have "sparsified" counterparts $\partial^w S$, $\mathrm{vol}^w(S)$, $\partial^w_U S, \mathrm{bnd}^w_U(S)$ for ∂S, $\mathrm{vol}(S)$, $\partial_U S, \mathrm{bnd}_U(S)$. More precisely, $\partial^w S = w(S, V \setminus S)$, $\partial^w_U S = w(S, U \setminus S)$, $\mathrm{bnd}^w_U(S) = w(S, V \setminus U)$, and $\mathrm{vol}^w(S)$ is the sum of weighted degrees of vertices of S in H. Then, $H[U]^\circ$ denotes the subgraph of H induced by U (retaining edge weights from w), where every vertex is attached $b/\phi \cdot \mathrm{bnd}^w_U(\{x\})$ self-loops. We then also introduce notation for the "sparsified" version of $\mathrm{vol}^\circ_U(S)$ and $\Phi^\circ_U(S)$: we denote $\mathrm{vol}_{H[U]^{b/\phi}}(S)$ and $\Phi_{H[U]^{b/\phi}}(S)$ by $\mathrm{vol}^{\circ w}_U(S)$ and $\Phi^{\circ w}_U(S)$ respectively. For clarity, we note that $\mathrm{vol}^{\circ w}_U(S) = \mathrm{vol}^w(S) + \frac{b-\phi}{\phi}\mathrm{bnd}^w_U(S)$, reflecting the fact that $\mathrm{vol}^\circ_U(S) = \mathrm{vol}(S) + \frac{b-\phi}{\phi}\mathrm{bnd}_U(S)$.

4 Sparsification for vertex-induced subgraphs

In order to give a space efficient implementation of Algorithm 1, we will take a subroutine computing an approximate most-balanced sparse cut of a graph, and run it as a black box on a vertex-induced subgraph of a sparsifier. From such sparsifier, we demand that the local cuts are approximated to within an additive error proportional the corresponding global cut. Formally, we rely on the following cut preserving property for vertex-induced subgraphs.

Definition 4.1 (Cluster sparsifier). Let $G = (V, E)$, let $\delta \in (0, 1)$, let $U \subseteq V$ be a cluster, and let $H = (V, E', w)$ be a weighted subgraph of G on the vertex set V. Then, H is a cluster sparsifier for U in G with error δ, denoted by $H[U] \approx_\delta G[U]$, if one has

$$\forall S \subseteq U,\ \partial_U S - \delta \cdot \partial S \leq \partial^w_U S \leq \partial_U S + \delta \cdot \partial S \quad \text{and} \quad \forall S \subseteq V,\ (1-\delta)\cdot \partial S \leq \partial^w S \leq (1+\delta)\cdot \partial S\,.$$

In Definition 4.1, the second property is the same as the first one when $U = V$. Therefore, if one is able to sample a sparsifier where the first property holds with high probability for any cluster $U \subseteq V$, then we get a sparsifier that with high probability embodies Definition 4.1. Perhaps surprisingly, we show that classical constructions of cut sparsifiers give the first property of Definition 4.1. Specifically, we use the approach of Fung et al. [FHHP11], based on sampling edges with probability proportional to their edge connectivity, together with its dynamic stream implementation by Ahn, Guha, and McGregor [AGM12b]. Hence, we obtain the following sparsification lemma.

Lemma 4.2 (Cluster sparsifiers in dynamic streams). Let $G = (V, E)$ be an n-vertex graph and let $\delta \in (0, 1)$. Then, there is a distribution $\mathcal{D}_\delta$ such that, for any cluster $U \subseteq V$ and a sample $H = (V, E', w) \sim \mathcal{D}_\delta$ one has $H[U] \approx_\delta G[U]$ with high probability, and $|E'| = \tilde{O}(n/\delta^2)$ edges. When G is given in a dynamic stream, there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n/\delta^2)$ space and decodes it to output a weighted subgraph $H = (V, E', w)$ in $\tilde{O}(n/\delta^2)$ space and $\mathrm{poly}(n)$ time such that $H \sim \mathcal{D}_\delta$.

The distribution $\mathcal{D}_\delta$ we use is a cosmetic modification of the algorithm of Ahn, Guha, and McGregor [AGM12b], hereafter referred to as the AGM algorithm. It provides a dynamic stream implementation of the sampling scheme of Fung et al. [FHHP11] for constructing cut sparsifiers. Even though we deviate only slightly from the original analysis of Ahn, Guha, and McGregor and Fung et al., we give a complete proof for the sake completeness. In Section 4.1, we give a few preliminaries on sparsification and dynamic streams that are needed to present and discuss the algorithm. In Section 4.2, we outline the algorithm and prove the technical lemmas that characterise its correctness. In Section 4.3, we use standard cut counting arguments to combine such technical lemmas into proving Lemma 4.2.

4.1 Sparsification primer

We give a few preliminaries needed to present the AGM algorithm. We start with the following result: for a graph given in a dynamic stream, we can output a subgraph that preserves the distinction between k-edge connected and not k-edge connected cuts.

Theorem 4.3 (Connectivity witness [AGM12a]). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $k \geq 1$ be an integer parameter. Then there is an algorithm ConnWit(k) that maintains a linear sketch of G in $\tilde{O}(kn)$ space. With high probability, the algorithm decodes the sketch to output a subgraph $G' = (V, E')$ in $\tilde{O}(kn)$ space and poly(n) time such that:

1. for all $\emptyset \neq S \subsetneq V$, if $|E(S, V \setminus S)| < k$ then $E(S, V \setminus S) \subseteq E'$;
2. for all $\emptyset \neq S \subsetneq V$, if $|E(S, V \setminus S)| \geq k$ then $|E(S, V \setminus S) \cap E'| \geq k$;
3. $|E'| = O(kn)$.

Proof. The algorithm works by maintaining k independent linear sketches for dynamic spanning forest [AGM12a], each taking $\tilde{O}(n)$ space to maintain and $\tilde{O}(n)$ space and poly(n) time to decode. We use the first sketch to recover a spanning forest of G. Then, we feed edge removals for every edge in the spanning forest to the second to k-th sketch. We proceed in this fashion until we have recovered k edge-disjoint maximal spanning forest $F_1, \ldots, F_k \subseteq E$ of G. Then we let E' be the union of these spanning forests. This takes $k \cdot \tilde{O}(n)$ space and one has $|E'| = O(kn)$.

Now consider a cut $\emptyset \neq S \subsetneq V$. Clearly $E'(S, V \setminus S) \subseteq E(S, V \setminus S)$. Now suppose $E(S, V \setminus S) < k$. For the sake of a contradiction, let us say $E(S, V \setminus S) \setminus E' \neq \emptyset$. By disjointness of the F_i's, there must be a $j \in [k]$ such that the residual graph $G \setminus (\cup_{i=1}^{j-1} F_i)$ still contains an edge from $E(S, V \setminus S)$. Hence, there are two components of the spanning forest F_j which could be joint by this edge, which contradicts its maximality.

Consider the case of $E(S, V \setminus S) \geq k$, and suppose for the sake of a contradiction that $E'(S, V \setminus S) < k$. By disjointness of the F_i's there must be a forest F_j such that $F_j \cap E(S, V \setminus S) = \emptyset$ (as otherwise $E'(S, V \setminus S) \geq k$). As before, this contradicts the maximality of F_j. □

Hereafter let $C > 0$ be a large enough constant, and define for every $e = \{u, v\} \in \binom{V}{2}$ the ideal sampling probabilities

$$p_e = \min \left\{ 1, \frac{1}{\lambda_e} \cdot \frac{C \log^3 n}{\delta^2} \right\},$$

where λ_e is the edge connectivity of e, i.e. the minimum number of edges crossing a uv-cut in G. The original AGM algorithm estimates edge connectivities (and hence sampling probabilities) via the sparsifier that is being constructed, in some sense. Here, we simplify the analysis and instead use a separate sparsifier for that purpose. For instance, one can use a spectral sparsifier, defined as follows.

Definition 4.4 (Spectral sparsifier). Let $G = (V, E)$ and let $\xi \in (0, 1)$. A $(1 \pm \xi)$-spectral sparsifier of G is a weighted subgraph $\tilde{G} = (V, \tilde{E}, \tilde{w})$ such that

$$\forall \mathrm{x} \in \mathbb{R}^V, \quad (1 - \xi)\mathrm{x}^\top L \mathrm{x} \leq \mathrm{x}^\top \tilde{L} \mathrm{x} \leq (1 + \xi)\mathrm{x}^\top L \mathrm{x},$$

where L and $\tilde{L}$ are the Laplacian matrices of G and $\tilde{G}$, respectively. The Laplacian matrix of a graph is $L = D - A$, where D denotes the degree diagonal matrix of the graph and A denotes its (weighted) adjacency matrix

It is known how to construct such sparsifiers in dynamic streams.

Theorem 4.5 ([KMM+20]). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $\xi \in (0, 1)$. Then, there is an algorithm Spectral(ξ) that maintains a linear sketch of G in $\tilde{O}(n/\xi^2)$ space. With high probability, the algorithm decodes the sketch to output in $\tilde{O}(n/\epsilon^2)$ time and space a weighted subgraph $\tilde{G} = (V, \tilde{E}, \tilde{w})$ with $|\tilde{E}| = \tilde{O}(n/\epsilon^2)$ that is a $(1 \pm \xi)$-spectral sparsifier of G.

4.2 Sampling algorithm for dynamic streams

With all the preliminaries in place, we can describe (our version of) the AGM algorithm. The high level approach consists in sampling edges at geometric rates as they arrive in the stream, and then use the i-th connectivity witness to recover the edges e that are sampled at rate $p_e \approx 2^i$. Algorithm 2 outlines this process, where we denote by $\tilde{\lambda}_e$ the edge connectivity of $e \in \binom{V}{2}$ in $\tilde{G}$. As one can see from the description of the algorithm, we actually let w be a function $E \to \mathbb{R}_{\geq 0}$ where $w(e) = 0$ for all $e \in E \setminus E'$. This facilitates the discussion.

We remark that Algorithm 2 assumes access to poly(n) bits of randomness. It is known that one can lift this assumption by using Nisan's pseudorandom generator [Nis92, Ind06] with only a polylog(n) space blow-up factor (and a poly(n) time blow-up). We remark that we can do this because the random bits are only used during the stream processing, and we only compute linear sketches in this phase, which in particular are oblivious to the order of updates in the stream. Then we can already conclude a bound on the space complexity, as well as on the size of the output graph.

Lemma 4.6. Algorithm 2 takes $\tilde{O}(n/\delta^2)$ bits of space and H has at most $\tilde{O}(n/\delta^2)$ edges.

Proof. The edges of H can only come from the output of one of the instances ConnWit. In other words, $E' \subseteq \cup_i E'_i$. From Theorem 4.3 we also know that each instance of ConnWit takes $\tilde{O}(kn)$ bits of space and that each E'_j contains at most $\tilde{O}(kn)$ edges. Hence, $|E'| = \tilde{O}(kn) = \tilde{O}(n/\delta^2)$ and Algorithm 2 takes $\tilde{O}(kn) = \tilde{O}(n/\delta^2)$ bits of space. □

We continue our analysis by bounding the probability that a certain bad event happens, which corresponds to the sampling or sketching primitives failing to preserve a needed value. To be more specific about this, first define for every $i = 0, \ldots, 2 \log n$ the event $\mathcal{B}_i$ that instance i of Theorem 4.3 ConnWit$_i(k)$ fails to correctly construct the graph G'_i. In addition, for every $e \in \binom{V}{2}$, let $\mathcal{B}_e^{(1)}$ be the event that $\tilde{\lambda}_e < 1/2 \cdot \lambda_e$ or $\tilde{\lambda}_e > 3/2 \cdot \lambda_e$. Finally, define for every $e \in E$ the quantity

$$\tau_e = \left\lfloor \log \frac{1}{\min\{1, p_e\}} \right\rfloor ,$$

and let $\mathcal{B}_e^{(2)}$ be the event that there exists $j \in \mathbb{N}$ with $\tau_e - 2 \leq j \leq \tau_e$ such that $\lambda_e(G_j) \geq k$, where k is defined in Algorithm 2 and $\lambda_e(G_i)$ is the edge connectivity of e in G_i for $i = 0, \ldots, 2 \log n$. With this notation, the bad event we want to avert is

$$\mathcal{B} = \left(\vee_{i=0}^{2 \log n} \mathcal{B}_i \right) \vee \left(\vee_{e \in \binom{V}{2}} \mathcal{B}_e^{(1)} \right) \vee \left(\vee_{e \in E} \mathcal{B}_e^{(2)} \right).$$

Intuitively, this is a bad event because the algorithm cannot trust the connectivity witnesses nor the edge connectivities computed via the spectral sparsifiers. On the contrary, when $\mathcal{B}$ does not happen, we know that all the graphs G'_i are constructed correctly (thanks to $\neg\mathcal{B}_i$), all the edges have j_e that is not too large (thanks to $\neg\mathcal{B}_e^{(1)}$), and we can recover the sampled edges via the graphs G'_i (thanks to $\neg\mathcal{B}_e^{(2)}$).

Lemma 4.7. When running Algorithm 2, the bad event $\mathcal{B}$ does not happen with high probability.

Algorithm 2 agm: sparsifier construction in dynamic streams

▷ $\delta \in (0,1)$ is the error parameter
▷ $C > 0$ is a large enough constant
▷ $k = 16C \cdot \delta^{-2} \cdot \log^3 n$

procedure PreProcessing
    Spectral ← instance of Spectral(1/2) from Theorem 4.5
    for $i = 1, \ldots, 2\log n$ do
        $h_i \sim \text{UNIF}\left(\{0,1\}^{\binom{V}{2}}\right)$ ▷ sample $2\log n$ independent uniform hash functions
        $\text{ConnWit}_i(k) \leftarrow$ i-th independent instance of ConnWit(k) from Theorem 4.3
    end for
end procedure

procedure OnUpdate(e) ▷ $e \in \binom{V}{2}$
    feed the update for e to Spectral
    for $i = 0, \ldots, 2\log n$ do
        if $\prod_{j=1}^{i} h_j(e) = 1$ then
            feed the update for e to $\text{ConnWit}_i(k)$
        end if
    end for
end procedure

procedure PostProcessing
    $\tilde{G} = (V, \tilde{E}, \tilde{w}) \leftarrow$ result of Spectral
    for $i = 0, \ldots, 2\log n$ do
        $G_i = (V, E_i) \leftarrow (V, \{e \in E : \prod_{j=1}^{i} h_j(e) = 1\})$ ▷ used for convenience of analysis only
        $G'_i = (V, E'_i) \leftarrow$ result of $\text{ConnWit}_i(k)$ ▷ used by the algorithm
    end for
    for $e \in \binom{V}{2}$ do
        $\tilde{p}_e \leftarrow \min\left\{1, \frac{2}{\tilde{\lambda}_e} \cdot \frac{C\log^3 n}{\delta^2}\right\}$
        $j_e \leftarrow \left\lfloor \log \frac{1}{\min\{1, \tilde{p}_e\}} \right\rfloor$.
        if $e \in E'_{j_e}$ then
            $w(e) \leftarrow 2^{j_e}$
        else
            $w(e) \leftarrow 0$
        end if
    end for
    $E' \leftarrow \text{supp}(w)$
    return $H = (V, E', w)$.
end procedure

Proof. By Theorem 4.5, we know that $\vee_e \mathcal{B}_e^{(1)}$ does not happen with high probability for any $e \in \binom{V}{2}$. Now let $i = 0, \ldots, 2\log n$, and note that the randomness used to define the input G_i to $\mathrm{ConnWit}_i(k)$ and the randomness used by $\mathrm{ConnWit}_i(k)$ itself are independent. Hence, by Theorem 4.3 and by union bound over i, we get that $\vee_i \mathcal{B}_i$ does not happen with high probability. Lastly, we deal with the events $\mathcal{B}_e^{(2)}$.

For ease of notation, define for any $\emptyset \neq S \subseteq V$ the size of its cut projected onto the edges of G_i as $\partial^i S = |E_i \cap E(S, V \setminus S)|$. Then consider any $e = \{s, t\} \in E$ and any $j \in \mathbb{N}$ with $\tau_e - 2 \leq j \leq \tau_e$, and let $S_e \subseteq V$ be an st-cut in G of minimum size, i.e. $\lambda_e = \partial S_e$. Observe that by the way the algorithm defines the sub-streams, each $f \in E$ is in E_i with probability 2^{-i} for all $i = 0, \ldots, 2\log n$, so in expectation the edge connectivity of e has dropped below $4C \cdot \delta^{-2} \cdot \log^3 n$ in the graph G_j. More precisely,

$$\begin{aligned}
\mathbb{E}[\partial^j S_e] = \partial S_e \cdot 2^{-j} &\leq 4 \cdot \partial S_e \cdot 2^{-\tau_e} \\
&\leq 8\lambda_e \min\{1, p_e\} \\
&\leq 8\lambda_e p_e \\
&\leq 8\lambda_e \frac{C \cdot \delta^{-2} \cdot \log n}{\lambda_e} = 8C \cdot \delta^{-2} \cdot \log^3 n \,.
\end{aligned}$$

Applying the Chernoff bound (since edges are sampled independently of each other) we get that

$$\begin{aligned}
\Pr\left[\partial^j S_e > 12C \cdot \delta^{-2} \cdot \log^3 n\right] &\leq \Pr\left[\partial^j S_e > \left(1 + \frac{1}{2}\right) \mathbb{E}[\partial^j S_e]\right] \\
&\leq \exp\left(-\frac{\mathbb{E}[\partial^j S_e]}{12}\right) \\
&\leq \exp\left(-\frac{\mathbb{E}[\partial^{\tau_e} S_e]}{12}\right) .
\end{aligned}$$

In order to make sure that the exponential above is in fact small enough, we now have two cases to consider. First consider the possibility that $\tau_e = 0$, or equivalently $p_e > \frac{1}{2}$. This means two things: $E_{\tau_e} = E$ (see Algorithm 2), and $\lambda_e < 2C \cdot \delta^{-2} \cdot \log^3 n$. Then, $\partial^{\tau_e} S_e = \lambda_e \leq 12C \cdot \delta^{-2} \cdot \log^3 n$ with probability 1, so we need not worry about this case. For the other possibility that $p_e \leq \frac{1}{2}$, one can deduce that $p_e = C \cdot \lambda_e^{-1} \delta^{-2} \cdot \log^3 n$ and thus lower bound the expectation of $\partial^{\tau_e} S_e$ as

$$\mathbb{E}[\partial^{\tau_e} S_e] = \partial S_e \cdot 2^{-\tau_e} \geq \lambda_e p_e = C \cdot \delta^{-2} \cdot \log^3 n \,,$$

Hence, the Chernoff bound above gives $\partial^{\tau_e} S_e \leq 12C \cdot \delta^{-2} \cdot \log^3 n < k$ with high probability. Thus, the edge connectivity of e in G_j must be at most $\partial^j S_e < k$. Taking a union bound gives that $\vee_e \mathcal{B}_e^{(2)}$ does not happen with high probability. □

Finally, we show properties of the weight function produced by the algorithm. We first focus on the special case of edges with very low connectivity, i.e. with $p_e = 1$, and show that these are indeed copied exactly in E', as one would expect.

Lemma 4.8. Let $H = (V, E', w)$ be the output of Algorithm 2. If $\mathcal{B}$ does not happen, then for any edge $e \in E$ with $p_e = 1$ we have $e \in E'$ and $w(e) = 1$.

Proof. Assume $\mathcal{B}$ does not happen. This in particular means that G'_0 is correctly constructed, so by Theorem 4.3 any cut of size less than $16C \cdot \delta^{-2} \cdot \log^3$ is exactly preserved in G'_0. On the other hand, if an edge $e \in E$ has $p_e = 1$ then its edge connectivity λ_e is at most $C \cdot \delta^{-2} \cdot \log^3 n$. Since $\tilde{\lambda}_e \leq 2\lambda_e$, it then follows that $\tilde{p}_e = 1$, so $j_e = 0$ and hence the claim. □

For edges which we do not know to have $p_e = 1$, but for which we at least know a lower bound p on their sampling probability, we wish to show that they are preserved with "very high" probability. We do this by using a Chernoff-style bound of Ahn, Guha, and McGregor.

Lemma 4.9. Let $H = (V, E', w)$ be the output of Algorithm 2. Then for any edge set $X \subseteq E$, any $p \leq \min_{e \in X} p_e$, and any $x \geq |X|$ one has

$$\Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x\right] \leq 2\exp\left(-\frac{1}{48} \cdot \delta^2 \cdot px\right).$$

Proof. For ease of notation, let $\tilde{\lambda}$ be the vector of edge connectivities $(\tilde{\lambda}_e)_{e \in E}$ from Algorithm 2, and λ is the vector of edge connectivities $(\lambda_e)_{e \in E}$ of G. We will condition on the values $\hat{\lambda}$ that $\tilde{\lambda}$ can take, and define

$$\hat{p}_e = \min\left\{1, \frac{2}{\hat{\lambda}_e} \cdot \frac{C \log^3 n}{\delta^2}\right\} \quad \text{and} \quad \hat{j}_e = \left\lfloor \log \frac{1}{\min\{1, \hat{p}_e\}} \right\rfloor.$$

Then,

$$\begin{aligned}
&\Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x\right] \\
&= \sum_{\hat{\lambda} \in \mathbb{N}^E} \Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x \text{ and } \tilde{\lambda} = \hat{\lambda}\right] \\
&= \sum_{\hat{\lambda} \in \mathbb{N}^E : \frac{1}{2}\lambda \leq \hat{\lambda} \leq \frac{3}{2}\lambda} \Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x \text{ and } \tilde{\lambda} = \hat{\lambda}\right].
\end{aligned}$$

By conditioning we get

$$\begin{aligned}
&\Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x\right] \\
&= \sum_{\hat{\lambda} \in \mathbb{N}^E : \frac{1}{2}\lambda \leq \hat{\lambda} \leq \frac{3}{2}\lambda} \Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x \middle| \tilde{\lambda} = \hat{\lambda}\right] \cdot \Pr\left[\tilde{\lambda} = \hat{\lambda}\right]
\end{aligned}$$

We now analyse the first factor in each term of the sum above. We use three facts to do this: (1) whenever we have $\neg\mathcal{B}$, for all $e \in X$ one has that $e \in E'_{j_e}$ if and only if $e \in E_{j_e}$ (this is the case because $\frac{1}{2}\lambda \leq \tilde{\lambda} \leq \frac{3}{2}\lambda$, so $\max\{0, \tau_e - 2\} \leq j_e \leq \tau_e$, and the event $\mathcal{B}_e^{(2)}$ does not happen); (2) recall that, for $i = 0, \ldots, 2\log n$, $e \in E_i$ if and only if $\prod_{t=1}^{i} h_t(e) = 1$ (see Algorithm 2); (3) also recall that Algorithm 2 assigns weight 2^{j_e} to an edge recovered from E'_{j_e} (see Algorithm 2). Using these three facts, we get

$$\Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} w(e) - |X|\right| > \delta \cdot x \middle| \tilde{\lambda} = \hat{\lambda}\right] = \Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e \in X} 2^{j_e} \prod_{t=1}^{j_e} h_t(e) - |X|\right| > \delta \cdot x \middle| \tilde{\lambda} = \hat{\lambda}\right].$$

Next, by the condition $\tilde{\lambda} = \hat{\lambda}$, we have $\hat{j}_e = j_e$ for all $e \in X$, so we get

$$\begin{aligned}\Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e\in X} w(e) - |X|\right| > \delta\cdot x \middle| \tilde{\lambda}=\hat{\lambda}\right] &= \Pr\left[\neg\mathcal{B} \text{ and } \left|\sum_{e\in X} 2^{\hat{j}_e}\prod_{t=1}^{\hat{j}_e} h_t(e) - |X|\right| > \delta\cdot x \middle| \tilde{\lambda}=\hat{\lambda}\right] \\ &\le \Pr\left[\left|\sum_{e\in X} 2^{\hat{j}_e}\prod_{t=1}^{\hat{j}_e} h_t(e) - |X|\right| > \delta\cdot x \middle| \tilde{\lambda}=\hat{\lambda}\right] \\ &= \Pr\left[\left|\sum_{e\in X} 2^{\hat{j}_e}\prod_{t=1}^{\hat{j}_e} h_t(e) - |X|\right| > \delta\cdot x\right] .\end{aligned}$$

Now we apply the following form of the Chernoff bound.

Fact 4.10 ([FKM23]). Let $X_1, \dots, X_N$ be independent random variables distributed in $[0, a]$. Let $X = \sum_{r\in[N]} X_r$ and $\mu = \mathbb{E}[X]$. Then, for $\zeta \in (0,1)$ and $\alpha \ge 0$ one has

$$\Pr\left[|X-\mu| > \zeta\mu + \alpha\right] \le 2\exp\left(-\frac{\zeta\alpha}{3a}\right)$$

Before applying this bound, we note that the random variables $2^{\hat{j}_e}\prod_{t=1}^{\hat{j}_e} h_t(e)$ are independent across $e \in X$ and have each mean 1. Moreover, each of this random variables have value at most $2^{\hat{j}_e} \le 2^{\lfloor \log \frac{1}{\min\{1,p\}}\rfloor}$, so we set $a = 1/p$. We also set $\zeta = \delta/2$ and $\alpha = \delta\cdot(x - |X|) + \delta|X|/2$. Then we get

$$\Pr\left[\left|\sum_{e\in X} 2^{\hat{j}_e}\prod_{t=1}^{\hat{j}_e} h_t(e) - |X|\right| > \delta\cdot x\right] \le 2\exp\left(-\frac{\delta^2}{48}px\right) .$$

This concludes the proof. □

4.3 Cut counting and proving the sparsification lemma

We hereafter fix a cluster $U \subseteq V$, and we partition the edge set E into $F_i = \{e \in E : 2^i \le \lambda_e \le 2^{i+1} - 1\}$ for $i = 0, \dots, \log n$. Note that these F_i's are pairwise disjoint and their union gives E. We recall that $E(A, B)$ denotes the set of edges in E with one endpoint in A and one in B, for $A, B \subseteq V$. Then, we write $\partial^i_U S = |F_i \cap E(S, U \setminus S)|$ to denote the size of the projection of the local cut of S onto the edge set F_i, for all $i = 0, \dots, \log n$ and $\emptyset \ne S \subsetneq U$. Analogously, we also write $\partial^{i\,w}_U S = w(F_i \cap E(S, U \setminus S))$ to denote the total weight of the projection of the local cut of S onto F_i, where w is the weight function of H.

We show that H preserves the cuts of a cluster $U \subseteq V$ within each F_i individually, with high probability. This follows by two of the technical lemmas we proved in the previous section, namely Lemma 4.8 and Lemma 4.9.

Lemma 4.11. Let $H = (V, E', w)$ be the output of Algorithm 2, and let $i = 0, \dots, \log n$. Then

$$\Pr\left[\neg B \text{ and } \exists\, \emptyset \ne S \subsetneq U : \left|\partial^{i\,w}_U S - \partial^i_U S\right| > \delta\cdot\frac{\partial S}{1+\log n}\right] \le \frac{1}{\mathrm{poly}(n)} .$$

Proof. We proceed by first showing a probability bound for each cut S, and then combine them using a union bound to get the final result. Specifically, for all $j \in [2\log n]$ we define the set

$$\mathcal{Y}_{ij} = \{E(S, V\setminus S) : \emptyset \ne S \subsetneq U \text{ and } 2^{i+j-1} \le \partial S \le 2^{i+j} - 1\} ,$$

and we will show a probability bound for all $j \in [2\log n]$ and $Y \in \mathcal{Y}_{ij}$. Taking a careful union bound over all such choices will be enough to get the lemma statement. It shall be useful to define $\Pi^i_U(Y) = Y \cap F_i \cap \binom{U}{2}$ to be the projection onto F_i and onto the cluster U of the edges in Y, for any $Y \subseteq E$. Moreover, we denote by $\mathcal{F}_Y$ the failure event that

$$\left|w(\Pi^i_U(Y)) - |\Pi^i_U(Y)|\right| > \delta \frac{|Y|}{1+\log n}\,,$$

For ease of writing we shall denote $\Pi^i_U(E(S, V\setminus S))$ and $\mathcal{F}_{E(S,V\setminus S)}$ by $\Pi^i_U(\partial S)$ and $\mathcal{F}_{\partial S}$ respectively. To appreciate their purpose, we remark that for any $\emptyset \neq S \subsetneq U$,

$$\mathcal{F}_{\partial S} \quad \iff \quad \left|\partial^{i\,w}_U S - \partial^i_U S\right| > \delta \frac{\partial S}{1+\log n}$$

since $\Pi^i_U(\partial S) = \Pi^i_U(E(S, V\setminus S)) = E(S, V\setminus S) \cap F_i \cap \binom{U}{2} = F_i \cap E(S, U \setminus S)$.

Claim 4.12. Let $j \in [2\log n]$ and $Y \in \mathcal{Y}_{ij}$. Then

$$\Pr[\neg\mathcal{B} \text{ and } \mathcal{F}_Y] \leq 2n^{-\frac{C}{768}2^j}\,.$$

Proof. We start by observing that we can consider only the set $X = \{e \in \Pi^i_U(Y) \,:\, p_e < 1\}$, since edges with $p_e = 1$ are always added to E' with weight set to 1 (when $\mathcal{B}$ does not occur) by Lemma 4.8, so

$$\left|w(\Pi^i_U(Y)) - |\Pi^i_U(Y)|\right| = |w(X) - |X||\,.$$

If $X = \emptyset$, the probability of $\mathcal{F}_Y$ is zero, so let us consider the case that $X \neq \emptyset$, for which we want to upper bound the probability

$$\Pr\left[\neg\mathcal{B} \text{ and } \; |w(X) - |X|| > \delta \frac{|Y|}{1+\log n}\right]\,.$$

To do so, we use Lemma 4.9 and set $p = \min\{1, C\cdot 2^{-(i+1)}\delta^{-2}\log^3 n\}$, which is a lower bound on all the probabilities $\{p_e\}_{e\in X}$. This is the case because every $e \in X$ has $p_e < 1$ and $\lambda_e \leq 2^{i+1}$ (since $X \subseteq F_i$). We also note that $|Y| \geq |X|$, thus we can safely apply Lemma 4.9 and get

$$\Pr\left[\neg\mathcal{B} \text{ and } \; |w(X) - |X|| > \delta \frac{|Y|}{1+\log n}\right] \leq 2\exp\left(-\frac{1}{48}\cdot\delta^2\cdot p\cdot\frac{|Y|}{(1+\log n)^2}\right)\,.$$

If we recall $Y \in \mathcal{Y}_{ij}$, and in particular $|Y| \geq 2^{i+j-1}$, we can then conclude the claim by plugging the value of p into the exponent. □

Now that we have bounded the probability that a single cut from $\mathcal{Y}_{ij}$ is not well preserved once projected onto U and F_i, we show that the union over j of the collections $\mathcal{Y}_{ij}$ is representative of all the local cuts of U.

Claim 4.13. Let $\emptyset \neq S \subsetneq U$ such that $\Pi^i_U(\partial S) \neq \emptyset$. Then there exists $j \in [2\log n]$ so that $E(S, V\setminus S) \in \mathcal{Y}_{ij}$.

Proof. Since $\Pi^i_U(\partial S) \neq \emptyset$, there exists $e = \{u, v\} \in E(S, V\setminus S)$ that is also in F_i. In particular this means that $E(S, V\setminus S)$ is a uv-cut in G, and e has edge connectivity at least 2^i, thus $\partial S \geq 2^i$. At the same time, $\partial S \leq 2^{i+j} - 1$ for some $j \in [2\log n]$ since $\partial S \leq |E| \leq n^2 - 1$. □

Now, for all $j \in [2\log n]$ and all $Z \subseteq F_i$, we define the worst-case edge set $Y_j^*(Z) \subseteq E$ of Z as

$$Y_j^*(Z) \in \operatorname*{argmin}_{Y \in \mathcal{Y}_{ij} : \Pi_U^i(Y) = Z} |Y| \,.$$

The reason why we call $Y_j^*(Z)$ worst case for Z is the following. Among all the global cuts in $\mathcal{Y}_{ij}$ whose projection onto F_i and U is Z, the cut $Y_j^*(Z)$ is the one which allows for the smallest error term, which intuitively makes $\mathcal{F}_Y$ more likely to happen. Formally we have the following claim, which follows from the definition.

Claim 4.14. If there exist $j \in [2\log n]$ and $Y \in \mathcal{Y}_{ij}$ such that the event $\mathcal{F}_Y$ happens, then there exist $j \in [2\log n]$ and $Z \in \{\Pi_U^i(Y') : Y' \in \mathcal{Y}_{ij}\}$ such that $\mathcal{F}_{Y_j^*(Z)}$ happens.

Claim 4.13 and 4.14 together suggest that we can restrain ourselves to union bound over $\{\Pi_U^i(Y) : Y \in \mathcal{Y}_{ij}\}$, which is exactly what we are going to do. Before that, we make sure that this set is not too large.

Claim 4.15. For every $j \in [2\log n]$ we have $|\{\Pi_U^i(Y) : Y \in \mathcal{Y}_{ij}\}| \leq n^{2\cdot 2^j}$.

Proof. We will employ the following cut counting result, where E^k denotes the set of edges $e \in E$ with edge connectivity $\lambda_e \geq k$, for any integer k.

Theorem 4.16 ([FHHP11] Theorem 1.6). For any $k, s \geq 1$ one has

$$|\{E(S, V \setminus S) \cap E^k \,:\, \emptyset \neq S \subsetneq V \text{ and } \partial S \leq s \cdot k\}| \leq n^{2s} \,,$$

i.e. the number of distinct subsets of edges of connectivity at least k from cuts of size at most $s \cdot k$ is at most n^{2s}.

Fix $j \in [2\log n]$. We recall that every $e \in F_i$ has $\lambda_e \geq 2^i$, which means $F_i \subseteq E^{2^i}$. Also, for any $Y \in \mathcal{Y}_{ij}$ there is by definition a cut $\emptyset \neq S \subsetneq V$ such that $\partial S \leq 2^{i+j} - 1$ and $Y = E(S, V \setminus S)$. Hence

$$|\{\Pi_U^i(Y) : Y \in \mathcal{Y}_{ij}\}| \leq |\{E^{2^i} \cap E(S, V \setminus S) : \emptyset \neq S \subsetneq V \text{ and } \partial S \leq 2^{i+j}\}| \,,$$

and applying Theorem 4.16 with $k \coloneqq 2^i$ and $s \coloneqq 2^j$ we conclude the claim. □

Using the above claims we finally have

$$\begin{aligned}
& \Pr[\neg\mathcal{B} \text{ and } \exists\, \emptyset \neq S \subsetneq U \text{ s.t. } \mathcal{F}_{\partial S}] \\
\text{by Claim 4.13} \quad & \leq \Pr[\exists j \in [2\log n] \text{ s.t. } \exists Y \in \mathcal{Y}_{ij} \text{ s.t. } \neg\mathcal{B} \text{ and } \mathcal{F}_Y] \\
\text{by Claim 4.14} \quad & \leq \Pr[\exists j \in [2\log n] \text{ s.t. } \exists Z \in \{\Pi_U^i(Y) : Y \in \mathcal{Y}_{ij}\} \text{ s.t. } \neg\mathcal{B} \text{ and } \mathcal{F}_{Y_j^*(Z)}] \\
\text{by Claim 4.12} \quad & \leq \sum_{j \in [2\log n]} \sum_{Z \in \{\Pi_U^i(Y) : Y \in \mathcal{Y}_{ij}\}} 2n^{-\frac{C}{768}2^j} \\
\text{by Claim 4.15} \quad & \leq \sum_{j \in [2\log n]} n^{2\cdot 2^j}\, 2n^{-\frac{C}{768}2^j} \\
& \leq \frac{1}{\operatorname{poly}(n)}
\end{aligned}$$

□

Finally, we can conclude the main sparsification lemma Lemma 4.2 by a simple combination of Lemma 4.7, Lemma 4.11, and Lemma 4.6.

Proof of Lemma 4.2. Recall that the F_i's partition and cover E entirely, and that $\neg\mathcal{B}$ happens with high probability by Lemma 4.7. Then, one can see that conditioning on $\neg\mathcal{B}$, summing over $i = 0, \ldots, \log n$, and applying Lemma 4.11 gives Lemma 4.2. □

5 Testing expansion and finding sparse cuts in sparsifiers

The key ingredient to our ED algorithms is that sparsifiers as per Definition 4.1 serve as good proxies to the subgraphs of G for any subroutine that either asserts expansion or outputs an approximate most-balanced sparse cut. We will in fact show that the result of running such subroutine on a boundary-linked subgraph of G is almost the same as if we run it on the corresponding subgraph of a sparsifier.

Before we proceed further, we need to define what we demand exactly from the approximate most-balanced sparse cut subroutine. Loosely speaking, it is a procedure that takes as input a graph and a sparsity parameter, and either asserts that the graph is an expander, or outputs a sparse cut. In particular, the returned cut should be not too small compared to the largest cut among those certifying that the input is not an expander. For technical reasons, we ask that the algorithm also returns an approximation to the volume of the returned cut. Inspired by the definition of [FKM23], we have the following definition, which captures what the subroutine should return.

Definition 5.1 (Balanced sparse cut witness). Let $\psi, \xi \in [0,1]$ and $\alpha, \lambda \geq 1$, and let $H = (U, E', w)$ be a weighted graph with self-loops. A $(\psi, \xi, \alpha, \lambda)$-balanced sparse cut witness of H (for short, $(\psi, \alpha, \lambda, \xi)$-BSCW) is an element ω of $\{\perp\} \cup (\{R : \emptyset \neq R \subsetneq U\} \times \mathbb{R}_{>0})$ such that:

1. if $\omega = \perp$, then H is a ψ-expander;
2. if $\omega = (R, \nu)$, then:
   (a) $\Phi_H(R) < \alpha \cdot \psi$,
   (b) for every other cut $\emptyset \neq T \subsetneq U$ with $\Phi_H(T) < \psi$ and $\mathrm{vol}_H(T) \leq \mathrm{vol}_H(U \setminus T)$ one has $\mathrm{vol}_H(T) \leq \lambda \cdot \mathrm{vol}_H(R)$,
   (c) $\mathrm{vol}_H(R) \leq (1+\xi)\,\mathrm{vol}_H(U \setminus R)$,
   (d) $(1-\xi) \cdot \mathrm{vol}^\circ_U(R) \leq \nu \leq (1+\xi) \cdot \mathrm{vol}^\circ_U(R)$.

A balanced sparse cut subroutine is then simply an algorithm returning a balanced sparse cut witness. For convenience of later discussions, we also have the following definition.

Definition 5.2 (Balanced sparse cut algorithm). Let $\alpha, \lambda \geq 1$. An algorithm $\mathrm{BalSparseCut}_{\alpha,\lambda}$ is an (α, λ)-balanced sparse cut algorithm (for short, (α, λ)-BSCA) if, taken as input a weighted graph with self-loops $H = (U, E', w)$ and a parameter $\psi \in (0,1)$, $\mathrm{BalSparseCut}_{\alpha,\lambda}(H, \psi)$ outputs a $(\psi, \alpha, \lambda, 0)$-BSCW of H.

Ultimately, we will use either an exponential time brute force BSCA, or a fast BSCA with worse parameters, leading to our exponential and polynomial time ED algorithms, respectively.

Using these definitions, the main result of this section is the following: one can almost seamlessly run a BSCA on the sparsifier, without heavily deteriorating the quality of the BSCW hence obtained.

Lemma 5.3 (Proxying lemma). Let $b, \phi, c, \delta \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$, $b \leq c$, $\delta \leq c^2 b / \log n$, and let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA with $\alpha \leq \frac{1}{2b}$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Let ω be the output of $\text{BalSparseCut}_{\alpha,\lambda}(H[U]^\circ, (1 + \frac{1}{2\log n}) \cdot \phi)$. Then, ω is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U]^\circ$.

In Section 5.1 and Section 5.2 we show properties that will let us compare cuts and volumes in $H[U]^\circ$ and $G[U]^\circ$. In Section 5.3 we use these tools to prove Lemma 5.3.

5.1 Characterizing boundary-linked cuts

Structurally, we will divide the cuts of a subgraph $G[U]^\circ$ into two groups: those those that are boundary-linked and those that are not. This concept[4]., which we introduce next, is key to our goal: it captures the cuts that we are able to sparsify to within multiplicative error. Loosely speaking, a cut of $G[U]^\circ$ is boundary-linked if its local cut is a decent fraction of its global cut.

Definition 5.4 (Boundary-linked cuts). Let $\beta \in (0,1)$ be a parameter, let $U \subseteq V$ be a cluster, and let $S \subseteq U$ be a cut. We say that S is β-boundary-linked in U if

$$\partial_U S \geq \beta \cdot \min\{\partial S, \partial(U \setminus S)\}.$$

We shall then group the cuts of $G[U]^\circ$ into boundary-linked ones and those that are not. As shown by the two lemmas below, each of these groups has a useful properties for our goal: for a correct set of parameters, boundary-linked cuts are very well approximated by a sparsifier, while those that are not boundary-linked must be sparse.

Lemma 5.5 (Characterization of boundary-linked cuts). Let $b, \phi, c, \delta \in (0,1)$ such that $c \leq 1/2$ is a constant, $\phi \leq b$, $b \leq c$, and $\delta \leq c^2 b / \log n$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Then, the following hold:

1. for all non $b/2$-boundary-linked cuts $\emptyset \neq S \subsetneq U$ we have $\Phi^\circ_U(S) < \phi$, and
2. for all $b/2$-boundary-linked cuts $S \subseteq U$ we have $(1 - \frac{c}{\log n}) \cdot \partial_U S \leq \partial^w_U S \leq (1 + \frac{c}{\log n}) \cdot \partial_U S$.

Proof. To show (1), let $\emptyset \neq S \subsetneq U$ be a cut with $\Phi^\circ_U(S) \geq \phi$. By definition this means that $\partial_U S \geq \phi \min\{\text{vol}^\circ_U(S), \text{vol}^\circ_S(U \setminus S)\}$. Since $\partial_U S = \partial_U(U \setminus S)$, we can in fact assume without loss of generality that $\partial_U S \geq \phi \, \text{vol}^\circ_U(S)$. Recalling how these volumes are defined, we have that every vertex in $u \in S$ has a contribution of at least $b\phi \cdot \text{bnd}_U(\{u\})$ to $\text{vol}^\circ_U(S)$, so $\text{vol}^\circ_U(S) \geq b/\phi \cdot \text{bnd}_U(S)$. Hence, $\partial_U S \geq \phi \, \text{vol}^\circ_U(S)$ yields $\partial_U S \geq b \cdot \text{bnd}_U(S)$. Finally, we observe that the global cut of S, i.e. ∂S, can be decomposed as the sum of $\partial_U S$ and $\text{bnd}_U(S)$. By virtue of this decomposition, from $\partial_U S \geq b \cdot \text{bnd}_U(S)$ we get $\partial_U S \geq b \cdot (\partial S - \partial_U S)$. Rearranging the terms, and since we assume that b is bounded by $c \leq 1/2$, we conclude $\partial_U S \geq b/2 \cdot \partial S$, i.e. S is $b/2$-boundary-linked.

To show (2), from definition of boundary-linked we know $\partial_U S \geq b/2 \cdot \min\{\partial S, \partial(U \setminus S)\}$, while from the assumption that $H[U] \approx_\delta G[U]$ we know $|\partial_U S - \partial^w_U S| \leq \delta \cdot \partial S$. Again, without loss of generality we can assume $\partial_U S \geq b/2 \cdot \partial S$, since $\partial_U S = \partial_U(U \setminus S)$ and $\partial^w_U S = \partial^w_U(U \setminus S)$. Therefore, the sparsification error $|\partial_U S - \partial^w_U S|$ can actually be bounded by $\delta \cdot \partial S \leq c^2 b / \log n \cdot 2/b \cdot \partial_U S \leq c / \log n \cdot \partial_U S$ (since $c \leq 1/2$). □

[4]The notion of boundary-linked cuts was already used in previous works, but not in the context of sparsification and streaming [GRST21, Li21]

5.2 Preserving volumes and sparsities

Lemma 5.5 says that for any cut S in U, either S is sparse, or the sparsifier gives a multiplicative approximation to its local cut. Intuitively, for the latter cuts, one would expect the sparsifier to preserve their sparsity as well. However, this is not trivially true, because the sparsity depends on both the local cut and the volume in the boundary-linked subgraph: while we have a multiplicative preservation of boundary-linked cuts, we may not have the same guarantee for their volume. To circumvent this issue, we consider the case of sparse and expanding cuts separately: the former have in fact their volumes preserved within a relative error, while for the latter we get that their volume in the sparsifier is still not too large. We combine these observations by treating separately boundary-linked and non boundary-linked cuts, and conclude that the distinction between sparse and expanding is very accurately preserved in a sparsifier. We begin with analysing the volume of sparse cuts in the sparsifier.

Lemma 5.6 (Multiplicative error for volumes of sparse cuts). Let $b, \phi, \hat{\phi}, c, \delta \in (0,1)$ such that $c \leq 1/5$ is a constant, $\phi < b$, $\delta \leq c^2/\log n$, and $\hat{\phi} \leq \phi/b$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Then, for all cuts $\emptyset \neq S \subsetneq U$ with $\Phi^\circ_U(S) < \hat{\phi}$ we have

$$\left(1 - \frac{c}{\log n}\right) \cdot \mathrm{vol}^\circ_U(S) \leq \mathrm{vol}^{\circ\, w}_U(S) \leq \left(1 + \frac{c}{\log n}\right) \cdot \mathrm{vol}^\circ_U(S)\,.$$

Proof. By definition of $\mathrm{vol}^{\circ\, w}_U(S)$, the error in estimating the volume can be written as

$$|\mathrm{vol}^{\circ\, w}_U(S) - \mathrm{vol}^\circ_U(S)| \leq |\mathrm{vol}^w(S) - \mathrm{vol}_U(S)| + \frac{b-\phi}{\phi} \left|\mathrm{bnd}^w_U(S) - \mathrm{bnd}_U(S)\right|\,. \tag{5.1}$$

The first term is easily upper-bounded by $\delta \cdot \mathrm{vol}(S)$, because of the global cut preservation property of $H[U] \approx_\delta G[U]$ (see Definition 4.1) applied to singleton cuts. For the second term, we can further split it as

$$|\mathrm{bnd}^w_U(S) - \mathrm{bnd}_U(S)| \leq |\partial^w S - \partial S| + |\partial^w_U S - \partial_U S|\,, \tag{5.2}$$

We now upper-bound the right hand side using again the cut preservation properties from Definition 4.1: we employ the multiplicative error for the global cut ∂S, and the additive error for $\partial_U S$. Combining these with the assumption that S is sparse, one gets

$$\begin{aligned}
\left|\partial^w S - \partial S\right| + \left|\partial^w_U S - \partial_U S\right| &\leq 2\delta \cdot \partial S && (5.3)\\
&= 2\delta \cdot \big(\partial_U S + \mathrm{bnd}_U(S)\big) && (5.4)\\
\text{since } \Phi^\circ_U(S) < \hat{\phi} \qquad &< 2\delta \cdot \hat{\phi} \cdot \mathrm{vol}^\circ_U(S) + 2\delta \cdot \mathrm{bnd}_U(S) && (5.5)\\
\text{since } \hat{\phi} \leq \phi/b \text{ and } \mathrm{vol}^\circ_U(S) \geq \frac{\phi}{b}\mathrm{bnd}_U(S) \qquad &\leq 2\delta \cdot \frac{\phi}{b} \cdot \mathrm{vol}^\circ_U(S) + 2\delta \cdot \frac{\phi}{b} \cdot \mathrm{vol}^\circ_U(S) && (5.6)\\
&\leq 4\delta \cdot \frac{\phi}{b-\phi} \cdot \mathrm{vol}^\circ_U(S)\,, && (5.7)
\end{aligned}$$

We now backtrack and plug (5.7) into (5.2), and then (5.2) into (5.1) to conclude the claim:

$$\begin{aligned}
|\mathrm{vol}_U^{\circ w}(S) - \mathrm{vol}_U^{\circ}(S)| &\leq |\mathrm{vol}^w(S) - \mathrm{vol}_U(S)| + \frac{b-\phi}{\phi}\,|\mathrm{bnd}_U^w(S) - \mathrm{bnd}_U(S)| \\
&\leq \delta \cdot \mathrm{vol}(S) + 4\delta \cdot \mathrm{vol}_U^{\circ}(S) \\
&\leq 5\delta \cdot \mathrm{vol}_U^{\circ}(S) \\
&\leq \frac{5c^2}{\log n} \cdot \mathrm{vol}_U^{\circ}(S) \\
&\leq \frac{c}{\log n} \cdot \mathrm{vol}_U^{\circ}(S)\,,
\end{aligned}$$

since $c \leq 1/5$. □

Using this fact together with the cut preservation properties of a cluster sparsifier, we can already conclude that there cannot be cuts that look "very" expanding in $H[U]^\circ$ but are actually sparse in $G[U]^\circ$. In particular, if the sparsity of a cut in the sparsifier is above ϕ by a factor that dominates the error of the sparsifier, then its sparsity must be at least ϕ in the original graph. We show this in Lemma 5.7.

Lemma 5.7 (Very expanding cuts in a sparsifier are indeed expanding). Let $b, \phi, \hat{\phi}, c, \delta \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$, $b \leq c$, $\delta \leq c^2 b/\log n$, and $\hat{\phi} \geq (1 + \frac{1}{3\log n}) \cdot \phi$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Then, for all cuts $\emptyset \neq S \subsetneq U$ with $\Phi_U^{\circ w}(S) \geq \hat{\phi}$ we have $\Phi_U^{\circ}(S) \geq \phi$.

Proof. For the sake of a contradiction, let $\emptyset \neq S \subsetneq U$ be a cut with $\Phi_U^{\circ w}(S) \geq \hat{\phi}$ such that $\Phi_U^{\circ}(S) < \phi$.

Consider first the case that S is $b/2$-boundary-linked. Then we know from $H[U] \approx_\delta G[U]$ (see Definition 4.1) that $\partial_U^w S = (1 \pm c/\log n)\partial_U S$. Also, as we are assuming $\Phi_U^{\circ}(S) < \phi$, from Lemma 5.6 (our parameters verify stronger conditions than what is demanded by the lemma, so it applies) we know $\mathrm{vol}_U^{\circ w}(S) = (1 \pm c/\log n)\,\mathrm{vol}_U^{\circ}(S)$. These two together give $\Phi_U^{\circ w}(S) < (1 + 10c/\log n)\phi < \hat{\phi}$ for small enough c, a contradiction.

Next consider the case that S is not $b/2$-boundary-linked. Then we do not have a multiplicative approximation to $\partial_U^w S$. However, S is ϕ-sparse in $G[U]^\circ$ so we still have $\mathrm{vol}_U^{\circ w}(S) = (1 \pm c/\log n)\,\mathrm{vol}_U^{\circ}(S)$ and $\mathrm{vol}_U^{\circ w}(U \setminus S) = (1 \pm c/\log n)\,\mathrm{vol}_U^{\circ}(U \setminus S)$ by Lemma 5.6. Therefore, because $\hat{\phi} \geq (1 + \frac{1}{3\log n})\phi$ we get

$$\begin{aligned}
\partial_U^w S &\geq \left(1 - \frac{c}{\log n}\right)\left(1 + \frac{1}{3\log n}\right) \cdot \phi \cdot \min\{\mathrm{vol}_U^{\circ}(S), \mathrm{vol}_U^{\circ}(U \setminus S)\} \\
&\geq \left(1 + \frac{1}{10\log n}\right) \cdot \phi \cdot \min\{\mathrm{vol}_U^{\circ}(S), \mathrm{vol}_U^{\circ}(U \setminus S)\}\,,
\end{aligned}$$

and at the same time the additive error approximation from Definition 4.1 is enough to yield

$$\partial_U^w S \leq \partial_U S + \delta \cdot \partial S < \phi \cdot \min\{\mathrm{vol}_U^{\circ}(S), \mathrm{vol}_U^{\circ}(U \setminus S)\} + \delta \cdot \partial S\,.$$

Therefore,

$$\begin{aligned}
\partial S &\geq \frac{1}{\delta \cdot 10\log n} \cdot \phi \cdot \min\{\mathrm{vol}_U^{\circ}(S), \mathrm{vol}_U^{\circ}(U \setminus S)\} \\
\text{since } \delta \leq c^2 b/\log n \qquad &\geq \frac{1}{10c^2} \cdot \min\{\mathrm{bnd}_U(S), \mathrm{bnd}_U(U \setminus S)\}\,.
\end{aligned}$$

Now let us assume $\mathrm{bnd}_U(S) \le \mathrm{bnd}_U(U \setminus S)$ without loss of generality. Then the above is essentially saying that ∂S is much larger than $\mathrm{bnd}_U(S)$. Recalling now that we can decompose $\partial S = \partial_U S + \mathrm{bnd}_U(S)$, the above lower bound is actually saying that much of the advantage that ∂S has over $\mathrm{bnd}_U(S)$ must come from $\partial_U S$. More precisely, since $c \le 1/30$ we get

$$\partial_U S \ge \frac{1}{c} \cdot \partial S > \frac{b}{2} \cdot \partial S \,,$$

which contradicts the assumption that S is not $b/2$-boundary-linked. □

Next, we handle the case of expanding cuts, and show that their volume does not get overshot too much. Even though this is a much weaker guarantee than the one we obtained in Lemma 5.6 for sparse cuts, it will still be enough for our purposes: this will later be useful in concluding that their sparsity has not dropped by a lot.

Lemma 5.8 (Upper bound for volumes of expanding cuts). Let $b, \phi, \hat{\phi}, c, \delta \in (0,1)$ such that $c \le 1/2$ is a constant, $\phi \le b$, $\delta \le c^2/\log n$, and $\hat{\phi} \le \phi/b$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Then, for all cuts $\emptyset \ne S \subsetneq U$ with $\Phi^\circ_U(S) \ge \hat{\phi}$ we have

$$\min\{\mathrm{vol}^{\circ\, w}_U(S), \mathrm{vol}^{\circ\, w}_U(U \setminus S)\} \le \frac{1 + \frac{c}{\log n}}{\hat{\phi}} \cdot \partial_U S \,.$$

Proof. Let $S^* \in \{S, U \setminus S\}$ be the side of the cut achieving the minimum $\min\{\mathrm{vol}^\circ_U(S), \mathrm{vol}^\circ_U(U \setminus S)\}$. Even though $\mathrm{bnd}^w_U(S^*)$ may not be a very good approximation of $\mathrm{bnd}_U(S^*)$, we can upper-bound it by decomposing $\partial^w S^* = \partial^w_U S + \mathrm{bnd}^w_U(S)$ and using the local cut preservation property of $H[U] \approx_\delta G[U]$ (see Definition 4.1): we get

$$\mathrm{bnd}^w_U(S^*) = \partial^w S^* - \partial^w_U S^* \le \mathrm{bnd}_U(S^*) + \delta \cdot \partial S^* \,.$$

Using the above bound, together with the global approximation from Definition 4.1 applied to singleton cuts, we can upper-bound $\mathrm{vol}^{\circ\, w}_U(S^*)$ as

$$\begin{aligned}
\mathrm{vol}^{\circ\, w}_U(S^*) &= \mathrm{vol}^w(S) + \frac{b-\phi}{\phi}\,\mathrm{bnd}^w_U(S) \\
&\le (1+\delta)\cdot \mathrm{vol}(S^*) + \frac{b-\phi}{\phi}\,\mathrm{bnd}_U(S^*) + \frac{b-\phi}{\phi}\cdot\delta\cdot\partial S^* \\
\text{since } \partial S^* = \partial_U S + \mathrm{bnd}_U(S) \quad &\le (1+\delta)\cdot \mathrm{vol}(S^*) + (1+\delta)\cdot\frac{b-\phi}{\phi}\cdot\mathrm{bnd}_U(S^*) + \frac{b-\phi}{\phi}\cdot\delta\cdot\partial_U S^* \\
\text{by definition of } \mathrm{vol}^\circ_U(S) \quad &\le (1+\delta)\cdot \mathrm{vol}^\circ_U(S^*) + \frac{b-\phi}{\phi}\cdot\delta\cdot\partial_U S^* \\
\text{since } \hat{\phi} \le \frac{\phi}{b} \quad &\le (1+\delta)\cdot \mathrm{vol}^\circ_U(S^*) + \delta\cdot\frac{1}{\hat{\phi}}\cdot\partial_U S^* \,.
\end{aligned}$$

By our assumption that $\Phi^{b/\phi}_U(S) \ge \hat{\phi}$, we also know $\mathrm{vol}^{b/\phi}_U(S^*) \le \frac{1}{\hat{\phi}}\partial_U S$. We then conclude

$$\mathrm{vol}^{\circ\, w}_U(S^*) \le (1+2\delta)\cdot\frac{1}{\hat{\phi}}\partial_U S \le \left(1 + \frac{c}{\log n}\right)\frac{1}{\hat{\phi}}\partial_U S \,,$$

since $c \le 1/2$. □

Having a bound on the volumes of expanding cuts, allows to bound the sparsity of those cuts in the sparsifier (since all such cuts are boundary-linked, and hence the local cut is well preserved). This is almost the reverse of Lemma 5.7.

Lemma 5.9 (Expanding cuts are almost expanding in the sparsifier). Let $b, \phi, \hat{\phi}, c, \delta \in (0,1)$ such that $c \leq 1/6$ is a constant, $\phi \leq b$, $b \leq c$, $\delta \leq c^2 b / \log n$, and $\phi \leq \hat{\phi} \leq \phi/b$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Then, for all cuts $\emptyset \neq S \subsetneq U$ with $\Phi^\circ_U(S) \geq \hat{\phi}$ we have $\Phi^{\circ\, w}_U(S) \geq (1 - \frac{1}{3 \log n})\hat{\phi}$.

Proof. First note that our parameter regime is strictly stronger than that of Lemma 5.5 and Lemma 5.8. Hence, by Lemma 5.5, we know that S is $b/2$-boundary-linked, since it is $\hat{\phi}$-expanding and $\hat{\phi} \geq \phi$. Lemma 5.5 then also guarantees that $\partial^w_U S \geq (1 - \frac{c}{\log n})\partial_U S$. On the other hand, Lemma 5.8 gives $\min\{\mathrm{vol}^{\circ\, w}_U(S), \mathrm{vol}^{\circ\, w}_U(U \setminus S)\} \leq (1 + \frac{c}{\log n})/\hat{\phi} \cdot \partial_U S$. Hence,

$$\min\{\mathrm{vol}^{\circ\, w}_U(S), \mathrm{vol}^{\circ\, w}_U(U \setminus S)\} \leq \frac{1 + \frac{c}{\log n}}{1 - \frac{c}{\log n}} \frac{1}{\hat{\phi}} \cdot \partial^w_U S \leq \frac{1}{1 - \frac{1}{3 \log n}} \cdot \frac{1}{\hat{\phi}} \cdot \partial^w_U S\,,$$

since $c \leq 1/6$ is small enough. □

5.3 Proving the proxying lemma

From the previous section, we have that the distinction between sparse and expanding cuts of $G[U]^\circ$ is very accurately preserved by $H[U]^\circ$, and also the volume of sparse cuts of $G[U]^\circ$ is approximated to within a small relative error in $H[U]^\circ$. As these are exactly the quantities that a BSCA is interested in, we can then prove the main lemma of the section, restated here for convenience of the reader.

Lemma 5.3 (Proxying lemma). Let $b, \phi, c, \delta \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$, $b \leq c$, $\delta \leq c^2 b / \log n$, and let $\mathrm{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA with $\alpha \leq \frac{1}{2b}$. Fix a cluster $U \subseteq V$ and also let $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$. Let ω be the output of $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[U]^\circ, (1 + \frac{1}{2 \log n}) \cdot \phi)$. Then, ω is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U]^\circ$.

Proof. By definition of $\mathrm{BalSparseCut}_{\alpha,\lambda}$, we know that ω is a $((1 + \frac{1}{2 \log n}) \cdot \phi, \alpha, \lambda, 0)$-BSCW of $H[U]^\circ$, i.e.

1. if $\omega = \perp$, then $H[U]^\circ$ is a $(1 + \frac{1}{2 \log n}) \cdot \phi$-expander;
2. if $\omega = (R, \nu)$, then:
   (a) $\Phi^{\circ\, w}_U(R) < (1 + \frac{1}{2 \log n}) \cdot \alpha \cdot \phi$,
   (b) $\mathrm{vol}^{\circ\, w}_U(R) \leq \mathrm{vol}^{\circ\, w}_U(U \setminus R)$,
   (c) for every other cut $\emptyset \neq T \subsetneq U$ with $\Phi^{\circ\, w}_U(R) < (1 + \frac{1}{2 \log n}) \cdot \phi$ and $\mathrm{vol}^{\circ\, w}_U(T) \leq \mathrm{vol}^{\circ\, w}_U(U \setminus T)$ one has $\mathrm{vol}^{\circ\, w}_U(T) \leq B \, \mathrm{vol}^{\circ\, w}_U(R)$:
   (d) $\nu = \mathrm{vol}^{\circ\, w}_U(R)$.

Our goal is to translate these properties to $G[U]^\circ$ by showing that ω is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U]^\circ$, i.e.

1. if $\omega = \perp$, then $G[U]^\circ$ is a ϕ-expander;
2. if $\omega = (R, \nu)$, then:

(a) $\Phi^\circ_U(R) < (1 + \frac{1}{\log n}) \cdot \alpha \cdot \phi$,

(b) $\mathrm{vol}^\circ_U(R) \leq (1 + c)\,\mathrm{vol}^\circ_U(U \setminus R)$,

(c) for every other cut $\emptyset \neq T \subsetneq U$ with $\Phi^\circ_U(R) < \phi$ and $\mathrm{vol}^\circ_U(T) \leq \mathrm{vol}^\circ_U(U \setminus T)$ one has $\mathrm{vol}^\circ_U(T) \leq (1 + c)B\,\mathrm{vol}^\circ_U(R)$:

(d) $(1 - c) \cdot \mathrm{vol}^\circ_U(R) \leq \nu \leq (1 + c) \cdot \mathrm{vol}^\circ_U(R)$.

Correctness of the case $\omega = \perp$. The idea is that if the cluster U is non-trivially more than a ϕ-expander in the sparsifier, then it should be at least a ϕ-expander in G. Let us say that $\omega = \perp$, i.e. $\Phi^{\circ\,w}_U(S) \geq (1 + \frac{1}{2\log n}) \cdot \phi$ for all $\emptyset \neq S \subsetneq U$. Let $\hat{\phi} = (1 + \frac{1}{2\log n}) \cdot \phi$, so that we are in the parameter regime of Lemma 5.7. Then, this lemma ensures that $\Phi^\circ_U(S) \geq \phi$ for all $\emptyset \neq S \subsetneq U$. In other words, $G[U]^\circ$ is a ϕ-expander.

Correctness of the case $\omega = (R, \nu)$. Assume $\omega = (R, \nu)$. We need to show the four properties of a BSCW for $G[U]^\circ$.

- Property (2a). Intuitively, R cannot be much more expanding in G than what it looks like in H, so if it is sparse in H it is basically just as sparse in G. To prove this formally, it is convenient to distinguish the case of this cut being boundary-linked or not.

  If R is not $b/2$-boundary-linked, then Lemma 5.5 implies $\Phi^\circ_U(R) < \phi$. The property is then already proved in this case.

  If it is $b/2$-boundary-linked, suppose for the sake of a contradiction that R has $\Phi^\circ_U(R) \geq (1 + 1/\log n)\alpha \cdot \phi$. Let $\hat{\phi} = (1 + \frac{1}{\log n})\alpha \cdot \phi$. This value of $\hat{\phi}$ fulfils the condition that $\hat{\phi} \leq 1/b$ from Lemma 5.9, since $\alpha \leq 1/(2b)$ by assumption. We then meet the requirements to apply Lemma 5.9 (all other parameters also meet its preconditions, as our parameter regime for b, ϕ, c, ϕ is no weaker than its), so

  $$\Phi^{\circ\,w}_U(R) \geq \left(1 - \frac{1}{3\log n}\right) \cdot \left(1 + \frac{1}{\log n}\right) \cdot \alpha \cdot \phi \geq \left(1 + \frac{1}{2\log n}\right) \alpha \cdot \phi,$$

  which contradicts the definition of $\mathrm{BalSparseCut}_{\alpha,\lambda}$. We have then showed property (2a).

- Properties (2b) and (2d). From the previous point we know that $\Phi^\circ_U(R) < (1+\frac{1}{\log n})\alpha\cdot\phi$. Since our parameter regime is only stronger than that of Lemma 5.6 if we set $\hat{\phi} = (1 + \frac{1}{\log n})\alpha \cdot \phi$, we get $\mathrm{vol}^{\circ\,w}_U(R) = (1 \pm c/\log n)\,\mathrm{vol}^\circ_U(R)$ and $\mathrm{vol}^{\circ\,w}_U(U \setminus R) = (1 \pm c/\log n)\,\mathrm{vol}^\circ_U(U \setminus R)$. Recalling that $\mathrm{vol}^{\circ\,w}_U(R) \leq \mathrm{vol}^{\circ\,w}_U(U \setminus R)$ (see (2b) for $H[U]^\circ$), this yields

  $$\mathrm{vol}^\circ_U(R) \leq \frac{1}{1 - \frac{c}{\log n}}\,\mathrm{vol}^{\circ\,w}_U(R) \leq \frac{1}{1 - \frac{c}{\log n}}\,\mathrm{vol}^{\circ\,w}_U(U \setminus R) \leq \frac{1 + \frac{c}{\log n}}{1 - \frac{c}{\log n}}\,\mathrm{vol}^\circ_U(U \setminus R)\,,$$

  so $\mathrm{vol}^\circ_U(R) \leq (1 + c)\,\mathrm{vol}^\circ_U(U \setminus R)$. Recalling that $\nu = \mathrm{vol}^{\circ\,w}_U(R)$ (see (2d) for $H[U]^\circ$), one also has

  $$|\nu - \mathrm{vol}^\circ_U(R)| = |\mathrm{vol}^{\circ\,w}_U(R) - \mathrm{vol}^\circ_U(R)| \leq c/\log n \cdot \mathrm{vol}^\circ_U(R) \leq c\,\mathrm{vol}^\circ_U(R)\,.$$

- Property (2c). This property aims to bound the volume of R against the volume of every other ϕ-sparse cut. We want to exploit the bound that $\mathrm{BalSparseCut}_{\alpha,\lambda}$ gives for these cuts in H, i.e. that for every cut $\emptyset \neq T \subsetneq U$ with $\Phi^{\circ\,w}_U(R) < (1 + \frac{1}{2\log n})\phi$ and $\mathrm{vol}^{\circ\,w}_U(T) \leq \mathrm{vol}^{\circ\,w}_U(U \setminus T)$ one has $\mathrm{vol}^{\circ\,w}_U(T) \leq \lambda\,\mathrm{vol}^{\circ\,w}_U(R)$ (see (2c) for $H[U]^\circ$). Now one can see that if we prove all

such T to be also ϕ-sparse in $G[U]^\circ$, then we could conclude simply using the fact that the volumes of $H[U]^\circ$ are good proxies for those in $G[U]^\circ$. We now implement this strategy.

By virtue of property (2a), we have that $\Phi^\circ_U(R) < (1+\frac{1}{\log n})\alpha\cdot\phi$, and from an application of Lemma 5.6 we have $\mathrm{vol}^{\circ\,w}_U(R) = (1\pm c/\log n)\,\mathrm{vol}^\circ_U(R)$ (again, the preconditions of the lemma are met if we take $\hat{\phi} = (1+\frac{1}{\log n})\alpha\cdot\phi$).

Consider now any other cut $\emptyset \neq T \subsetneq U$ with $\Phi^\circ_U(T) < \phi$ and $\mathrm{vol}^\circ_U(T) \leq \mathrm{vol}^\circ_U(U\setminus T)$. Note that for any such T we have $\mathrm{vol}^{\circ\,w}_U(T) = (1\pm c/\log n)\,\mathrm{vol}^\circ_U(T)$ and $\mathrm{vol}^{\circ\,w}_U(U\setminus T) = (1\pm c/\log n)\,\mathrm{vol}^\circ_U(U\setminus T)$, again by applying Lemma 5.6 (setting $\hat{\phi} = \phi \leq \phi/b$, and all other parameters satisfy the needed preconditions).

Moreover, any such T is also $((1+\frac{1}{2\log n})\ \cdot\phi)$-sparse in $H[U]^\circ$, by applying Lemma 5.7 with parameter $\hat{\phi} = (1+\frac{1}{2\log n})\phi \geq (1+\frac{1}{3\log n})\phi$. From (2c) for $H[U]^\circ$, we then know

$$\min\{\mathrm{vol}^{\circ\,w}_U(T), \mathrm{vol}^{\circ\,w}_U(U\setminus T)\} \leq \lambda\,\mathrm{vol}^{\circ\,w}_U(R)\,.$$

Recall that $\mathrm{vol}^{\circ\,w}_U(R)$, $\mathrm{vol}^{\circ\,w}_U(T)$, $\mathrm{vol}^{\circ\,w}_U(U\setminus T)$ all are within a $(1\pm c/\log n)$ multiplicative error of $\mathrm{vol}^\circ_U(R)$, $\mathrm{vol}^\circ_U(T)$, $\mathrm{vol}^\circ_U(U\setminus T)$ respectively. Hence, $\min\{\mathrm{vol}^{\circ\,w}_U(T), \mathrm{vol}^{\circ\,w}_U(U\setminus T)\} \leq \lambda\,\mathrm{vol}^{\circ\,w}_U(R)$ translates to

$$\mathrm{vol}^\circ_U(T) = \min\{\mathrm{vol}^\circ_U(T), \mathrm{vol}^\circ_U(U\setminus T)\} \leq (1+10c/\log n)\lambda\cdot\mathrm{vol}^\circ_U(R) \leq (1+c)\lambda\cdot\mathrm{vol}^\circ_U(R)\,.$$

We have then proved (2c). □

6 Space efficient recursive partitioning

The main goal of this section is to show Theorems 2.1 and 2.2, restated here for convenience.

Theorem 2.1 (Exponential time decoding BLD). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $b \in (0,1)$ be a parameter such that $b \leq 1/\log^2 n$. Then, there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n/b^3)$ space. For any $\epsilon \in [n^{-2}, b\log n]$, the algorithm decodes the sketch to compute, with high probability and in $\tilde{O}(n/b^3)$ space and $2^{O(n)}$ time, a (b,ϵ,ϕ,γ)-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\log n}\right) \quad \text{and} \quad \gamma = O(1)\,.$$

Theorem 2.2 (Polynomial time decoding BLD). Let $G = (V, E)$ be a graph given in a dynamic stream, and let $b \in (0,1)$ be a parameter such that $b \leq 1/\log^5 n$. Then, there is an algorithm that maintains a linear sketch of G in $n/b^3 \cdot \log^{O(\log n/\log\frac{1}{b})} n$ space. For any $\epsilon \in [n^{-2}, b\log n]$, the algorithm decodes the sketch to compute, with high probability and in $n/b^3 \cdot \log^{O(\log n/\log\frac{1}{b})} n$ space and poly(n) time, a (b,ϵ,ϕ,γ)-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\log^4 n}\right) \quad \text{and} \quad \gamma = \log^{O\left(\frac{\log n}{\log 1/b}\right)} n\,.$$

The only difference between the algorithms of Theorem 2.1 and 2.2 lies in the balanced sparse cut subroutine that they employ. Therefore, we present and analyse them in a unified manner by deferring the choice of the balanced sparse cut algorithm to later. Using the results from Section 4 and Section 5 (i.e. Lemma 4.2 and Lemma 5.3 respectively), we prove the following.

Lemma 6.1 (Unified algorithm). Let $G = (V, E)$ be a graph given in a dynamic stream, let $b \in (0, 1)$ be a parameter such that $b \leq 1/\log n$. Let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA where $\alpha \leq \frac{1}{b \log n}$ and $\lambda = O(1)$. Also let k be an integer such that $k \geq \log(n^5\alpha)/\log(b^{-1/2}/\lambda)$, and $k \leq \log n$. Then there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n/b^3) \cdot \alpha^{O(k)}$ space. For all $\epsilon \in [n^{-2}, b \log n]$, the algorithm decodes the sketch using $\text{BalSparseCut}_{\alpha,\lambda}$ to compute, with high probability, a $(b, \epsilon, \phi, \gamma)$-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\alpha \log n}\right) \quad \text{and} \quad \gamma = O(\alpha^{k+1}).$$

Moreover, the decoding runs in $\tilde{O}(n/b^3) \cdot \alpha^{O(k)} + S(n, \tilde{O}(n/b^2) \cdot \alpha^{O(k)})$ space and $T(n, \tilde{O}(n/b^2) \cdot \alpha^{O(k)}) \cdot \text{poly}(n)$ time, where $S(p, q)$ and $T(p, q)$, denote the space and time complexity of $\text{BalSparseCut}_{\alpha,\lambda}$ on a graph with p vertices and q edges respectively. Furthermore, the decoding only makes calls to $\text{BalSparseCut}_{\alpha,\lambda}(\cdot, \psi)$ with sparsity parameters $\psi \in (0, 1)$ that satisfy $\psi \leq \frac{1}{10\alpha}$.

From Lemma 6.1 one readily obtains the theorems by either using a brute force $(1, 1)$-BSCA or a polynomial time $(O(\log^3 n), O(1))$-BSCA.

Proof of Theorem 2.1. It is easy to get an exponential time $(1, 1)$-BSCA as per Definition 5.2. Given a graph and a parameter $\psi \in (0, 1)$, one can do this by brute-forcing all possible cuts in the input graph: either return $\omega = \perp$ if all the cuts are ψ-expanding, or return $\omega = (R, \nu)$ where R is the ψ-sparse cut with maximum volume, and ν is its volume. The space requirement of this algorithm is linear in the size of its input.

We then have parameters $\alpha = 1, \lambda = 1$, and we set $k = \log n$. Hence, for any $b \leq 1/\log^2 n$ and $\epsilon \leq b \log n$, we meet the preconditions of Lemma 6.1: $b \leq 1/\log n$, $\alpha = 1 \leq \frac{1}{b \log n}$, $\lambda = O(1)$, $k \leq \log n$, and

$$\frac{\log n^5 \alpha}{\log \frac{1}{\lambda\sqrt{b}}} \leq \frac{O(\log n)}{\Omega(\log \log n)} \leq k,$$

where the first inequality follows because $\alpha = \lambda = 1$, and $b \leq 1/\log^2 n$. Therefore, by Lemma 6.1 there is a dynamic stream algorithm that with high probability outputs a $(b, \epsilon, \phi, \gamma)$-BLD of G where

$$\phi = \Omega\left(\frac{\epsilon}{\alpha \log n}\right) = \Omega\left(\frac{\epsilon}{\log n}\right) \quad \text{and} \quad \gamma = O(\alpha^{k+1}) = O(1).$$

Its space complexity is

$$\tilde{O}(k \cdot n/b^3) \cdot \alpha^{O(k)} = \tilde{O}(n/b^3),$$

since for the brute force algorithm we have $S(p, q) = \tilde{O}(q)$. The running time is $2^{O(n)}$ since the brute force algorithm has $T(p, q) = \tilde{O}(q) \cdot 2^p$. □

Proof of Theorem 2.2. For the polynomial time version, we use the following BSCA.

Theorem 6.2 ([SW19, LNPS23, LR99]). For $b \in (0, 1)$, let $p \leq n$, $q \leq \min\{p^2, n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n\}$ and $W \leq \text{poly}(n)$ be integers. For p-vertex q-edge input graphs with weighted edges and weighted self-loops, and total volume bounded by W, there is a $(c_{\text{bsc}} \log^3 n, C_{\text{bsc}})$-BSCA that runs in $\text{poly}(n)$ time and $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ bits of space, uses $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ random bits, and works with probability $1 - 1/\text{poly}(n)$ for every input sparsity parameter $0 < \psi \leq \frac{1}{10 c_{\text{bsc}} \log^3 n}$, where $c_{\text{bsc}}, C_{\text{bsc}} \geq 1$ are absolute constants.

Hereafter let us set $\alpha = c_{\mathrm{bsc}} \log^3 n, \lambda = C_{\mathrm{bsc}}$ and $k = \frac{\log n^5 \alpha}{\log b^{-1/2}/\lambda}$. We want to apply Lemma 6.1 in this parameter regime with Theorem 6.2 as $\mathrm{BalSparseCut}_{\alpha,\lambda}$.

One can see that the decoding of Lemma 6.1 (i.e. Algorithms 3 and 4) calls $\mathrm{BalSparseCut}_{\alpha,\lambda}$ on disjoint portions of $b^{-1} \cdot O(k \log n) = \tilde{O}(1/b)$ many sparsifiers (see the proof of Lemma 6.1). Then, because each sparsifier has at most $\tilde{O}(n/b^2) \cdot \alpha^{O(k)} = \tilde{O}(n/b^2) \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ edges (see the proof of Lemma 6.1), we only need $\tilde{O}(n/b^3) \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ random bits in order to avoid adaptivity and at the same time ensure that all the calls work correctly with probability $1 - 1/\operatorname{poly}(n)$. As this fits into our claimed space budget, we can then assume that this BSCA always outputs correctly. From Lemma 6.1 we also know that the algorithm never calls $\mathrm{BalSparseCut}_{\alpha,\lambda}$ with sparsity parameters larger than $\frac{1}{10\alpha}$, so our setting matches the requirement of Theorem 6.2. We defer the proof of Theorem 6.2 to Section A.1.

We now verify the rest of the preconditions of Lemma 6.1. For any $b \leq 1/\log^5 n$ and $\epsilon \leq b \log n$, have that $\lambda = C_{\mathrm{bsc}} = O(1)$ and

$$\alpha = c_{\mathrm{bsc}} \log^3 n \leq \log^4 n \leq \frac{1}{b \log n}\,.$$

We also have that $k = O(\log n / \log\log n) \leq \log n$, since $\alpha = O(\log^3 n)$, $\lambda = O(1)$, and $b \leq 1/\log^5 n$. Therefore, the lemma gives a dynamic stream algorithm that with high probability outputs a $(b, \epsilon, \phi, \gamma)$-BLD of G where

$$\phi = \Omega\left(\frac{\epsilon}{\alpha \log n}\right) = \Omega\left(\frac{\epsilon}{\log^4 n}\right) \quad \text{and} \quad \gamma = O(\alpha^{k+1}) = \log^{O(\frac{\log n}{\log 1/b})} n\,.$$

Since the BSCA from Theorem 6.2 takes at most $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ bits of space, the overall space complexity is

$$\tilde{O}(k \cdot n/b^3) \cdot \alpha^{O(k)} + n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n = n/b^3 \cdot \log^{O(\frac{\log n}{\log 1/b})} n\,.$$

The running time is $\operatorname{poly}(n)$, since the BSCA from Theorem 6.2 takes at most $\operatorname{poly}(n)$ time. □

Overview of the algorithm. We now give the BLD construction for Lemma 6.1. Algorithms 3 and 4 outline the core procedure, which is almost the same as the one of [FKM23]. For BLD parameters b and ϵ we set ϕ appropriately in terms of ϵ, and our goal is to show that calling Algorithm 3 as $\mathrm{Decompose}(V, 0)$ gives a BLD of G.

Algorithm 3 is a space efficient implementation of Algorithm 1 for BLD. In line (6), it uses a BSCA on a sparsifier either to certify that the given cluster is already an expander, or to find a sparse cut in it. In the former case it can simply return the cluster. In the latter case, if the condition (10) finds the cut to be balanced enough, the algorithm can afford to recurse on both sides keeping the recursion depth under control. Otherwise, it calls Algorithm 4. This procedure is supposed to either trim off a small volume mass from the cluster in line (18) so that the remainder induces an expander (see return statement in line (12)), or find a less sparse but more balanced cut than the one obtained in line (6) of Algorithm 3 (see return statement in line (16)). In the former case, Algorithm 3 can then recurse only on the trimmed part of the cluster (recursion of line (14)), whereas in the latter it is still acceptable to recurse on both sides while keeping the recursion depth under control (recursion of line (16)).

As both algorithms run a BSCA on weighted subgraphs of G, we use Lemma 5.3 to analyse their behaviour with respect to the input G. It is then crucial that the graphs on which the BSCA is run are sparsifiers for the cluster at hand. Therefore, we will implement lines (5) and (10) in Algorithms 3 and 4 respectively via the algorithm from Lemma 4.2. Crucially, we want to

instantiate enough independent copies of the sparsification algorithm, so that lines (5) and (10) can always access a fresh sparsifier. The memory requirement is then determined by the sparsifiers that we maintain, while the correctness will be proved using Lemma 5.3.

Roadmap. In the next sections we prove Lemma 6.1. In Section 6.1, we work under the assumption that line (5) of Algorithm 3 and (10) of Algorithm 4 always behave as expected, meaning that they return a sparsifier for the cluster at hand. In this ideal setting, we show that Algorithm 3 outputs a BLD in small recursion depth and few iterations. Then, with Lemma 4.2, in Section 6.2 we lift the assumption that line (5) of Algorithm 3 and (10) of Algorithm 4 are deterministically correct, thus concluding a proof of Lemma 6.1.

6.1 Offline analysis

The algorithms have hardwired parameters k, α, λ, C, c. The BLD parameters $b, \phi \in (0,1)$ are given as input (for now, we parametrize the BLD by b and ϕ instead of b and ϵ, and will later specify how to define ϕ from ϵ). Hereafter, whenever we use $b, \phi, k, C, c, \alpha, \lambda, (b_j)_j, (\phi_j)_j$, we are referring to the corresponding parameter in Algorithms 3 and 4.

As one can see from the pseudocode, the input graph G is not simply fed to Algorithms 3 and 4, but they have restricted access to it: their interaction with G is limited to lines (5) and (10) respectively, i.e. they only have sparsifier-access to it. In this section we work under the assumption that the algorithms can always obtain a sparsifier that meets the precondition of Lemma 5.3, i.e. they are cluster sparsifiers with appropriate error (see Definition 4.1).

Assumption 1. Every time line (5) of Algorithm 3 calls $\text{Sparsifier}^c_\ell(U, b)$ for some $\ell \geq 0$, $U \subseteq V$, it gets a graph $H = (V, E', w)$ such that $H[U] \approx_\delta G[U]$, where $\delta = c^2 b / \log n$.

Assumption 2. Every time line (10) of Algorithm 4 calls $\text{Sparsifier}^c_{\ell,j,h}(A, b_j)$ for some $\ell \geq 0$, $j, h \geq 1$, $A \subseteq V$, it gets a graph $H = (V, E', w)$ such that $H[A] \approx_{\delta_j} G[A]$, where $\delta_j = c^2 b_j / \log n$.

Throughout this section, we will use the definition $\delta = c^2 b / \log n$ and $\delta_j = c^2 b_j / \log n$ for all $j \in [k+1]$, as in Assumption 1 and Assumption 2. To illustrate why in the second assumption we have δ_j instead of δ, we make the following observation.

Observation 6.3. For a constant $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, a cluster $A \subseteq V$, and a sparsifier $H = (V, E', w)$ such that $H[A] \approx_{\delta_j} G[A]$, Lemma 5.3 guarantees that calling an (α, λ)-BSCA as $\text{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1 + \frac{1}{2 \log n}) \cdot \phi_j)$ gives a $(\phi_j, (1 + \frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A]^\circ$ (as in line (11) of Algorithm 4).

Proof of Observation 6.3. For a constant $c \leq 1/30$, parameters b, ϕ, δ, α such that $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, a cluster $A \subseteq V$, and a sparsifier $H = (V, E', w)$ such that $H[A] \approx_\delta G[A]$, Lemma 5.3 guarantees that calling an (α, λ)-BSCA as $\text{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1 + 0.5/\log n) \cdot \phi)$ gives a $(\phi, (1 + \frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A]^\circ$. Then, we recall that $G[A]^\circ = G[A]^{b/\phi}$ and $H[A]^\circ = H[A]^{b/\phi}$, so it also holds that $G[A]^\circ = G[A]^{b_j/\phi_j}$ and $H[U]^\circ = H[U]^{b_j/\phi_j}$, since we can see from Algorithm 4 that $b_j/\phi_j = b/\phi$ for all $j \in [k+1]$. Moreover, observe the following: $\phi < b$ if and only if $\phi_j < b_j$, $\alpha \leq \frac{1}{2b}$ implies $\alpha \leq \frac{1}{2b_j}$ (by definition of b_j in Algorithm 4), and $b \leq c$ implies $b_j \leq c$ (again, by definition of b_j in Algorithm 4). One can then use Lemma 5.3 replacing b, ϕ, δ with b_j, ϕ_j, δ_j respectively, hence getting Observation 6.3. □

The goal of this section is threefold: in Section 6.1.3, we bound the number of iterations that any execution of Algorithm 4 can go through; in Section 6.1.4, we bound the recursion depth of

Algorithm 3; in Section 6.1.5, we show the output of calling Algorithm 3 as Decompose$(V, 0)$ is a BLD of G. Interestingly, Section 6.1.5 crucially uses the results from Section 6.1.3 and Section 6.1.4 about recursion depth and number of iterations. Before, we give some preliminary results: in Section 6.1.1 we give properties of cuts of boundary-linked subgraphs; in Section 6.1.2, we show how relations between volume estimates (such as the comparisons performed by Algorithm 3 in line (10) and by Algorithm 4 in lines (16) and (17)) translate to relations between the actual volumes of the corresponding cuts.

6.1.1 Properties of nested cuts

When Algorithm 3 makes a cut in the input cluster U, this cut is then fed as input to the recursive call in line (10), which will in turn try to find a cut inside it. Similarly, Algorithm 4 makes a cut in the current cluster A in line (18), and in the next iteration it will try to find a cluster inside the remainder of A. The leitmotif of these algorithms is that they try to make cuts inside a cluster that is one side of a cut previously made. It is then useful for the analysis to relate volume and sparsity of nested cuts.

Lemma 6.4 (Properties of nested cut). Let $b, \phi \in (0,1)$ and $\delta > 0$ be such that $\phi \le b$, $\delta < 1/b$. Fix a cluster $U \subseteq V$, let $\emptyset \neq S \subsetneq U$ be a cut such that $\Phi^\circ_U(S) < \delta \cdot \phi$, and let $T \subseteq S$. Then we have the following relations between volumes:

$$\mathrm{vol}^\circ_U(T) \overset{(1)}{\le} \mathrm{vol}^\circ_S(T) \overset{(2)}{<} \mathrm{vol}^\circ_U(T) + \delta b \cdot \min\{\mathrm{vol}^\circ_U(S), \mathrm{vol}^\circ_U(U \setminus S)\} \overset{(3)}{\le} \mathrm{vol}^\circ_U(U)\,.$$

If it also holds that $\emptyset \neq T \subsetneq S$ has $\Phi^\circ_S(T) < \delta \cdot \phi$ and $\mathrm{vol}^\circ_U(T \cup (U \setminus S)) < \frac{1}{2}\mathrm{vol}^\circ_U(U)$ then

$$\Phi^\circ_U(T \cup (U \setminus S)) < \delta \cdot \phi\,.$$

Proof. For convenience let $\psi = \delta \cdot \phi$. Inequality (1) holds simply because $T \subseteq S \subseteq U$, so the number of self-loops attached to T cannot decrease when going from $G[U]^\circ$ to $G[S]^\circ$. Inequality (2) can be obtained by rewriting the volumes in $G[S]^\circ$ as

$$\begin{aligned}\mathrm{vol}^\circ_S(T) = \mathrm{vol}(T) + \frac{b-\phi}{\phi}\,\mathrm{bnd}_S(T) &= \mathrm{vol}^\circ_U(T) + \frac{b-\phi}{\phi}|E(T, U \setminus S)| &(6.1)\\ &\le \mathrm{vol}^\circ_U(T) + \frac{b}{\phi}|E(T, U \setminus S)|\,. &(6.2)\end{aligned}$$

Since S is ψ-sparse in $G[U]^\circ$, we can upper-bound $|E(T, U \setminus S)|$ by $\partial_U S < \psi \min\{\mathrm{vol}^\circ_U(S), \mathrm{vol}^\circ_U(U \setminus S)\}$, thus getting the bound. For inequality (3), note $\mathrm{vol}^\circ_U(T) \le \mathrm{vol}^\circ_U(S)$ because $T \subseteq S$ and $\delta b \cdot \min\{\mathrm{vol}^\circ_U(S), \mathrm{vol}^\circ_U(U \setminus S)\} \le \mathrm{vol}^\circ_U(U \setminus S)$ for $\delta \le 1/b$. Since $\mathrm{vol}^\circ_U(S) + \mathrm{vol}^\circ_U(U \setminus S) = \mathrm{vol}^\circ_U(U)$, we have the bound (3).

Suppose now that $\emptyset \neq T \subsetneq S$ has $\Phi^\circ_S(T) < \psi$ and $\mathrm{vol}^\circ_U(T \cup (U \setminus S)) < \frac{1}{2}\mathrm{vol}^\circ_U(U)$. First observe that because $\Phi^\circ_S(T) < \psi$, the definition of $\Phi^\circ_S(T)$ yields $\partial_S T < \psi\,\mathrm{vol}^\circ_S(T)$. By Eq. (6.2), we also have

$$\partial_S T < \psi\,\mathrm{vol}^\circ_S(T) \le \psi\,\mathrm{vol}^\circ_U(T) + \frac{\psi b}{\phi} \cdot |E(T, U \setminus S)| \le \psi\,\mathrm{vol}^\circ_U(T) + |E(T, U \setminus S)|\,, \tag{6.3}$$

where we used that $\psi b/\phi \le 1$ since $\psi = \delta\phi$ and $\delta \le 1/b$. Due to $\mathrm{vol}^\circ_U(T \cup (U \setminus S)) < \frac{1}{2}\mathrm{vol}^\circ_U(U)$,

the definition of $\Phi^\circ_U(T \cup (U \setminus S))$ finally yields

$$\begin{aligned}
\Phi^\circ_U(T \cup (U \setminus S)) &= \frac{\partial_U(T \cup (U \setminus S))}{\mathrm{vol}^\circ_U(T \cup (U \setminus S))} \\
\text{since } T \subseteq S, \text{ so } T \cap (U \setminus S) = \emptyset \quad &= \frac{\partial_S T + \partial_U S - |E(T, U \setminus S)|}{\mathrm{vol}^\circ_U(T) + \mathrm{vol}^\circ_U(U \setminus S)} \\
\text{by (6.3)} \quad &< \frac{\psi \,\mathrm{vol}^\circ_U(T) + |E(T, U \setminus S)| + \partial_U S - |E(T, U \setminus S)|}{\mathrm{vol}^\circ_U(T) + \mathrm{vol}^\circ_U(U \setminus S)} \\
&= \frac{\psi \,\mathrm{vol}^\circ_U(T) + \partial_U S}{\mathrm{vol}^\circ_U(T) + \mathrm{vol}^\circ_U(U \setminus S)} \\
&\leq \max\left\{\psi, \frac{\partial_U S}{\mathrm{vol}^\circ_U(U \setminus S)}\right\} = \max\left\{\psi, \Phi^\circ_U(S)\right\} \\
\text{since } \Phi^\circ_U(S) < \psi \quad &\leq \psi\,.
\end{aligned}$$

□

Algorithm 4 makes cuts iteratively. It finds a cut in the cluster, it trims it off in line (18), it finds another cut in the remainder, it trims it off, and so on, provided that these cuts verify certain conditions: their individual volumes are neither too large (if one of them was very large in volume, the algorithm would return in line (16)) nor too small (see condition (17)). The following lemma proves properties of such cut sequences. Figure 3 should help the reader to visualize the statement and the proof.

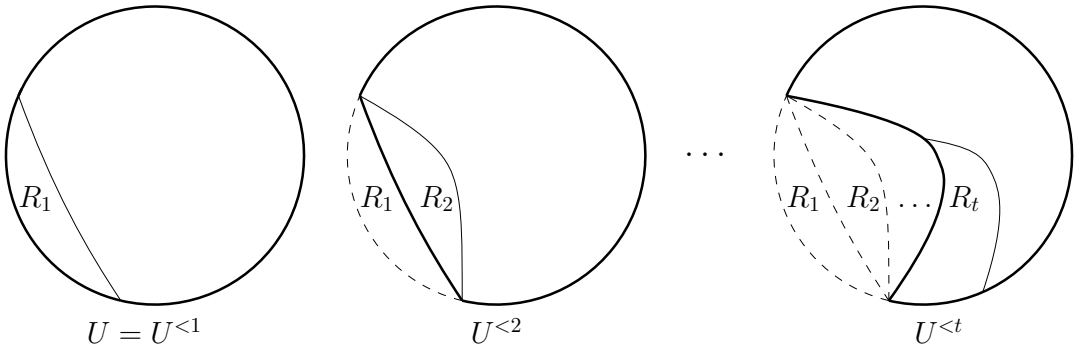


Figure 3: Sequence of nested cuts.

Lemma 6.5 (Properties of a sequence of nested cuts). Let $b, \phi \in (0,1)$ be such that $\phi \leq b$, and $\delta \in (0,1]$. Fix a cluster $U \subseteq V$, let $\theta \in \mathbb{R}_{\geq 0}$ be such that $\theta \leq \frac{1}{5}\,\mathrm{vol}^\circ_U(U)$, and let $R_1, \ldots, R_t$ be a sequence of cuts $\emptyset \neq R_i \subsetneq U^{<i}$ where $U^{<i} = U \setminus (\cup_{z=1}^{i-1} R_z)$ for all $i \in [t]$. If

1. for all $\emptyset \neq X \subsetneq U$ with $\Phi^\circ_U(X) < \delta\phi$ and $\mathrm{vol}^\circ_U(X) \leq \mathrm{vol}^\circ_U(U \setminus X)$ one has $\mathrm{vol}^\circ_U(X) \;<\; \theta$, and
2. for all $i \in [t]$ one has $\Phi^\circ_{U^{<i}}(R_i) < \delta\phi$ and $\mathrm{vol}^\circ_{U^{<i}}(R_i) < \frac{1}{5}\,\mathrm{vol}^\circ_U(U)$,

then we have the following.

1. For all $i \in [t]$, one has $\Phi^\circ_U(\cup_{z=1}^{i} R_z) < \delta\phi$ and $\mathrm{vol}^\circ_U(\cup_{z=1}^{i} R_z) < \theta$.
2. For all $i \in [t]$, if there is $\theta^{\mathrm{low}} \in \mathbb{R}_{\geq 0}$ such that $\mathrm{vol}^\circ_{U^{<i}}(R_i) \geq \theta^{\mathrm{low}}$ then one also has $\mathrm{vol}^\circ_U(R_i) \geq \theta^{\mathrm{low}} - b \cdot \theta$.

3. Let $U^{<t+1} = U \setminus (\cup_{z=1}^{t} R_z)$. If $\emptyset \neq R \subsetneq U^{<t+1}$ is a cut such that $\Phi^{\circ}_{U^{<t+1}}(R) < \delta\phi$ and $\theta^{\text{low}} \leq \text{vol}^{\circ}_{U^{<t+1}}(R) \leq \frac{5}{9}\,\text{vol}^{\circ}_{U^{<t+1}}(U^{<t+1})$ for some $\theta^{\text{low}} \leq \frac{4}{9}\,\text{vol}^{\circ}_{U}(U)$, then $\Phi^{\circ}_{U}(R) < \delta\phi \cdot \frac{\text{vol}^{\circ}_{U}(U)}{\theta^{\text{low}} - b\cdot\theta}$ and $\theta^{\text{low}} - b\cdot\theta \leq \text{vol}^{\circ}_{U}(R) \leq \frac{5}{9}\,\text{vol}^{\circ}_{U}(U)$.

The end goal of part (1) is to upper bound the number of edges crossing and the remaining volume when Algorithm 4 returns $(A, \top)$ and therefore Algorithm 3 recurses on $U \setminus A$ only (see line (7) in Algorithm 3). The idea is that in this case $U \setminus A$ corresponds to the union of nested cuts that are trimmed off by Algorithm 4.

Part (2) will be useful to bound the number of iterations that Algorithm 4 can take. In every iteration a cut is trimmed off, and from condition (17) we know the volume of these cuts to be not too small in $U^{<i}$. From part (2) we then know these volumes to be not too small in U either, and hence the number of such cuts cannot be too large, since they are disjoint.

The end goal of part (3) is to upper bound the number of edges crossing and the remaining volume when Algorithm 4 returns $(R, \bot)$ and therefore Algorithm 3 recurses on R and $U \setminus R$ (see line (10) in Algorithm 3). The idea is that in this case, every cut trimmed off by Algorithm 4 is sparse and far from being balanced (because of condition (16) in Algorithm 4), whereas the last one is both sparse and somewhat balanced (because of condition (16) in Algorithm 4).

In the remainder of this section we prove the three parts of this lemma.

Proof of Lemma 6.5, part (1). Intuitively, the union of a sequence of nested sparse and small cuts should be sparse and small too. We prove this by induction on $i \in [t]$. Denote for convenience $\psi = \delta\,\phi$.

The base case for $i = 1$ readily follows. Since $U^1 = U$, we have $\Phi^{\circ}_{U}(R_1) < \psi$ and also $\text{vol}^{\circ}_{U}(R_1) < \frac{1}{5}\,\text{vol}^{\circ}_{U}(U)$ from precondition (2). Moreover, by precondition (1), there is no ψ-sparse cut X with $\theta \leq \text{vol}^{\circ}_{U}(X) < \frac{1}{2}\,\text{vol}^{\circ}_{U}(U)$. Hence, it must be the case that $\text{vol}^{\circ}_{U}(R_1) < \theta$.

For the inductive step, take $i \in [t-1]$ and assume that $\Phi^{\circ}_{U}(\cup_{z=1}^{i} R_z) < \psi$ and $\text{vol}^{\circ}_{U}(\cup_{z=1}^{i} R_z) < \theta$. We now apply Lemma 6.4 with $S = U^{<i+1}$ and $T = R_{i+1}$. We can do so because $\Phi^{\circ}_{U}(S) = \Phi^{\circ}_{U}(\cup_{z=1}^{i} R_z) < \psi$, so the lemma gives

$$\text{vol}^{\circ}_{U}(R_{i+1}) \leq \text{vol}^{\circ}_{U^{<i+1}}(R_{i+1}) \leq \frac{1}{5}\,\text{vol}^{\circ}_{U}(U) < \frac{1}{4}\,\text{vol}^{\circ}_{U}(U)\,.$$

Combining this bound with the inductive hypothesis that $\text{vol}^{\circ}_{U}(\cup_{z=1}^{i} R_z) < \theta$, one has

$$\text{vol}^{\circ}_{U}(\cup_{z=1}^{i+1} R_z) = \text{vol}^{\circ}_{U}(\cup_{z=1}^{i} R_z) + \text{vol}^{\circ}_{U}(R_{i+1}) < \theta + \text{vol}^{\circ}_{U}(R_{i+1}) < \frac{1}{5}\,\text{vol}^{\circ}_{U}(U) + \frac{1}{4}\,\text{vol}^{\circ}_{U}(U) < \frac{1}{2}\,\text{vol}^{\circ}_{U}(U)\,,$$

which rewrites as $\text{vol}^{\circ}_{U}(T \cup (U \setminus S)) < \frac{1}{2}\,\text{vol}^{\circ}_{U}(U)$ with $S = U^{<i+1}$ and $T = R_{i+1}$. Also, from precondition (2), we have $\Phi^{\circ}_{U^{<i+1}}(R_{i+1}) < \psi$, or in other words $\Phi^{\circ}_{S}(T) < \psi$ with $S = U^{<i+1}$ and $T = R_{i+1}$. Then, the second part of Lemma 6.4 gives $\Phi^{\circ}_{U}(\cup_{z=1}^{i+1} R_z) < \psi$, so one part of the induction is proved. For the other part, we combine $\Phi^{\circ}_{U}(\cup_{z=1}^{i+1} R_z) < \psi$ and $\text{vol}^{\circ}_{U}(\cup_{z=1}^{i+1} R_z) < \frac{1}{2}\,\text{vol}^{\circ}_{U}(U)$ with precondition (1): since we assume there is no ψ-sparse cut in $G[U]^{\circ}$ with volume sandwiched between θ and $\frac{1}{2}\,\text{vol}^{\circ}_{U}(U)$, it must be the case that $\text{vol}^{\circ}_{U}(\cup_{z=1}^{i+1} R_z) < \theta$. □

Proof of Lemma 6.5, part (2). We prove this using part (1) and Lemma 6.4. The case of $i = 1$ follows from the preconditions, since $U^1 = U$. Let then $i = 2, \ldots, t$, and denote for convenience $\psi = \delta\,\phi$. Then, part (1) gives that $\Phi^{\circ}_{U}(\cup_{z=1}^{i-1} R_z) < \psi$. If we set $S = U^{<i}$ and $T = R_i$ we have $\Phi^{\circ}_{U}(S) = \Phi^{\circ}_{U}(\cup_{z=1}^{i-1} R_z) < \psi$, and can then apply Lemma 6.4 to get the lower bound

$$\begin{aligned} \text{vol}^{\circ}_{U}(R_i) &> \text{vol}^{\circ}_{U^{<i}}(R_i) - \delta b \cdot \text{vol}^{\circ}_{U}(\cup_{z=1}^{i-1} R_z) \\ \text{since } \delta \leq 1 \quad &\geq \text{vol}^{\circ}_{U^{<i}}(R_i) - b \cdot \text{vol}^{\circ}_{U}(\cup_{z=1}^{i-1} R_z)\,. \end{aligned}$$

Part (1) also gives $\mathrm{vol}^\circ_U(\cup_{z=1}^{i-1} R_z) < \theta$, and since we assume that $\mathrm{vol}^\circ_{U^{<i}}(R_i) \geq \theta^{\mathrm{low}}$, one concludes

$$\mathrm{vol}^\circ_U(R_i) > \mathrm{vol}^\circ_{U^{<i}}(R_i) - b \cdot \mathrm{vol}^\circ_U(\cup_{z=1}^{i-1} R_z) \geq \theta^{\mathrm{low}} - b \cdot \theta\,.$$

□

Proof of Lemma 6.5, part (3). We begin by analysing the volume $\mathrm{vol}^\circ_U(R)$. Again, from part (1) we have

$$\Phi^\circ_U(\cup_{i=1}^t R_i) < \delta\phi\,, \quad \mathrm{vol}^\circ_U(\cup_{i=1}^t R_i) < \theta\,.$$

Setting $\psi = \delta\phi$, $S = U^{<t+1}$ and $T = R$, we have $\Phi^\circ_U(S) = \Phi^\circ_U(\cup_{z=1}^t R_z) < \psi$, so we can apply Lemma 6.4 to yield

$$\mathrm{vol}^\circ_U(R) > \mathrm{vol}^\circ_{U^{<t+1}}(R) - b \cdot \mathrm{vol}^\circ_U(\cup_{i=1}^t R_i) > \theta^{\mathrm{low}} - b \cdot \theta\,, \tag{6.4}$$

and

$$\mathrm{vol}^\circ_U(R) \leq \mathrm{vol}^\circ_{U^{<t+1}}(R_t) \leq \frac{5}{9}\,\mathrm{vol}^\circ_{U^{<t+1}}(U^{<t+1})\,. \tag{6.5}$$

We can once more apply Lemma 6.4 with $S = T = U^{<t+1}$ (since again $\Phi^\circ_U(S) = \Phi^\circ_U(\cup_{z=1}^t R_z) < \psi$), so

$$\mathrm{vol}^\circ_{U^{<t+1}}(U^{<t+1}) \leq \mathrm{vol}^\circ_U(U)\,. \tag{6.6}$$

We then have from (6.4), (6.5), (6.6)

$$\theta^{\mathrm{low}} - b \cdot \theta \leq \mathrm{vol}^\circ_U(R) \leq \frac{5}{9}\,\mathrm{vol}^\circ_{U^{<t+1}}(U^{<t+1}) \leq \frac{5}{9}\,\mathrm{vol}^\circ_U(U)\,. \tag{6.7}$$

Next we analyse the sparsity of R by first upper-bounding the quantity $\partial_U R$:

$$\begin{aligned}
\partial_U R &\leq \partial_{U^{<t+1}} R + \partial_U(\cup_{i=1}^t R_i) && (6.8)\\
\text{since } \Phi^\circ_{U^{<t+1}}(R) < \psi \text{ by assumption} \quad &< \psi \cdot \mathrm{vol}^\circ_{U^{<t+1}}(R) + \partial_U(\cup_{i=1}^t R_i) && (6.9)\\
\text{since } \Phi^\circ_U(\cup_{i=1}^t R_i) < \psi \text{ from part (1)} \quad &< \psi \cdot \mathrm{vol}^\circ_{U^{<t+1}}(R) + \psi \cdot \mathrm{vol}^\circ_U(\cup_{i=1}^t R_i) && (6.10)\\
\text{since } \mathrm{vol}^\circ_U(R) < \frac{5}{9}\,\mathrm{vol}^\circ_U(U) \text{ from (6.7)} \quad &< \psi \cdot \frac{5}{9} \cdot \mathrm{vol}^\circ_U(U) + \psi \cdot \mathrm{vol}^\circ_U(\cup_{i=1}^t R_i) && (6.11)\\
\text{since } \mathrm{vol}^\circ_U(\cup_{i=1}^t R_i) < \theta \text{ from part (1)} \quad &< \psi \cdot \frac{5}{9} \cdot \mathrm{vol}^\circ_U(U) + \psi \cdot \theta && (6.12)\\
\text{by assumption on } \theta \quad &< \psi \cdot \mathrm{vol}^\circ_U(U)\,. && (6.13)
\end{aligned}$$

On the other hand, the bounds in (6.7) yield

$$\min\{\mathrm{vol}^\circ_U(R), \mathrm{vol}^\circ_U(U \setminus R)\} \geq \min\left\{\theta^{\mathrm{low}} - b \cdot \theta, \frac{4}{9}\,\mathrm{vol}^\circ_U(U)\right\} \tag{6.14}$$

$$\begin{aligned}
\text{since } \theta^{\mathrm{low}} \leq \frac{4}{9}\,\mathrm{vol}^\circ_U(U) \text{ by assumption} \quad &= \theta^{\mathrm{low}} - b \cdot \theta\,. && (6.15)
\end{aligned}$$

From the bound (6.13) and (6.15) we finally conclude

$$\Phi^\circ_U(R) = \frac{\partial_U R}{\min\{\mathrm{vol}^\circ_U(R), \mathrm{vol}^\circ_U(U \setminus R)\}} < \frac{\psi \cdot \mathrm{vol}^\circ_U(U)}{\theta^{\mathrm{low}} - b \cdot \theta}\,.$$

□

6.1.2 Relations between volume estimates

In this section we show how relations between volume estimates (such as the comparisons performed by Algorithm 3 in line (10) and by Algorithm 4 in lines (16) and (17)) translate to relations between the actual volumes of the corresponding cuts.

Lemma 6.6 (Relations between volume estimates). Let $b, \phi, c \in (0,1)$ be such that $c \leq 1/4$ is a constant and $\phi \leq b$. Fix a cluster $U \subseteq V$, and also let $H = (V, E', w)$ such that $H[U]^\circ \approx_c G[U]^\circ$, and let ω be a $(\psi, \alpha', \lambda', c)$-BSCW of $G[U]^\circ$ for some $\psi \in (0, \phi]$ and $\alpha', \lambda' \geq 1$. If $\omega = (R, \nu)$ for some $\emptyset \neq R \subsetneq U$ and $\nu \in \mathbb{R}_{>0}$, then for any $\rho, \tau \in [0,1]$ and $X \subseteq V$ we have

1. if $\nu \geq \rho \cdot (\mathrm{vol}_X^{\circ w}(X))^\tau$, then $\mathrm{vol}_U^\circ(R) \geq \rho/2 \cdot (\mathrm{vol}_X^\circ(X))^\tau$.
2. if $\nu < \rho \cdot (\mathrm{vol}_X^{\circ w}(X))^\tau$, then $\mathrm{vol}_U^\circ(R) < 2\rho \cdot (\mathrm{vol}_X^\circ(X))^\tau$ and for every cut $\emptyset \neq S \subsetneq U$ with $\Phi_U^\circ(S) < \psi$ and $\mathrm{vol}_U^\circ(S) \leq \mathrm{vol}_U^\circ(U \setminus S)$ one has $\mathrm{vol}_U^\circ(S) < 2\lambda'\rho \cdot (\mathrm{vol}_X^\circ(X))^\tau$.

Proof. From property (2d) of the definition of BSCW (see Definition 5.1), we know $(1-c)\,\mathrm{vol}_U^\circ(R) \leq \nu \leq (1+c)\,\mathrm{vol}_U^\circ(R)$, and from property (2b) we know that for every cut $\emptyset \neq S \subsetneq U$ with $\Phi_U^\circ(S) < \psi$ and $\mathrm{vol}_U^\circ(S) \leq \mathrm{vol}_U^\circ(U \setminus S)$ one has $\mathrm{vol}_U^\circ(S) < \lambda' \cdot \mathrm{vol}_U^\circ(R)$.

On the other hand, $(1-c)\,\mathrm{vol}_X^\circ(X) \leq \mathrm{vol}_X^{\circ w}(X) \leq (1+c)\,\mathrm{vol}_X^\circ(X)$ for any $X \subseteq V$. To see why, first note that $\mathrm{vol}_X^{\circ w}(X) = \mathrm{vol}^w(X) + \frac{b-\phi}{\phi}\,\mathrm{bnd}_X^w(X) = \mathrm{vol}^w(X) + \frac{b-\phi}{\phi}\partial^w X$, and analogously $\mathrm{vol}_X^\circ(X) = \mathrm{vol}(X) + \frac{b-\phi}{\phi}\partial X$. Then, since $H[U]^\circ \approx_c G[U]^\circ$, we have the approximation guarantee for global cuts (see Definition 4.1): applying it to all singleton cuts in X we can approximate $\mathrm{vol}(X)$, and applying it to X we can approximate ∂X.

From the above discussion it follows that for all $\rho, \tau \in [0,1]$ and $X \subseteq V$, $\nu \geq \rho \cdot (\mathrm{vol}_X^{\circ w}(X))^\tau$ implies $\mathrm{vol}_U^\circ(R) \geq \rho\frac{1-c}{1+c} \cdot (\mathrm{vol}_X^\circ(X))^\tau \geq \rho/2 \cdot (\mathrm{vol}_X^\circ(X))^\tau$, since $c \leq 1/4$. If instead $\nu < \rho \cdot (\mathrm{vol}_X^{\circ w}(X))^\tau$, we have $\mathrm{vol}_U^\circ(R) < \rho\frac{1+c}{1-c} \cdot (\mathrm{vol}_X^\circ(X))^\tau \leq 2\rho \cdot (\mathrm{vol}_X^\circ(X))^\tau$. Moreover, since by definition of ω one has $\mathrm{vol}_U^\circ(S) < \lambda' \cdot \mathrm{vol}_U^\circ(R)$ for every cut $\emptyset \neq S \subsetneq U$ with $\Phi_U^\circ(S) < \psi$ and $\mathrm{vol}_U^\circ(S) \leq \mathrm{vol}_U^\circ(U \setminus S)$, we get $\mathrm{vol}_U^\circ(S) < \lambda' \cdot \mathrm{vol}_U^\circ(R) \leq \lambda' 2\rho \cdot (\mathrm{vol}_X^\circ(X))^\tau$ for any such S. □

6.1.3 Bounding the number of iterations

Combining the structural properties of nested cuts together with Lemma 6.6, we can show the correctness of the algorithms. We begin by observing that whenever Algorithm 4 breaks the inner loop in line (20) we have $j \leq k$. This ensures that restraining the outer loop to indices $j = 1, \ldots, k+1$ (see line (7)) does not lead to undefined behaviour.

Lemma 6.7 (Algorithm 4 does few outer loop iterations). Let $b, \phi, c \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$ and $b \leq c$. Also let $C \geq 3$ be a constant, and let $\mathrm{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA where $\alpha \leq \frac{1}{2b}$. If Assumption 2 holds, then if Algorithm 4 breaks in line (20) it is in some outer loop iteration j with $j \leq k$.

Proof. We prove this by contradiction, so suppose the algorithm enters line (20) with $j = k+1$ for some inner loop iteration $h \geq 1$. Let A be the value of the corresponding variable in Algorithm 4 at the beginning of inner loop iteration h within outer loop iteration $k+1$. Also let $H = (V, E', w)$ be the graph gotten from line (10) in the same iteration. From the description of the algorithm, since it entered line (20), the call $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1+1/(2\log n)) \cdot \phi_{k+1})$ from line (11) has returned a BSCW of the form (R, ν) such that $\nu < \frac{1}{C\lambda}(\mathrm{vol}_U^{\circ w}(U))^{1-(k+1-1)/k} = \frac{1}{C\lambda}(\mathrm{vol}_U^{\circ w}(U))^0$. Moreover, since we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and $H[A] \approx_{\delta_{k+1}} G[A]$ (from Assumption 2), Lemma 5.3 ensures that (R, ν) is a $(\phi_j, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A]^\circ$ (see Observation 6.3).

We now want to use Lemma 6.6 with $\psi = \phi_{k+1}$, $\alpha' = (1 + \frac{1}{\log n})\alpha$, $\lambda' = (1 + c)\lambda$, $\rho = \frac{1}{C\lambda}$, $\tau = 0$, and $X = U$. These parameters verify the conditions for applying Lemma 6.6 to (R, ν) on $G[A]^\circ$: $c \leq 1/30 \leq 1/4$, $\psi = \phi_{k+1} \leq \phi$ (by definition of ϕ_{k+1} in the algorithm), $\alpha', \lambda' \geq 1$, $\rho \leq 1$, $\tau \in [0, 1]$, $H[A] \approx_c G[A]$ (since $H[A] \approx_{\delta_{k+1}} G[A]$ and $\delta_{k+1} \leq c$), and hence (R, ν) is a $(\phi_j, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[A]^\circ$. Then Lemma 6.6 gives

$$\mathrm{vol}^\circ_A(R) < 2\rho(\mathrm{vol}^\circ_U(U))^\tau = \frac{2}{C\lambda} < 1\,,$$

which in particular implies $R = \emptyset$. However, this contradicts the fact that (R, ν) is a BSCW of $G[A]^\circ$ (see Definition 5.1). □

Next, we show that Algorithm 4 verifies the preconditions (1) and (2) of Lemma 6.5. Specifically, if Algorithm 3 calls Algorithm 4, there cannot be sparse and balanced cuts in U, and also every cut trimmed by Algorithm 4 is sparse and small in volume. This will later allow us to apply Lemma 6.5 for bounding the number of inner loop iterations (and also for bounding the recursion depth, in the next section).

Lemma 6.8 (Algorithm 4 makes a sequence of nested sparse small cuts). Let $b, \phi, c \in (0, 1)$ such that $c \leq 1/30$ is a constant, $\phi < b$ and $b \leq c$. Also let $C \geq 11$ be a constant, and let $\mathrm{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA where $\alpha \leq \frac{1}{2b}$. Consider an execution of Algorithm 4 called from Algorithm 3 on input cluster $U \subseteq V$. Also let $R_1, \ldots, R_q \subseteq U$ be the sequence of q cuts trimmed in line (18) by Algorithm 4, and let $U^{<i} = U \setminus (\cup_{z=1}^{i-1} R_z)$. If Assumption 1 and Assumption 2 hold, then

1. for all $\emptyset \neq S \subsetneq U$ with $\Phi^\circ_U(S) < \phi$, $\mathrm{vol}^\circ_U(S) \leq \mathrm{vol}^\circ_U(U \setminus S)$, one has $\mathrm{vol}^\circ_U(S) < \frac{3}{C}\,\mathrm{vol}^\circ_U(U)$,
2. for all $i \in [q]$ one has $\Phi^\circ_{U^{<i}}(R_i) < \phi$ and $\mathrm{vol}^\circ_{U^{<i}}(R_i) < \frac{1}{5}\,\mathrm{vol}^\circ_U(U)$.

Proof. From the description of Algorithms 3 and 4, we see that Algorithm 4 starts only if line (6) of Algorithm 3 returns a BSCW $\omega = (R, \nu)$ such that $\nu < \frac{1}{C\lambda}\,\mathrm{vol}^{\circ w}_U(U)$, as per line (12) (otherwise, Algorithm 3 would not call Algorithm 4). Now we want to apply Lemma 5.3 to ω. Because we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\delta = c^2 b/\log n$, $\alpha \leq \frac{1}{2b}$ and $H[U] \approx_\delta G[U]$ from Assumption 1, all the preconditions of Lemma 5.3 are met for us to apply it to the call $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[U]^\circ, (1+\frac{1}{2\log n})\cdot \phi)$ from line (6) of Algorithm 3: hence, ω is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U]^\circ$. Since we observed just above that ω is of the form (R, ν) and $\nu < \frac{1}{C\lambda}\,\mathrm{vol}^{\circ w}_U(U)$, we now use Lemma 6.6 with parameters $\psi = \phi$, $\alpha' = (1 + \frac{1}{\log n})\alpha$, $\lambda' = (1 + c)\lambda$, $\rho = \frac{1}{C\lambda}$, $\tau = 1$, and $X = U$. These parameters verify the requirement of Lemma 6.6 since we have $c \leq 1/30 \leq 1/4$, $H[U] \approx_c G[U]$ (from Assumption 1, since $\delta \leq c$), $\psi \leq \phi$, $\alpha', \lambda' \geq 1$, $\rho, \tau \leq 1$, and (R, ν) is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U]^\circ$. Then we get that for every cut $\emptyset \neq S \subsetneq U$ with $\Phi^\circ_U(S) < \phi$ and $\mathrm{vol}^\circ_U(S) \leq \mathrm{vol}^\circ_U(U \setminus S)$ one has

$$\mathrm{vol}^\circ_U(S) < 2\lambda'\rho \cdot \mathrm{vol}^\circ_U(U) = \frac{2(1 + c)\lambda}{C\lambda}\,\mathrm{vol}^\circ_U(U) \leq \frac{3}{C} \cdot \mathrm{vol}^\circ_U(U)\,.$$

This gives part (1).

For all $i \in [q]$, denote by $j(i) \in [k + 1]$ the outer loop iteration of Algorithm 4 where R_i was trimmed, and also let $U^{<i} = U \setminus (\cup_{z=1}^{i-1} R_z)$ (i.e., at the beginning of the iteration where R_i has been trimmed we had $A = U^{<i}$). Then, from the description of the algorithm we know that R_i was paired with a value ν_{R_i} such that (R_i, ν_{R_i}) was the result of $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[U^{<i}]^\circ, (1+\frac{1}{2\log n})\cdot\phi_{j(i)-1})$, where $H = (V, E', w)$ verifies $H[U^{<i}] \approx_{\delta_{j(i)}} G[U^{<i}]$ because of Assumption 2. As we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, Lemma 5.3 ensures that the pair (R_i, ν_{R_i}) is a $(\phi_{j(i)}, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U^{<i}]^\circ$ for all $i \in [q]$ (see Observation 6.3). From the description of the algorithm

we also know that ν_{R_i} must also not satisfy condition (16), as otherwise Algorithm 4 would have returned, so $\nu_{R_i} < \frac{1}{C\lambda}\operatorname{vol}_U^{\circ w}(U)$ for all $i \in [q]$. For each $i \in [q]$, we now use Lemma 6.6 with parameters $\psi = \phi_{j(i)}$, $\alpha' = (1+\frac{1}{\log n})\alpha$, $\lambda' = (1+c)\lambda$, $\rho = \frac{1}{C\lambda}$, $\tau = 1$, and $X = U$. These parameters verify the requirement for applying Lemma 6.6 to (R_i, ν_{R_i}) in $G[U^{<i}]$: $c \leq 1/30 \leq 1/4$, $H[U^{<i}] \approx_c G[U^{<i}]$ (from Assumption 2 and $\delta_{j(i)} \leq c$), $\psi = \phi_{j(i)} \leq \phi$, $\alpha', \lambda' \geq 1$, $\rho, \tau \leq 1$, and (R_i, ν_{R_i}) is a $(\phi_{j(i)}, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U^{<i}]^\circ$. Then, for all $i \in [q]$ Lemma 6.6 gives

$$\operatorname{vol}_{U^{<i}}^\circ(R_i) < \frac{2}{C\lambda}\operatorname{vol}_U^\circ(U) < \frac{1}{5}\operatorname{vol}_U^\circ(U)\,, \tag{6.16}$$

since $C \geq 11$. Furthermore, because every (R_i, ν_{R_i}) is a $(\phi_{j(i)}, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U^{<i}]^\circ$, we have by Definition 5.1

$$\Phi_{U^{<i}}^\circ(R_i) < \left(1+\frac{1}{\log n}\right)\alpha\phi_{j(i)} = \left(1+\frac{1}{\log n}\right)\alpha\frac{\phi}{\left(\left(1+\frac{1}{\log n}\right)\alpha\right)^{j(i)}} \leq \phi\,, \tag{6.17}$$

since $j(i) \geq 1$. The bounds in (6.16) and (6.17) give part (2). □

Finally, we argue that Algorithm 4 goes through few inner loop iterations within each outer loop iteration. The idea for this is that we can upper bound the total volume trimmed in line (18), but we can also lower bound the volume of each individual cut.

Lemma 6.9 (Algorithm 4 does few inner loop iterations). Let $b, \phi, c \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$ and $b \leq c$. Also let $C \geq 31$ be a constant, let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α,λ)-BSCA where $\alpha \leq \frac{1}{2b}$ and $\lambda \leq \frac{c}{2b}$, and let k be an integer such that $k \geq \log\frac{n^2}{\phi}/\log\frac{c}{\lambda\cdot b}$. If Assumption 1 and Assumption 2 hold, then Algorithm 4 goes through at most $1/b$ inner loop iterations within each outer loop iteration.

Proof. Let U be the input cluster of Algorithm 4. Fix hereafter an outer loop iteration $j = 1, \ldots, k+1$, and let A be the value of the corresponding variable of the algorithm at the beginning of the j-th outer loop iteration. Because line (18) of Algorithm 4 removes a non-empty cut from A at every inner loop iteration, there are finitely many iterations. Denote then by t the number of inner loop iterations for outer loop iteration j. Then, for every inner loop iteration $h \in [t]$ define $A^{<h}$ to be the value of variable A at the beginning of inner loop iteration h.

Since Assumption 2 holds, we also know that to every inner loop iteration $h \in [t]$ corresponds a graph $H = (V, E', w)$ from line (10) which satisfies $H[A^{<h}] \approx_{\delta_j} G[A^{<h}]$. As we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and $H[A^{<h}] \approx_{\delta_j} G[A^{<h}]$, Lemma 5.3 ensures that calling $\text{BalSparseCut}_{\alpha,\lambda}(H[A^{<h}]^\circ, (1+\frac{1}{2\log n})\cdot\phi_j)$ gives a $(\phi_j, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A^{<h}]^\circ$ in line (11) of Algorithm 4 (see Observation 6.3). Moreover, from the description of the algorithm, it must be the case that the BSCW obtained is of the form (R_h, ν_h) for all $h \in [t-1]$, where $\emptyset \neq R_h \subsetneq A^{<h}$ and $\nu_h \in \mathbb{R}_{>0}$. One can then see that for every $h \in [t]$ we have $A^{<h} = A \setminus (\cup_{z=1}^{h-1} R_z)$, and because of conditions (16) and (17) we have that for every $h \in [t-1]$ the following holds:

$$\frac{1}{C\lambda}\left(\operatorname{vol}_U^{\circ w}(U)\right)^{1-(j-1)/k} \leq \nu_h < \frac{1}{C\lambda}\operatorname{vol}_U^{\circ w}(U)\,.$$

We now want to translate the above bounds to $\operatorname{vol}_{A^{<h}}^\circ(R_h)$ using Lemma 6.6. To do so, note that $H[A^{<h}] \approx_{\delta_j} G[A^{<h}]$ in particular implies $H[A^{<h}] \approx_c G[A^{<h}]$ (since we have defined $\delta_j = c^2 b_j/\log n \leq c$), and as discussed before we know that

$$(R_h, \nu_h) \text{ is a } \left(\phi_j, \left(1+\frac{1}{\log n}\right)\alpha, (1+c)\lambda, c\right)\text{-BSCW of } G[A^{<h}]^\circ \tag{6.18}$$

Let us then set $\psi = \phi_j$, $\alpha' = (1 + \frac{1}{\log n})\alpha$, $\lambda' = (1 + c)\lambda$, $\rho = \frac{1}{C\lambda}$, τ being either $1 - (j-1)/k$ (for the lower bound) or 1 (for the upper bound), and $X = U$. These parameters verify the conditions for applying Lemma 6.6 to (R_h, ν_h) on $G[A^{<h}]^\circ$: $c \leq 1/30 \leq 1/4$, $\psi = \phi_j \leq \phi$, $\alpha', \lambda' \geq 1$, $\rho \leq 1$, both values of τ are bounded by 1 (this is because $j \leq k+1$), $H[A^{<h}] \approx_c G[A^{<h}]$ (since from Assumption 2 we have $H[A^{<h}] \approx_{\delta_j} G[A^{<h}]$), and (R_h, ν_h) is a $(\phi_j, (1 + \frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A^{<h}]^\circ$ (see (6.18)). Then Lemma 6.6 gives

$$\frac{1}{2C\lambda} \left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-1)/k} \leq \mathrm{vol}^\circ_{A^{<h}}(R_h) < \frac{2}{C\lambda} \mathrm{vol}^\circ_U(U)\,, \tag{6.19}$$

where the lower bound is obtained with $\tau = 1 - (j-1)/k$ and the upper bound with $\tau = 1$. The application of Lemma 6.6 with $\tau = 1$ also gives that for every cut $\emptyset \neq S \subsetneq A^{<h}$ with $\Phi^\circ_{A^{<h}}(S) < \phi_j$ and $\mathrm{vol}^\circ_{A^{<h}}(S) \leq \mathrm{vol}^\circ_{A^{<h}}(A^{<h} \setminus S)$ one has

$$\mathrm{vol}^\circ_{A^{<h}}(S) < 2\lambda'\rho \cdot \mathrm{vol}^\circ_U(U) = \frac{2(1+c)\lambda}{C\lambda} \mathrm{vol}^\circ_U(U) \leq \frac{3}{C} \cdot \mathrm{vol}^\circ_U(U)\,. \tag{6.20}$$

We consider now the case of $j = 1$ and $2 \leq j \leq k+1$ separately.

Case $j = 1$. In this case A is identical to the input cluster U. We then want to apply Lemma 6.5 with $\delta = 1$ and $\theta = \frac{3}{C}\mathrm{vol}^\circ_U(U)$ to the sequence of cuts $R_1, R_2, \ldots, R_{t-1}$ in the cluster U. Let for convenience $U^i = U \setminus (\cup_{z=1}^{i-1} R_z)$ for all $i \in [t]$.

First note that this value of θ meets the requirement of Lemma 6.5 that $\theta < \frac{1}{5}\mathrm{vol}^\circ_U(U)$, since $C \geq 16$. Secondly, our set of assumptions matches that of Lemma 6.8, so we get that for all $\emptyset \neq S \subsetneq U$ with $\Phi^\circ_U(S) < \phi$, $\mathrm{vol}^\circ_U(S) \leq \mathrm{vol}^\circ_U(U \setminus S)$, one has $\mathrm{vol}^\circ_U(S) < \frac{3}{C}\mathrm{vol}^\circ_U(U)$, and for all $i \in [q]$ one has $\Phi^\circ_{U^{<i}}(R_i) < \phi$ and $\mathrm{vol}^\circ_{U^{<i}}(R_i) < \frac{1}{5}\mathrm{vol}^\circ_U(U)$. These facts match preconditions (1) and (2) of Lemma 6.5 for δ and θ as we set them above.

Part (1) of Lemma 6.5 then upper-bounds the total volume of the sequence as

$$\mathrm{vol}^\circ_U(\cup_{h=1}^{t-1} R_h) < \frac{3}{C}\mathrm{vol}^\circ_U(U)\,. \tag{6.21}$$

Since the lower bound of (6.18) with $j = 1$ means that $\mathrm{vol}^\circ_{U^{<h}}(R_h) \geq \frac{1}{2C\lambda}\mathrm{vol}^\circ_U(U)$ for all $h \in [t-1]$, using part (2) of Lemma 6.5 with $\theta^{\mathrm{low}} = \frac{1}{2C\lambda}\mathrm{vol}^\circ_U(U)$ we further get

$$\mathrm{vol}^\circ_U(R_h) \geq \frac{1}{2C\lambda}\mathrm{vol}^\circ_U(U) - b \cdot \frac{3}{C}\mathrm{vol}^\circ_U(U) \geq \frac{1}{3C\lambda}\mathrm{vol}^\circ_U(U) \quad \text{for all } h \in [t-1]\,, \tag{6.22}$$

since $\lambda \leq \frac{1}{18b}$. Therefore, taking the ratio of the cumulative upper bound in (6.21) and the individual lower bounds from (6.22), we conclude that $t - 1$ cannot exceed

$$\frac{\mathrm{vol}^\circ_U(\cup_{h=1}^{t-1} R_h)}{\min_{h\in[t-1]} \mathrm{vol}^\circ_U(R_h)} \leq \frac{\frac{3}{C}\mathrm{vol}^\circ_U(U)}{\frac{1}{3C\lambda}\mathrm{vol}^\circ_U(U)} \leq 9\lambda\,.$$

Thus, the number of inner loop iterations for $j = 1$ is at most $9\lambda + 1 \leq \frac{1}{2b} + 1 \leq 1/b$.

Case $2 \leq j \leq k+1$. The approach is analogous to the first iteration, except that we use the bounds from Lemma 6.5 on the sequence $R_1, \ldots, R_{t-1}$ in the cluster A (for $j = 1$ we applied the lemma to the cluster U). We remark that, in this case, A can also be seen as the value of the corresponding variable of Algorithm 4 at the very end of outer loop iteration $j - 1$.

Consider the call $\text{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1+\frac{1}{2\log n})\cdot\phi_{j-1})$ to $\text{BalSparseCut}_{\alpha,\lambda}$ in the last inner loop iteration within outer loop iteration $j-1$, where $H=(V,E',w)$ is accordingly the graph obtained from line (10) at the last inner loop iteration within outer loop iteration $j-1$. Since Assumption 2 holds, we know that the graph H satisfies $H[A]\approx_{\delta_{j-1}} G[A]$. As we have $c\le 1/30$, $\phi<b$, $b\le c$, $\alpha\le\frac{1}{2b}$, and $H[A]\approx_{\delta_{j-1}} G[A]$, Lemma 5.3 ensures that the call $\text{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1+\frac{1}{2\log n})\cdot\phi_{j-1})$ gives a $(\phi_{j-1}, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW ω of $G[A]^\circ$ (see Observation 6.3). On the other hand, from the description of the algorithm we know that iteration j starts only if this ω is of the form (R,ν) with $\nu<\frac{1}{C\lambda}(\text{vol}_U^{\circ w}(U))^{1-(j-2)/k}$ (due to condition (17)). We now use Lemma 6.6 with parameters $\psi=\phi_{j-1}$, $\alpha'=(1+\frac{1}{\log n})\alpha$, $\lambda'=(1+c)\lambda$, $\rho=\frac{1}{C\lambda}$, $\tau=1-(j-2)/k$, and $X=U$. These parameters verify the requirement of Lemma 6.6 since we have $c\le 1/30\le 1/4$, $H[A]\approx_c G[A]$ (from Assumption 2 and $\delta_{j-1}\le c$), $\psi=\phi_{j-1}\le\phi$, $\alpha',\lambda'\ge 1$, $\rho,\tau\le 1$, and (R,ν) is a $(\phi_{j-1},(1+\frac{1}{\log n})\alpha,(1+c),c\lambda)$-BSCW of $G[A]^\circ$. Then we get that for every cut $\emptyset\neq S\subsetneq A$ with $\Phi_A^\circ(S)<\phi_{j-1}$ and $\text{vol}_A^\circ(S)\le\text{vol}_A^\circ(A\setminus S)$ one has

$$\text{vol}_A^\circ(S)<2\lambda'\rho\,(\text{vol}_U^\circ(U))^\tau=\frac{2(1+c)\lambda}{C\lambda}\,(\text{vol}_U^\circ(U))^{1-(j-2)/k}\le\frac{3}{C}\cdot(\text{vol}_U^\circ(U))^{1-(j-2)/k}\,. \tag{6.23}$$

This is still not enough to apply Lemma 6.5 to cluster A: the bound in (6.19) bounds the volumes of nested cuts R_h in $A^{<h}$ by $\frac{2}{C\lambda}\,\text{vol}_U^\circ(U)$, but applying Lemma 6.5 demands an upper bound of $\frac{1}{5}\,\text{vol}_A^\circ(A)$ on the volumes of cuts R_h in $A^{<h}$. Hence, we first show the volume of U to be not much larger than that of A.

Claim 6.10. We have $\text{vol}_U^\circ(U)\le 2\,\text{vol}_A^\circ(A)$.

Proof. If $A=U$ the statement is trivial. Otherwise, we know there must have been a sequence of nested cuts $S_1,\ldots,S_r\subseteq U$ that Algorithm 4 trimmed off from U in line (18) so that $A=U\setminus(\cup_{i\in[r]}S_i)$. We then want to apply Lemma 6.5 to the sequence of cuts $S_1,S_2,\ldots,S_r$ in the cluster U with $\delta=1$ and $\theta=\frac{3}{C}\,\text{vol}_U^\circ(U)$. Let for convenience $U^{<i}=U\setminus(\cup_{z=1}^{i-1}S_z)$ for all $i\in[r+1]$.

First note that this value of θ meets the requirement of Lemma 6.5 that $\theta<\frac{1}{5}\,\text{vol}_U^\circ(U)$, since $C\ge 16$. Secondly, our set of assumptions matches that of Lemma 6.8, so we get that for all $\emptyset\neq S\subsetneq U$ with $\Phi_U^\circ(S)<\phi$, $\text{vol}_U^\circ(S)\le\text{vol}_U^\circ(U\setminus S)$, one has $\text{vol}_U^\circ(S)<\frac{3}{C}\,\text{vol}_U^\circ(U)$, and for all $i\in[r]$ one has $\Phi_{U^{<i}}^\circ(S_i)<\phi$ and $\text{vol}_{U^{<i}}^\circ(S_i)<\frac{1}{5}\,\text{vol}_U^\circ(U)$ (the latter follows because part (2) of Lemma 6.8 bounds the sparsity and volume of every cut in the entire sequence of cuts trimmed by the algorithm, and $S_1,\ldots,S_r$ is a prefix of the entire sequence). These facts match preconditions (1) and (2) of Lemma 6.5 for δ and θ as set above. Part (1) of Lemma 6.5 then upper-bounds the total volume of the sequence as

$$\text{vol}_U^\circ(\cup_{i=1}^r S_i)<\frac{3}{C}\,\text{vol}_U^\circ(U)\,. \tag{6.24}$$

To conclude it is sufficient to notice

$$\begin{aligned}
\text{vol}_U^\circ(U)&=\text{vol}_U^\circ(A)+\text{vol}_U^\circ(U\setminus A)\\
&=\text{vol}_U^\circ(A)+\text{vol}_U^\circ(\cup_{i=1}^r S_i)\\
\text{vol}_A^\circ(A)\text{ cannot have less self-loops than }\text{vol}_U^\circ(A)\quad&\le\text{vol}_A^\circ(A)+\text{vol}_U^\circ(\cup_{i=1}^r S_i)\\
\text{by (6.24)}\quad&<\text{vol}_A^\circ(A)+\frac{3}{C}\,\text{vol}_U^\circ(U)\,,
\end{aligned}$$

and the claim follows since $C\ge 6$. □

By Claim 6.10 and (6.19), we have that for every $h\in[t-1]$

$$\text{vol}_{A^{<h}}^\circ(R_h)<\frac{2}{C\lambda}\,\text{vol}_U^\circ(U)\le\frac{4}{C\lambda}\,\text{vol}_A^\circ(A)<\frac{1}{5}\,\text{vol}_A^\circ(A)\,, \tag{6.25}$$

since $C \geq 21$. Furthermore, because every (R_h, ν_h) is a $(\phi_j, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[A^{<h}]^\circ$, we have

$$\Phi^\circ_{A^{<h}}(R_h) < \left(1+\frac{1}{\log n}\right)\alpha\phi_j = \left(1+\frac{1}{\log n}\right)\alpha\frac{\phi}{\left(\left(1+\frac{1}{\log n}\right)\alpha\right)^j} \leq \phi_{j-1}\,. \tag{6.26}$$

We are now in shape to apply Lemma 6.5 to the sequence of cuts $R_1, R_2, \ldots, R_{t+1}$ in the cluster A. Specifically, we do this with $\delta = \phi_{j-1}/\phi$ and $\theta = \frac{3}{C}(\mathrm{vol}^\circ_U(U))^{1-(j-2)/k}$. First observe that $\delta \leq 1$ (by definition of ϕ_{j-1}) and $\theta \leq \frac{3}{C}\,\mathrm{vol}^\circ_U(U) \leq \frac{1}{5}\,\mathrm{vol}^\circ_A(A)$ (again by Claim 6.10 and $C \geq 31$), as demanded by Lemma 6.5. For these parameters, one can see that precondition (2) is given by (6.25) and (6.26). Precondition (1) is instead given by (6.23). Part (1) of Lemma 6.5 then upper-bounds the total volume of the sequence as

$$\mathrm{vol}^\circ_A(\cup_{h=1}^{t-1} R_h) < \theta = \frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-2)/k}\,. \tag{6.27}$$

We are left with lower bounding the volume of each individual cut. We do this with part (2) of Lemma 6.5 with $\theta^{\mathrm{low}} = \frac{1}{2C\lambda}(\mathrm{vol}^\circ_U(U))^{1-(j-1)/k}$, which meets the requirement that $\mathrm{vol}^\circ_A(R_h) \geq \theta^{\mathrm{low}}$ thanks to the bound in (6.19). Part (2) of Lemma 6.5 then gives that for all $h \in [t-1]$

$$\mathrm{vol}^\circ_A(R_h) \geq \theta^{\mathrm{low}} - b\cdot\theta = \frac{1}{2C\lambda}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-1)/k} - b\cdot\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-2)/k}\,. \tag{6.28}$$

Therefore, taking the ratio of the upper bound on the total volume from (6.27) and the lower bound on the volume of each individual cut from (6.28), we conclude that $t-1$ is at most

$$\begin{aligned}\frac{\mathrm{vol}^\circ_A(\cup_{h=1}^{t-1}R_h)}{\min_{h\in[t-1]}\mathrm{vol}^\circ_A(R_h)} &\leq \frac{\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-2)/k}}{\frac{1}{2C\lambda}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-1)/k} - b\cdot\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1-(j-2)/k}} \\ &= \frac{\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1/k}}{\frac{1}{2C\lambda} - b\cdot\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1/k}}\,.\end{aligned}$$

Next, we observe that $\mathrm{vol}^\circ_U(U) \leq \mathrm{vol}(V)\cdot b/\phi \leq \mathrm{vol}(V)/\phi \leq n^2/\phi$ (since in the worst case we over-count every edge in G as a self-loop in $G[U]^\circ$ and at most b/ϕ times). Then, using that $k \geq \log(\frac{n^2}{\phi})/\log(\frac{c}{\lambda\cdot b})$, it holds that $\left(\mathrm{vol}^\circ_U(U)\right)^{1/k} \leq \frac{c}{\lambda\cdot b}$. Now we can upper-bound the above as

$$\frac{\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1/k}}{\frac{1}{2C\lambda} - b\cdot\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1/k}} \leq \frac{\frac{3}{C}\left(\mathrm{vol}^\circ_U(U)\right)^{1/k}}{\frac{1}{2C\lambda} - b\cdot\frac{3}{C}\frac{c}{\lambda\cdot b}} \leq \frac{\frac{3}{C}\cdot\left(\frac{n^2}{\phi}\right)^{1/k}}{\frac{1}{5C\lambda}} \leq \frac{\frac{3}{C}\cdot\frac{c}{\lambda\cdot b}}{\frac{1}{5C\lambda}} = 15\frac{c}{b}\,,$$

since $c \leq \frac{1}{30}$. Hence, the number of inner loop iterations for $2 \leq j \leq k+1$ is at most $\frac{15c}{b}+1 \leq \frac{1}{b}$. □

6.1.4 Bounding the recursion depth

From the description of the algorithms one can see that we recurse on both sides of a cut (in line (16) of Algorithm 3) only when the conditions (10) in Algorithm 3 and (16) in Algorithm 4 suggest that the cut is balanced, so as to maintain the recursion depth small. When instead Algorithm 3 recurses on one side only, in line (14), we can use the properties of nested cuts to upper bound the volume of the recursed side. To control the number of self-loops that we add after each recursion, we also want to ensure that we recurse on sparse cuts only.

Lemma 6.11 (Algorithm 3 recurses on balanced sparse cuts). Let $b, \phi, c \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$, $b \leq c$. Also let $C \geq 16$ be a constant, and let $\mathrm{BalSparseCut}_{\alpha,\lambda}$ be an (α,λ)-BSCA where $\alpha \leq \frac{1}{2b}$ and $\lambda \leq \frac{1}{18b}$. If Assumption 1 and Assumption 2 hold, whenever Algorithm 3 with input cluster $U \subseteq V$ makes a recursive call, the recursion is on a cluster $\emptyset \neq X \subsetneq U$ such that

$$\Phi^{\circ}_U(X) < (3C\lambda + 2\alpha) \cdot \phi \quad \text{and} \quad \mathrm{vol}^{\circ}_U(X) \leq \left(1 - \frac{1}{3C\lambda}\right) \mathrm{vol}^{\circ}_U(U)\,.$$

Proof. There are three lines where a recursive call can happen: (10), (14), (16). We analyse them separately.

Recursion in line (10). Let $H = (V, E', w)$ be the graph returned in line (5) of Algorithm 3. Since the algorithm recurses in line (10), the call $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[U]^{\circ}, (1 + \frac{1}{2\log n}) \cdot \phi)$ of line (6) must have returned a BSCW of the form (R, ν) such that

$$\nu \geq \frac{1}{C\lambda} \mathrm{vol}^{\circ w}_U(U)\,. \tag{6.29}$$

As we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and $H[U] \approx_{\delta} G[U]$ (because of Assumption 1), Lemma 5.3 gives that (R, ν) is a $(\phi, (1 + \frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U]^{\circ}$. By Definition 5.1, this means

$$\Phi^{\circ}_U(R) < \left(1 + \frac{1}{\log n}\right) \alpha \cdot \phi \leq 2\alpha \cdot \phi\,.$$

We now want to use Lemma 6.6 with $\psi = \phi$, $\alpha' = (1 + \frac{1}{\log n})\alpha$, $\lambda' = (1+c)\lambda$, $\rho = \frac{1}{C\lambda}$, $\tau = 1$, and $X = U$. These parameters verify the conditions for applying Lemma 6.6 to (R, ν) on $G[U]^{\circ}$: $c \leq 1/30 \leq 1/4$, $\psi \leq \phi$, $\alpha', \lambda' \geq 1$, $\rho \leq 1$, $\tau \leq 1$, $H[U] \approx_c G[U]$ (since $H[U] \approx_{\delta} G[U]$ and $\delta \leq c$), and (R, ν) is a $(\phi, (1 + \frac{1}{\log n}, c)\alpha, (1+c)\lambda)$-BSCW of $G[U]^{\circ}$. Then Lemma 6.6 with (6.29) gives

$$\mathrm{vol}^{\circ}_U(R) \geq \frac{\rho}{2}(\mathrm{vol}^{\circ}_U(U))^{\tau} = \frac{1}{2C\lambda} \mathrm{vol}^{\circ}_U(U)\,,$$

which we can use to upper bound the volume of $U \setminus R$:

$$\mathrm{vol}^{\circ}_U(U \setminus R) = \mathrm{vol}^{\circ}_U(U) - \mathrm{vol}^{\circ}_U(R) \leq \left(1 - \frac{1}{2C\lambda}\right) \mathrm{vol}^{\circ}_U(U).$$

Finally, by property (2c) of BSCW applied to R, $\mathrm{vol}^{\circ}_U(R) \leq (1+c)\,\mathrm{vol}^{\circ}_U(U \setminus R)$. Because $c \leq 1/30 \leq 1/2$ and $C \geq 2$,

$$\mathrm{vol}^{\circ}_U(R) \leq \frac{1+c}{2} \mathrm{vol}^{\circ}_U(U) \leq \left(1 - \frac{1}{2C\lambda}\right) \mathrm{vol}^{\circ}_U(U).$$

Since in this case the algorithm recurses on R and $U \setminus R$, we have the claim.

Recursion in lines (14) and (16). These cases can only happen if Algorithm 3 resorts to Algorithm 4. Let then $R_1, \ldots, R_q \subseteq U$ be the sequence of cuts trimmed off by Algorithm 4 in line (12). We then want to apply Lemma 6.5 to the sequence of cuts $R_1, R_2, \ldots, R_q$ in the cluster U with $\delta = 1$ and $\theta = \frac{3}{C} \mathrm{vol}^{\circ}_U(U)$. Let for convenience $U^{<i} = U \setminus (\cup_{z=1}^{i-1} R_z)$ for all $i \in [q+1]$.

First note that this value of θ meets the requirement of Lemma 6.5 that $\theta < \frac{1}{5} \mathrm{vol}^{\circ}_U(U)$, since $C \geq 16$. Secondly, our set of assumptions matches that of Lemma 6.8, so we get that for all $\emptyset \neq S \subsetneq U$ with $\Phi^{\circ}_U(S) < \phi$, $\mathrm{vol}^{\circ}_U(S) \leq \mathrm{vol}^{\circ}_U(U \setminus S)$, one has $\mathrm{vol}^{\circ}_U(S) < \frac{3}{C} \mathrm{vol}^{\circ}_U(U)$, and for all $i \in [q]$ one has $\Phi^{\circ}_{U^{<i}}(R_i) < \phi$ and $\mathrm{vol}^{\circ}_{U^{<i}}(R_i) < \frac{1}{5} \mathrm{vol}^{\circ}_U(U)$. These facts match preconditions (1) and (2) of Lemma 6.5 for δ and θ as we set them above.

- Line (14). Part (1) of Lemma 6.5 then upper-bounds the total sparsity and volume of the sequence as

$$\Phi^\circ_U(\cup_{z=1}^q R_z) < \phi \quad \text{and} \quad \mathrm{vol}^\circ_U(\cup_{z=1}^q R_z) < \frac{3}{C}\,\mathrm{vol}^\circ_U(U)\,. \tag{6.30}$$

  For Algorithm 3 to recurse in line (14), it must be the case that Algorithm 4 returned a pair $(S, \top)$ in line (12) (see condition (13)). Looking into Algorithm 4, one can then see that if it returns a pair of the form $(S, \top)$, it must be the case that S is the value of variable A at the end of the algorithm, i.e. $S = A = U \setminus (\cup_{z\in[q]} R_z)$. In line (14), the recursive call is Decompose$(U \setminus S, \ell + 1)$, so we recurse on $U \setminus S = \cup_{z\in[q]} R_z$. From (6.30) we then have the claim since $C \geq 6$.

- Line (16). The difference from the previous case is that for Algorithm 3 to recurse in line (16), it must be the case that Algorithm 4 returned a pair $(S, \bot)$ in line (12) (see condition (13)), and in particular this implies that Algorithm 4 returned in line (16).

  Let p be the outer loop iteration where Algorithm 4 returns, and let $H = (V, E', w)$ be the graph from line (10) in the last inner loop iteration of outer loop iteration p (i.e. the inner loop iteration where it returns). Since in such iteration we must have that variable A actually equals $U^{<q+1}$, let ω be the result of $\mathrm{BalSparseCut}_{\alpha,\lambda}(H[U^{q+1}]^\circ, (1 + \frac{1}{2\log n})\phi_p)$ in that same iteration. As we have $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and $H[U^{<q+1}] \approx_{\delta_p} G[U^{<q+1}]$ (from Assumption 2), Lemma 5.3 ensures that ω is a $(\phi_p, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U^{<q+1}]^\circ$ (see Observation 6.3). Moreover, since Algorithm 4 returned in line (16), it must be the case that ω is of the form (S, ν), where S is the cut returned from Algorithm 4 to Algorithm 3 in the pair $(S, \bot)$. From the description of the algorithm it must also be the case that

$$\nu \geq \frac{1}{C\lambda}\,\mathrm{vol}^{\circ\,w}_U(U)\,. \tag{6.31}$$

  As we have $c \leq 1/30 \leq 1/4$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and $H[U^{<q+1}] \approx_c G[U^{<q+1}]$ (from Assumption 2 and $\delta_p \leq c$), the fact that (S, ν) is a $(\phi_p, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U^{<q+1}]^\circ$, and the bound (6.31), Lemma 6.6 gives

$$\mathrm{vol}^\circ_{U^{<q+1}}(S) \geq \frac{1}{2C\lambda}\,\mathrm{vol}^\circ_U(U)\,. \tag{6.32}$$

  Since (S, ν) is a $(\phi_p, (1 + \frac{1}{\log n})\alpha, (1 + c)\lambda, c)$-BSCW of $G[U^{<q+1}]^\circ$, by Definition 5.1 we also know

$$\mathrm{vol}^\circ_{U^{<q+1}}(S) \leq (1 + c)\,\mathrm{vol}^\circ_{U^{<q+1}}(U \setminus S)\,,$$

  so

$$\mathrm{vol}^\circ_{U^{<q+1}}(S) \leq \frac{5}{9}\,\mathrm{vol}^\circ_{U^{<q+1}}(U)\,, \tag{6.33}$$

  since $c \leq 1/30 \leq 1/10$. Now, setting $R = S$ and $\theta^{\mathrm{low}} = \frac{1}{2C\lambda}\,\mathrm{vol}^\circ_U(U)$, the bounds in (6.32) and (6.33), together with the fact that $\theta^{\mathrm{low}} \leq \frac{4}{9}\,\mathrm{vol}^\circ_U(U)$ (since $C \geq 2$), allow us to apply part (3) of Lemma 6.5. This gives

$$\Phi^\circ_U(S) < \phi\frac{\mathrm{vol}^\circ_U(U)}{\theta^{\mathrm{low}} - b\cdot\theta} \tag{6.34}$$

$$= \phi \cdot \frac{\mathrm{vol}^\circ_U(U)}{\frac{1}{2C\lambda}\,\mathrm{vol}^\circ_U(U) - b\cdot\frac{3}{C}\,\mathrm{vol}^\circ_U(U)} \tag{6.35}$$

$$\text{since } b \leq \frac{1}{18\lambda} \quad \leq 3C\lambda\cdot\phi\,, \tag{6.36}$$

and

$$\frac{1}{3C\lambda}\operatorname{vol}^\circ_U(U) \le \theta^{\text{low}} - b\cdot\theta \le \operatorname{vol}^\circ_U(S) \le \frac{5}{9}\operatorname{vol}^\circ_U(U)\,. \tag{6.37}$$

We can now conclude: since the result of Algorithm 4 is $(S,\perp)$, Algorithm 3 recurses on both S and $U\setminus S$ in line (16). Let $X\in\{S,U\setminus S\}$. Then from (6.36) we have that $\Phi^\circ_U(X) < 3C\lambda\cdot\phi$, and from (6.37) we have that $\operatorname{vol}^\circ_U(X) \le (1-\frac{1}{3C\lambda})\operatorname{vol}^\circ_U(U)$ since $C\ge 2$.

□

We now upper-bound the recursion depth of the algorithm, using the above lemma.

Lemma 6.12 (Algorithm 3 has small recursion depth). Let $b,\phi,c\in(0,1)$ such that $c\le 1/30$ is a constant, $\phi<b$, $b\le c$. Also let $C\ge 16$ be a constant, and let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α,λ)-BSCA where $\alpha\le\frac{1}{144C\lambda b}$, $\lambda\le\min\{\frac{1}{18b},\frac{1}{15C\sqrt{b}}\}$. If Assumption 1 and Assumption 2 hold, then the recursion depth of $\text{Decompose}(V,0)$ (i.e. Algorithm 3) is at most $9C\lambda\cdot\log n$.

Proof. The crux is that recursion only happens on one of the sides of a not too large cut. As our set of assumptions matches that of Lemma 6.11 (note that the preconditions are the same except $\alpha\le\frac{1}{144C\lambda b}\le\frac{1}{2b}$ and $\lambda\le\min\{\frac{1}{18b},\frac{1}{15C\sqrt{b}}\}\le\frac{1}{18b}$), we get the following: for any recursion from U to X

$$\Phi^\circ_U(X) < (2\alpha+3C\lambda)\phi \quad\text{ and }\quad \operatorname{vol}^\circ_U(X) \le \left(1-\frac{1}{3C\lambda}\right)\operatorname{vol}^\circ_U(U) \tag{6.38}$$

To get a bound on the recursion depth, we translate these bounds to a bound on $\operatorname{vol}^\circ_X(X)$.

We use Lemma 6.4 with $\delta = 3C\lambda+2\alpha$, $S=X$, and $T=X$. These setting verifies the requirements of Lemma 6.4: $\delta\le 1/b$ (since $\alpha\le\frac{1}{4b}$, and $\lambda\le\frac{1}{6Cb}$), $\Phi^\circ_U(S)<\delta\phi$ from (6.38), and $T\subseteq S$. Hence, inequality (2) from Lemma 6.4 gives

$$\begin{aligned}
\operatorname{vol}^\circ_X(X) &= \operatorname{vol}^\circ_S(T)\\
\text{inequality (2) of Lemma 6.4}\quad &< \operatorname{vol}^\circ_U(T) + \delta b\cdot\min\{\operatorname{vol}^\circ_U(S),\operatorname{vol}^\circ_U(U\setminus S)\}\\
&\le (1+\delta b)\cdot\operatorname{vol}^\circ_U(X)\\
\text{from (6.38)}\quad &\le (1+\delta b)\left(1-\frac{1}{3C\lambda}\right)\operatorname{vol}^\circ_U(U)\\
\text{since } \alpha\le\frac{1}{144C\lambda b}\text{ and }\lambda\le\frac{1}{15C\sqrt{b}}\quad &\le \left(1+\frac{1}{36C\lambda}\right)\left(1-\frac{1}{3C\lambda}\right)\operatorname{vol}^\circ_U(U)\\
&\le \left(1-\frac{1}{4C\lambda}\right)\operatorname{vol}^\circ_U(U)\,.
\end{aligned}$$

From this we conclude that if we do a recursion on X at level ℓ, then

$$\operatorname{vol}^\circ_X(X) \le \left(1-\frac{1}{4C\lambda}\right)^\ell \operatorname{vol}^\circ_V(V) = \left(1-\frac{1}{4C\lambda}\right)^\ell \operatorname{vol}(V) < \left(1-\frac{1}{4C\lambda}\right)^\ell n^2\,.$$

The bound follows. □

6.1.5 Correctness

Finally, one can see from the algorithms that they recurse only on the sides of cuts resulting from $\text{BalSparseCut}_{\alpha,\lambda}$. We know such cuts to be sparse, so intuitively there should be few inter-cluster edges in the resulting partition of V. We also know that when the algorithm returns a cluster $\{U\}$, it is because the corresponding BSCW declares it to be an expander. One can then prove that the result of this process is a BLD.

Lemma 6.13 (Algorithm 3 outputs a BLD). Let $b, \phi, c \in (0,1)$ such that $c \leq 1/30$ is a constant, $\phi < b$, $b \leq c$. Also let $C \geq 31$ be a constant, let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α,λ)-BSCA where $\alpha \leq \frac{1}{144C\lambda b}$, $\lambda \leq \min\{\frac{1}{18b}, \frac{1}{15C\sqrt{b}}\}$, and let k be an integer such that $k \leq \log n$. If Assumption 1 and Assumption 2 hold, then the result of $\text{Decompose}(V,0)$ (i.e. Algorithm 3) is a (b,ϵ,ϕ,γ)-BLD of G where

$$\epsilon = 4(3C\lambda + 2\alpha) \cdot \phi \cdot 9C\lambda \cdot \log n \cdot \mathrm{e}^{2b(3C\lambda+2\alpha)9C\lambda \cdot \log n} \quad \text{and} \quad \gamma = 6\alpha^{k+1}\,.$$

Proof. First note that our assumptions subsume the preconditions of Lemma 6.7. Hence, if Algorithm 4 breaks in line (20) it is in some outer loop iteration j with $j \leq k$, so the behaviour of the algorithm is well defined (see the outer loop for $j = 1, \ldots, k+1$ in line (7) of Algorithm 4).

From the description we then see that $\text{Decompose}(V,0)$ always outputs a valid partition $\mathcal{U}$ of V. We show that $\mathcal{U}$ verifies the two properties of Definition 3.1.

Property (2). Let $X \in \mathcal{U}$, which can either be the result of the return statement in line (7) or in line (14) of Algorithm 3.

- Return statement in line (7). Let $U \subseteq V$ be the input cluster to the instance of Algorithm 4 that returned X. Also let $H = (V, E', w)$ be the graph from line (5) and ω be the result of $\text{BalSparseCut}_{\alpha,\lambda}(H[U]^\circ, (1+\frac{1}{2\log n})\cdot\phi)$ in line (6) of that same instance. From the description of the algorithm, we know that the return statement in line (7) only occurs if $\omega = \perp$, and in particular X is the input cluster U.

  As our parameter regime implies $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and also we have $H[U] \approx_\delta G[U]$ from Assumption 1, we can use Lemma 5.3. This ensures that ω is a $(\phi, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U]^\circ$. From Definition 5.1, $\omega = \perp$ implies that $G[U]^\circ$ is a ϕ-expander.

- Return statement in line (14). In this case, Algorithm 3 must have called Algorithm 4, which in turn returned a pair $(S, \top)$. Then, Algorithm 4 must have returned in line (12). Denoting by A the value of the corresponding variable in Algorithm 4 when it returns, we must have $A = X$. Also let $p \in [k+1]$ be the outer loop iteration where Algorithm 4 returns, let $H = (V, E', w)$ be the graph from line (10), and let ω be the result of $\text{BalSparseCut}_{\alpha,\lambda}(H[A]^\circ, (1+\frac{1}{2\log n})\cdot\phi_p)$ in line (6) of the inner loop iteration where the algorithm returns.

  As our parameter regime implies $c \leq 1/30$, $\phi < b$, $b \leq c$, $\alpha \leq \frac{1}{2b}$, and also we have $H[A] \approx_{\delta_p} G[A]$ from Assumption 2, we can use Lemma 5.3. This ensures that ω is a $(\phi_p, (1+\frac{1}{\log n})\alpha, (1+c)\lambda, c)$-BSCW of $G[U]^\circ$ (see Observation 6.3). From Definition 5.1, $\omega = \perp$ implies that $G[A]^\circ = G[X]^\circ$ is a ϕ_p-expander. Since $p \leq k+1$, $k \leq \log n$, and ϕ_p is defined as

  $$\phi_p = \frac{\phi}{\left(\left(1+\frac{1}{\log n}\right)\alpha\right)^p}\,,$$

  we conclude that $G[X]^\circ$ is in fact a ψ-expander with

  $$\psi = \frac{\phi}{\left(\left(1+\frac{1}{\log n}\right)\alpha\right)^p} \geq \frac{\phi}{\left(\left(1+\frac{1}{\log n}\right)\alpha\right)^{k+1}} \geq \frac{\phi}{6\alpha^{k+1}}\,.$$

From the above case analysis we conclude that for every $X \in \mathcal{U}$, $G[U]^\circ$ is a ϕ/γ-expander with $\gamma = 6\alpha^{k+1}$.

Property (1). We will bound the number of inter-cluster edges by counting the number of edges cut at every recursion level, and then sum over levels. For convenience, let then $D = 9C\lambda \cdot \log n$. We can then employ Lemma 6.12 (since our set of assumptions matches that of the lemma), which bounds the recursion depth by D.

Next, for $\ell = 0, 1, \ldots, D$, let $\mathcal{I}^\ell$ be the set of instances of Algorithm 3 at the ℓ-th level of the recursion triggered from Decompose$(V, 0)$. Each instance $\mathcal{I} \in \mathcal{I}^\ell$ is a pair $\mathcal{I} = (U, X)$, where $U \subseteq V$ is the input cluster to the instance, and:

1. if the instance returns $\{U\}$ in line (7), then $X = \emptyset$;
2. if the instance makes a recursive call, then $\emptyset \neq X \subsetneq U$ is a set on which the instance invokes Algorithm 3.

Then, since our set of assumptions matches that of Lemma 6.11, we get that $\Phi^\circ_U(X) < (2\alpha + 3C\lambda)\phi$ for every $\ell = 0, \ldots, D$ and every $(U, X) \in \mathcal{I}^\ell$ such that $X \neq \emptyset$. Hence,

$$\text{for all } \ell = 0, \ldots, D, \text{ and all } (U, X) \in \mathcal{I}^\ell, \text{ we have} \quad \partial_U X < (2\alpha + 3C\lambda)\phi \operatorname{vol}^\circ_U(X)\,. \tag{6.39}$$

With this notation, define the set of edges cut at level ℓ to be

$$C_\ell = \bigcup_{(U,X)\in\mathcal{I}^\ell} E(X, U \setminus X)\,.$$

Now let $\mu = (3C\lambda + 2\alpha)$, and for $\ell = 0, \ldots, D$ also define

$$T(\ell) = \mu\phi \operatorname{vol}(V) + 2\mu \cdot b \sum_{\ell'=0}^{\ell-1} T(\ell')\,.$$

To bound the number of inter cluster edges, we first show that $|C_\ell| \leq T(\ell)$, then we resolve the recurrence $T(\ell)$, and finally we sum over $\ell = 0, \ldots, D$ to conclude.

Claim 6.14. For every $\ell = 0, 1, \ldots, D$ one has $|C_\ell| \leq T(\ell)$.

Proof. We prove the statement by induction. At level 0 there can be only one instance $(V, X) \in \mathcal{I}^0$, so the number of edges cut is $|C_0| = \partial_V X$. By (6.39)

$$|C_0| < \mu\phi \operatorname{vol}^\circ_V(X) = \mu\phi \operatorname{vol}(X) \leq \mu\phi \operatorname{vol}(V) = T(0)\,,$$

thus proving the base case.

Next take any $1 \leq \ell \leq D$, and assume that $|C_{\ell'}| \leq T(\ell')$ for all $\ell' = 0, \ldots, \ell - 1$. By (6.39) we know $\partial_U X < \mu\phi \cdot \operatorname{vol}^\circ_U(X)$ for all $(U, X) \in \mathcal{I}^\ell$. Using the definition of $\operatorname{vol}^\circ_U(X)$, we get

$$\partial_U X < \mu\phi \cdot \operatorname{vol}^\circ_U(X) = \mu\phi \cdot \left(\operatorname{vol}(X) + \frac{b - \phi}{\phi} \operatorname{bnd}_U(X)\right) \tag{6.40}$$

$$\text{since } \operatorname{bnd}_U(X) \leq \partial U \quad \leq \mu\phi \cdot \operatorname{vol}(X) + \mu b \cdot \partial U\,. \tag{6.41}$$

Observe that the union of all $E(U, V \setminus U)$ on level ℓ is a subset of all edges cut on previous recursion levels. This means that

$$\sum_{(U,X)\in\mathcal{I}^\ell} \partial U \leq 2 \sum_{\ell'=0}^{\ell-1} |C_{\ell'}|\,, \tag{6.42}$$

since every $e \in C_{\ell'}$ is counted at most twice by the sum over $\mathcal{I}^\ell$.

Using the induction hypothesis and observing that the cuts $\{X : (U, X) \in \mathcal{I}^\ell\}$ must be disjoint, we can upper-bound $|C_\ell|$ as

$$\begin{aligned}
|C_\ell| &= \sum_{(U,X)\in\mathcal{I}^\ell} \partial_U X \\
\text{by (6.41) and disjointness of the } X\text{'s} \quad &< \mu\phi \operatorname{vol}(V) + \mu b \sum_{(U,X)\in\mathcal{I}^\ell} \partial U \\
\text{by (6.42)} \quad &\leq \mu\phi \operatorname{vol}(V) + 2\mu b \sum_{\ell'=0}^{\ell-1} |C_{\ell'}| \\
\text{by the induction hypothesis} \quad &\leq \mu\phi \operatorname{vol}(V) + 2\mu b \sum_{\ell'=0}^{\ell-1} T(\ell') \\
&= T(\ell)\,.
\end{aligned}$$

thus showing the inductive statement. □

Now we solve the recurrence.

Claim 6.15. $T(\ell) \leq \mu\phi \operatorname{vol}(V)(1 + 2\mu b)^\ell$ for all $\ell \geq 0$.

Proof. Let $\ell \geq 1$ and assume the statement holds true for all $0 \leq \ell' \leq \ell - 1$. Then,

$$\begin{aligned}
T(\ell) &= \mu\phi \operatorname{vol}(V) + 2\mu b \sum_{\ell'=0}^{\ell-1} T(\ell') \\
&\leq \mu\phi \operatorname{vol}(V) + 2\mu b \sum_{\ell'=0}^{\ell-1} \mu\phi \operatorname{vol}(V)(1 + 2\mu b)^{\ell'} \\
&= \mu\phi \operatorname{vol}(V) \left(1 + 2\mu b \sum_{\ell'=0}^{\ell-1} (1 + 2\mu b)^{\ell'}\right) \\
&= \mu\phi \operatorname{vol}(V) \left(1 + 2\mu b \frac{(1 + 2\mu b)^\ell - 1}{1 + 2\mu b - 1}\right) \\
&= \mu\phi \operatorname{vol}(V)(1 + 2\mu b)^\ell\,.
\end{aligned}$$

□

Now combing Claim 6.14 and Claim 6.15, one has that the number of inter-cluster edges in $\mathcal{U}$ is

$$\sum_{U\in\mathcal{U}} \partial U \leq 2 \sum_{\ell=0}^{D} |C_\ell| \leq 2 \sum_{\ell=0}^{D} T(\ell) \leq 2\mu\phi \operatorname{vol}(V) \cdot (D + 1) \cdot (1 + 2\mu b)^D \leq 4\mu \cdot \phi \cdot \operatorname{vol}(V) \cdot D \cdot e^{2b\mu D}\,.$$

□

6.2 Dynamic stream implementation

In the previous sections we analysed Algorithm 3 and Algorithm 4 in an ideal, offline setting with "free" access to sparsifiers. To get Lemma 6.1, we only need to lift this assumption, and combine lemmas from the previous section.

Lemma 6.1 (Unified algorithm). Let $G = (V, E)$ be a graph given in a dynamic stream, let $b \in (0, 1)$ be a parameter such that $b \leq 1/\log n$. Let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA where $\alpha \leq \frac{1}{b \log n}$ and $\lambda = O(1)$. Also let k be an integer such that $k \geq \log(n^5\alpha)/\log(b^{-1/2}/\lambda)$, and $k \leq \log n$. Then there is an algorithm that maintains a linear sketch of G in $\tilde{O}(n/b^3) \cdot \alpha^{O(k)}$ space. For all $\epsilon \in [n^{-2}, b \log n]$, the algorithm decodes the sketch using $\text{BalSparseCut}_{\alpha,\lambda}$ to compute, with high probability, a $(b, \epsilon, \phi, \gamma)$-BLD of G for

$$\phi = \Omega\left(\frac{\epsilon}{\alpha \log n}\right) \quad \text{and} \quad \gamma = O(\alpha^{k+1}).$$

Moreover, the decoding runs in $\tilde{O}(n/b^3) \cdot \alpha^{O(k)} + S(n, \tilde{O}(n/b^2) \cdot \alpha^{O(k)})$ space and $T(n, \tilde{O}(n/b^2) \cdot \alpha^{O(k)}) \cdot \text{poly}(n)$ time, where $S(p, q)$ and $T(p, q)$, denote the space and time complexity of $\text{BalSparseCut}_{\alpha,\lambda}$ on a graph with p vertices and q edges respectively. Furthermore, the decoding only makes calls to $\text{BalSparseCut}_{\alpha,\lambda}(\cdot, \psi)$ with sparsity parameters $\psi \in (0, 1)$ that satisfy $\psi \leq \frac{1}{10\alpha}$.

Proof. Let $C = 40, c = 1/40$, and define the quantities $D = 9C\lambda \cdot \log n$, $\delta = c^2 b/\log n$, and $\delta_j = \delta \cdot (\alpha(1 + 1/\log n))^{-j}$ for $j \in [k + 1]$. We process the stream as follows.

1. We maintain $D + 1$ independent samples from the distribution $\mathcal{D}_\delta$ using the algorithm of Lemma 4.2, resulting in a collection of sparsifiers H_ℓ for $0 \leq \ell \leq D$.

2. For every $j \in [k + 1]$, we maintain $(D + 1) \cdot 1/b$ samples from the distribution $\mathcal{D}_{\delta_j}$ using the algorithm of Lemma 4.2, resulting in a collection of sparsifiers $H_{\ell,j,h}$ for $0 \leq \ell \leq D$, $j \in [k + 1]$, $h \in [1/b]$.

After having processed the stream as described above, we want to use Algorithm 3 and Algorithm 4 to decode these samples into a BLD. In Algorithm 3 and Algorithm 4 we have hardwired parameters k, α, λ, C, c: the parameters k, α, λ are as in the lemma statement, and C, c are set as defined above (i.e. $C = 40, c = 1/40$). Algorithm 3 and Algorithm 4 also use BLD parameters b, ϕ: b is just the value in the lemma statement, and we set

$$\phi = \frac{\epsilon}{4(3C\lambda + 2\alpha) \cdot D \cdot \mathrm{e}^{2b(3C\lambda+2\alpha)D}}.$$

The only thing left to specify about the algorithms is the sparsifier-access. Since we process the stream by essentially computing a number of sparsifiers, there is a natural way to do this: for $0 \leq \ell \leq D$, we define the procedure $\text{Sparsifier}^c_\ell(U, \cdot)$ from line (5) in Algorithm 3 to return H_ℓ for every $U \subseteq V$; similarly, for $0 \leq \ell \leq D$, $j \in [k + 1]$, $h \in [1/b]$, we define the procedure $\text{Sparsifier}^c_{\ell,j,h}(A, \cdot)$ from line (10) in Algorithm 4 to return $H_{\ell,j,h}$ for every $A \subseteq V$.

Correctness. Since $C, \lambda = O(1)$, $\alpha b \leq 1/\log n$, $D = O(\log n)$, one can see that $\phi = \Omega(\frac{\epsilon}{\alpha \log n})$. From this value of ϕ we also get $\phi < b$, since $C, \lambda, \alpha \geq 1$, $\epsilon \leq b \log n$, the exponential is at least 1, and $D > \log n$. Then, our parameter regime has all the conditions for using Lemma 6.12, Lemma 6.9, Lemma 6.13: we have an (α, λ)-BSCA $\text{BalSparseCut}_{\alpha,\lambda}$, $\phi < b$, $b \leq 1/\log n \leq c \leq 1/30$, $C \geq 31$, $\alpha \leq \frac{1}{b \log n} \leq \frac{1}{144C\lambda b} \leq \frac{1}{2b}$, $\lambda = O(1)$ so $\lambda \leq \min\{\frac{1}{18b}, \frac{1}{15C\sqrt{b}}, \frac{c}{2b}\}$, $k \geq \frac{\log n^5\alpha}{\log b^{-1/2}/\lambda}$ implies

$k \geq \log \frac{n^2}{\phi} / \log \frac{c}{\lambda \cdot b}$ (since $\phi = \Omega(\frac{\epsilon}{\alpha \log n})$, $\epsilon \geq 1/n^2$, and c is a constant), and $k \leq \log n$. We now just want to lift Assumption 1 and Assumption 2, so as to apply these lemmas. From our definition of $\text{Sparsifier}_\ell(U, \cdot)$ and $\text{Sparsifier}_{\ell,j,h}(A, \cdot)$ above, we have the following claim.

Claim 6.16. Assumptions 1 and 2 hold with high probability as long as $\ell \leq D$ and $h \leq 1/b$.

Proof. The claim follows from the way we process the stream, described above, and from our definition of $\text{Sparsifier}^c_\ell(U, \cdot)$ and $\text{Sparsifier}^c_{\ell,j,h}(A, \cdot)$.

- For every $0 \leq \ell \leq D$ and any $U \subseteq V$, $\text{Sparsifier}^c_\ell(U, \cdot)$ outputs the graph H_ℓ, independently sampled from $\mathcal{D}_\delta$. Algorithm 3 only calls $\text{Sparsifier}^c_\ell(U, b)$ for a cluster U that was found using randomness from levels before ℓ (simply by the description of the algorithm, see recursive calls in lines (10), (14), (16) of Algorithm 3). Thus, by Lemma 4.2 we know that $H_\ell[U] \approx_\delta G[U]$ with high probability. Since there can be at most n calls to $\text{Sparsifier}^c_\ell(U, b)$ for any ℓ, we can conclude that H_ℓ works for each of them with high probability.
- For every $0 \leq \ell \leq D$, $j \in [k+1]$, $h \in [1/b]$ and any $A \subseteq V$, $\text{Sparsifier}^c_{\ell,j,h}(A, \cdot)$ outputs the graph $H_{\ell,j,h}$, independently sampled from $\mathcal{D}_{\delta_j}$. Similarly as before, we can see from the description of Algorithm 4 that $\text{Sparsifier}^c_{\ell,j,h}(A, b_j)$ can only be called on a cluster A that depends on the randomness of a previous level (in case A equals the input cluster U to Algorithm 4) or a previous iteration (in case A was updated in line (18)). Thus, A is independent from $H_{\ell,j,h}$. Thus, by Lemma 4.2 we know that $H_{\ell,j,h}[A] \approx_{\delta_j} G[A]$ with high probability. Also in this case there can be at most n calls to $\text{Sparsifier}^c_{\ell,j,h}(A, b_j)$ for any triple ℓ, j, h, so $H_{\ell,j,h}$ works for each of them with high probability.

□

Provided Assumption 1 and Assumption 2 hold, Lemma 6.12 and Lemma 6.9 are guaranteeing that there is no point in the execution of the algorithm with recursion level ℓ larger than D or with inner loop index h (see line (9)) larger than $1/b$. On the other hand, Claim 6.16 ensures that Assumption 1 and Assumption 2 hold with high probability as long as $\ell \leq D$ and $h \leq 1/b$. Hence, we have that the results from Lemma 6.12, Lemma 6.9, Lemma 6.13 hold simultaneously with high probability. In particular, the output $\mathcal{U}$ of $\text{Decompose}(V, 0)$ is a $(b, \epsilon, \phi, \gamma)$-BLD of G with high probability for $\gamma = O(\alpha^{k+1})$ (note that we set ϕ exactly by rearranging the value of ϵ given by Lemma 6.13).

Space complexity. The space requirement for processing the stream is dominated by the sparsifiers that we maintain, plus the space used by any call to $\text{BalSparseCut}_{\alpha,\lambda}$ for the decoding phase. From Lemma 4.2 we know that sampling from $\mathcal{D}_\delta$ can be done by maintaining a linear sketch of $\tilde{O}(n/\delta^2)$ bits, and sampling from $\mathcal{D}_{\delta_j}$ can be done maintaining a linear sketch of $\tilde{O}(n/\delta_j^2)$ bits. Since we maintain $D+1$ samples from $\mathcal{D}_\delta$, and $(D+1) \cdot 1/b$ samples from $\mathcal{D}_{\delta_j}$ for every $j \in [k+1]$, the space complexity for processing the stream is

$$(D+1) \cdot \tilde{O}\left(\frac{n}{\delta^2}\right) + (D+1) \cdot \frac{1}{b} \cdot \sum_{j=1}^{k+1} \tilde{O}\left(\frac{n}{\delta_j^2}\right).$$

Because we defined $D = 9C\lambda \cdot \log n$, $\delta = c^2 b / \log n$, $\delta_j = \delta \cdot (\alpha(1 + 1/\log n))^{-j}$, and we have $C, \lambda = O(1)$, $k = O(\log n), c = \Omega(1)$, we use at most

$$O(\log n)\tilde{O}\left(\frac{n}{b^2}\right) + O(\log n)O\left(\frac{1}{b}\right) O(k) \cdot \tilde{O}\left(\frac{n}{b^2}\right) \cdot \alpha^{O(k)} = \tilde{O}\left(\frac{n}{b^3}\right) \cdot \alpha^{O(k)}$$

bits of space for the sparsifiers. Finally, note that Algorithms 3 and 4 run $\text{BalSparseCut}_{\alpha,\lambda}$ on sparsifiers with at most n vertices and at most

$$\max\left\{\tilde{O}(n/\delta^2),\ \max_{j\in[k+1]}\left\{\tilde{O}(n/\delta_j^2)\right\}\right\} = \tilde{O}\left(\frac{n}{b^2}\right)\cdot\alpha^{O(k)}$$

edges, by Lemma 4.2. As we can reuse the space for each call, this takes an extra $S(n, \tilde{O}(n/b^2)\alpha^{O(k)})$ bits of space, leading to the claimed space complexity for the decoding.

Time complexity. Throughout the recursion of $\text{Decompose}(V, 0)$, Algorithms 3 and 4 make at most $\text{poly}(n)$ calls to $\text{BalSparseCut}_{\alpha,\lambda}$. As noted above, each of these calls is on a graph with at most n vertices and $\tilde{O}(n/b^2)\alpha^{O(k)}$ edges. Lemma 4.2 also guarantees that decoding the linear sketch to get a sample from $\mathcal{D}_\delta$ and $\mathcal{D}_{\delta_j}$ takes $\text{poly}(n)$ time. Hence the running time of the decoding is $\text{poly}(n)\cdot T(n, \tilde{O}(n/b^2)\alpha^{O(k)})$.

Sparsity parameter in BSCA calls. By the way we set ϕ and by definition of the algorithm, we have that every call to $\text{BalSparseCut}_{\alpha,\lambda}(\cdot,\psi)$ is made with a sparsity parameter $\psi \in (0,1)$ that is $\psi \le \phi \le \frac{1}{10\alpha}$.

□

7 Lower bound for two-level expander decomposition

The goal of this section is to show that repeatedly computing EDs on the inter-cluster edges necessarily incurs a dependence on the sparsity parameter. Formally, the result is the following.

Theorem 3 (RED lower bound). Let $\ell \ge 2$ and let $\epsilon, \phi \in (0,1)$ such that $\epsilon = 1 - \Omega(1)$, $\phi \le \epsilon$, and $\phi \ge C \cdot \max\{\epsilon^2, 1/n\}$ for a large enough constant $C > 0$. Any streaming algorithm that with probability at least $9/10$ computes an ℓ-level (ϵ,ϕ)-RED requires $\Omega(n/\epsilon)$ bits of space.

By Yao's minimax principle, it is sufficient to give a distribution $\mathcal{G}$ over graphs with vertex set V such that any deterministic streaming algorithm that computes, with probability at least 0.9, an (ϵ,ϕ,ℓ)-RED of $G \sim \mathcal{G}$ takes $\Omega(n/\epsilon)$ bits of space. We are not going to use the fact that the stream can be dynamic to show the lower bound, so any stream that reveals the edges of $G \sim \mathcal{G}$ in an arbitrary order serves our purposes.

We then proceed as follows. In Section 7.1 we describe our hard distribution over graphs, and show a structural property of its two-level (ϵ,ϕ)-RED. In Section 7.2 we introduce a communication problem, recover, and show a reduction to RED. Finally, in Section 7.3 we give a communication lower bound for recover. We combine these results into proving Theorem 3 in Section 7.4.

7.1 Hard distribution

The randomness of our hard distribution $\mathcal{G}$ arises from sampling Erdős-Rényi graphs. We recall the definition for convenience of the reader.

Definition 7.1. Let N be an integer and let $p \in [0,1]$. The distribution of Erdős-Rényi graphs $\text{ER}(N,p)$ is the distribution over N-vertex graphs $H = (U,F)$ where each pair $e \in \binom{U}{2}$ is in F with probability p independently of other pairs.

Let d, m be integers such that $3 \le d < m < n$. We now define our hard distribution $\mathcal{G}$ over n-vertex graphs $G = (V,E)$, and we illustrate it in Figure 4.

Definition 7.2 (Distributions $\mathcal{G}$ and $\mathcal{G}'$). We partition $V = [n]$ arbitrarily into two sets S and T with $n/2$ vertices each, and further partition S into n/m sets $S_1, \ldots, S_{n/m}$ with $m/2$ vertices each. The distribution $\mathcal{G}_{n,d,m}$ is supported over graphs $G = (V, E)$ where E is defined as follows.

1. For each $i \in [n/m]$, the induced subgraph $G[S_i]$ is sampled independently from the Erdős-Rényi distribution $\mathrm{ER}(m/2, 4d/m)$ (see Definition 7.1 above). We denote by $\mathcal{G}'_{n,d,m}$ the distribution of the subgraph $G[S]$, i.e. $\mathcal{G}'_{n,d,m} = \mathrm{ER}(m/2, 4d/m)^{\otimes n/m}$.
2. The induced subgraph $G[T]$ is a fixed d-regular ψ-expander, for $\psi = \Omega(1)$ independent of d, m, n.
3. We fix for convenience an arbitrary labelling $s_{i,1}, \ldots, s_{i,m/2}$ of the vertices in each S_i, $i \in [n/m]$, and we sample $K \sim \mathrm{UNIF}([m/2])$. Then, for every $i \in [n/m]$, we add $dm/2$ edges from $s_{i,K}$ to T so that each $t \in T$ has d incident edges in $E(S, T)$ (one way to do so is by partitioning T into $n/(dm)$ subsets $T_1, \ldots, T_{n/(dm)}$ with $dm/2$ vertices each and connecting $s_{i,K}$ to each vertex of $T_{\lceil i/d \rceil}$ for every $i \in [n/m]$).

Since the parameters n, d, m are fixed hereafter, we may drop the subscript from $\mathcal{G}_{n,d,m}$ and $\mathcal{G}'_{n,d,m}$ and simply denote them by $\mathcal{G}$ and $\mathcal{G}'$ respectively.

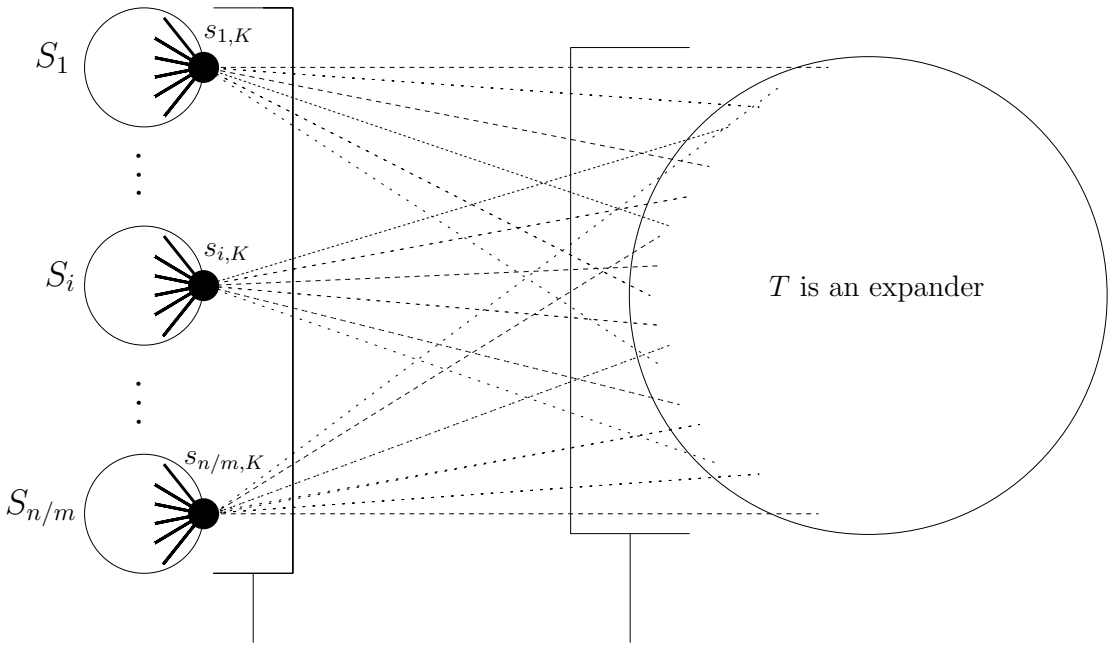


Figure 4: Illustration of the graph we use for proving the lower bound. Thick bullets represent important vertices, thick lines represent important edges, dotted lines represent edges connecting the important vertices to T.

Roughly speaking, our hard instances should be composed of n/m regular expanders that are densely connected to T, which is also an expander, through a selection of "special" vertices. We will show that the hardness arises from recovering information about certain important vertices and edges, defined below and also illustrated in Figure 4.

Definition 7.3 (Important vertices and edges). Let $G = (V, E) \in \mathrm{supp}(\mathcal{G})$. Denote by $k \in [m/2]$ the unique index such that $\{s_{i,k} : i \in [n/m]\} = \{s \in S : \{t \in T : \{s, t\} \in E\} \neq \emptyset\}$. Then, we define the set of important vertices $V^* = \{s_{i,k} : i \in [n/m]\}$ to be the set of vertices of S that are adjacent to T, and define the set of important edges $E^* = \{\{u, v^*\} : u \in S\,, v^* \in V^*\}$ to be the set of edges in the induced subgraph $G[S]$ that are incident on V^*.

Remark 1. Note that any sample $G \sim \mathcal{G}$ can be defined by (G', K), where $G' = (S, E')$ is the graph consisting of the disjoint union of the subgraphs $(G[S_i])_{i\in[n/m]}$, and $K \in [m/2]$ is the random index determining the important vertices and edges. This is because the edges within $G[T]$ are the same for all $G \sim \mathcal{G}$, and the edges $E(S, T)$ are determined by K. By virtue of this observation we abuse notation and write $G = (G', K) \sim \mathcal{G}$. Moreover, for any $(G', K) \sim \mathcal{G}$ one has $G' \sim \mathcal{G}'$, and conversely, for any $G' \sim \mathcal{G}'$ and $K \sim \mathrm{UNIF}([m/2])$ one also has $(G', K) \sim \mathcal{G}$.

In the remainder of the section, we call a graph $(1 \pm \delta)\Delta$-regular for an integer $\Delta > 0$ and $\delta \in [0, 1]$, if each of its vertices has degree in the range $[(1 - \delta)\Delta, (1 + \delta)\Delta]$. With this terminology, we show that in any valid (ϵ, ϕ)-ED of G, most of the important edges are crossing edges, as depicted in Figure 5. This works under the assumption that each $G[S_i]$ is a near-regular expander.

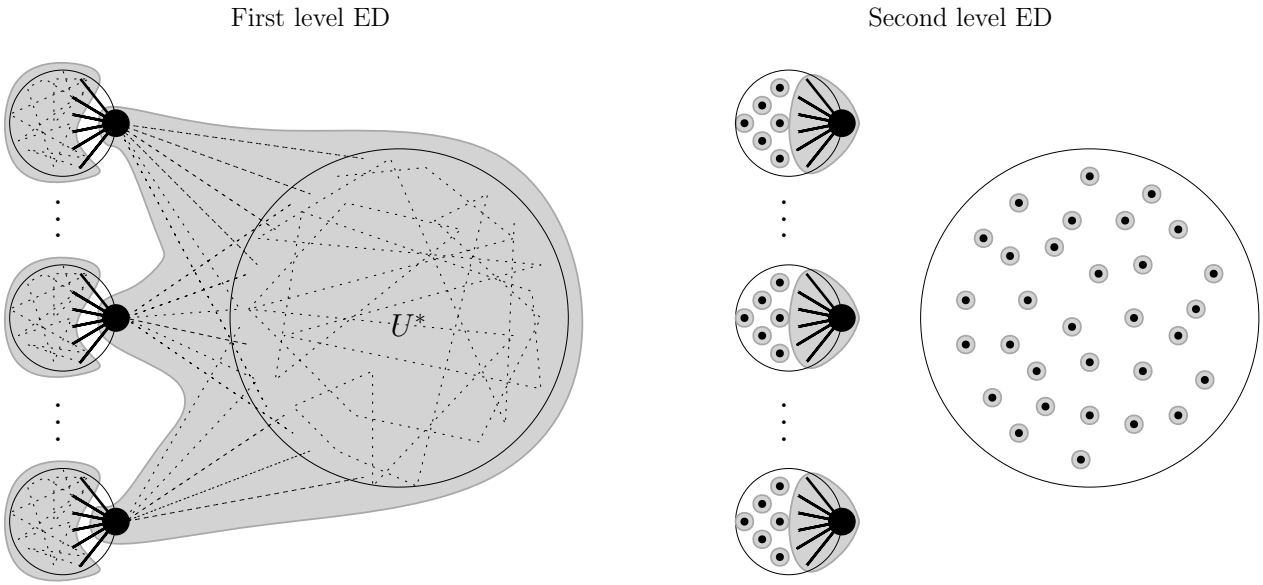


Figure 5: Illustration of the ideal expander decomposition of the graph. Thick bullets represent important vertices, smaller bullets represent ordinary vertices, thick lines represent important edges, dotted lines represent the rest of the edges. Grey areas represent the clusters in the decomposition.

Lemma 7.4. Let $G = (V, E) \in \mathrm{supp}(\mathcal{G})$, and let $\epsilon, \phi \in (0, 1)$ such that $\epsilon \leq 10^{-5} \cdot \psi \cdot \frac{d}{m}$ and $\phi \geq 11/m$. If $n \geq 100m$, $m \geq 500$, and every subgraph $(G[S_i])_{i\in[n/m]}$ is a $(1 \pm 1/10)2d$-regular $\frac{1}{3}$-expander, then any (ϵ, ϕ)-ED $\mathcal{U}$ of G satisfies

$$|E^* \setminus \mathcal{U}| \geq \frac{4}{5} \cdot |E^*| \,.$$

Proof. Let $G = (V, E) \in \mathrm{supp}(\mathcal{G})$ and let $k \in [m/2]$ such that the important vertices are $s_{i,k}$ for $i \in [n/m]$ (note that such value of k can be uniquely inferred from seeing G since the ordering of the vertices is fixed). Note that by Definition 7.2 together with our assumption, the graph G has at most $11/10 \cdot dn/2 + 3/4 \cdot dn \leq 1.3dn$ edges.

Hereafter, let $\mathcal{U}$ be any (ϵ, ϕ)-ED of G. We begin by arguing that there is a giant cluster in $\mathcal{U}$ that covers most of T and most of the important vertices.

Claim 7.5. There is a component $U^* \in \mathcal{U}$ such that

$$|T \cap U^*| \geq \left(1 - 12\frac{\epsilon}{\psi}\right)|T| \quad \text{and} \quad |V^* \cap U^*| \geq \left(1 - 12\frac{\epsilon}{\psi}\right)|V^*|\,.$$

Proof. For any valid (ϵ, ϕ)-ED $\mathcal{U}$, we must have $|E \setminus \mathcal{U}| \leq \epsilon \cdot 1.3dn$. Since $G[T]$ is a d-regular ψ-expander, there is a component $U^* \in \mathcal{U}$ that contains at least $(1 - 3\epsilon/\psi) \cdot n/2$ vertices from T, i.e.

$$|T \cap U^*| \geq \left(1 - 3\frac{\epsilon}{\psi}\right) \cdot \frac{n}{2} = \left(1 - 3\frac{\epsilon}{\psi}\right)|T|\,.$$

To see why, note that if this was not the case then we could construct a cut $S = \cup_{U \in \mathcal{U}'} U \cap T$ for some $\mathcal{U}' \subseteq \mathcal{U}$ such that $3\epsilon/\psi \cdot |T| < |S| \leq |T|/2$ and conclude that there would be at least $\psi \operatorname{vol}_{G[T]}(S) > \epsilon \cdot 1.5dn$ edges crossing $\mathcal{U}$.

Since each vertex in T has an edge to d important vertices (see Definition 7.2), there are at least $(1 - 6\epsilon/\psi) \cdot n/m$ important vertices with at least $dm/4$ neighbours in $U^* \cap T$, i.e.

$$\left|\left\{s \in V^* : |\{t \in U^* \cap T : \{s,t\} \in E\}| \geq \frac{1}{4}dm\right\}\right| \geq \left(1 - 6\frac{\epsilon}{\psi}\right) \cdot \frac{n}{m} = \left(1 - 6\frac{\epsilon}{\psi}\right) \cdot |V^*|\,.$$

To see why, note that if this was not the case, we would have $|\{s,t\} \in E : s \in V^*, t \in T \setminus U^*\}|$ to be larger than $(6\epsilon/\psi \cdot n/m) \cdot dm/4$, which is impossible since every $t \in T$ has d incident edges in $E(S,T)$ and $|T \setminus U^*| \leq 3\epsilon/\psi \cdot n/2$.

Now, because most of the important vertices have many edges to $T \cap U^*$, most of these should also be in U^*, otherwise too many edges would cross: since $|E \setminus \mathcal{U}| \leq \epsilon \cdot 1.3dn$, there must be at most $6\epsilon/\psi \cdot n/m + \frac{\epsilon \cdot 1.3dn}{dm/4} \leq 12\epsilon/\psi \cdot n/m$ important vertices that are not in U^*, i.e.

$$|V^* \cap U^*| \geq \left(1 - 12\frac{\epsilon}{\psi}\right) \cdot \frac{n}{m} = \left(1 - 12\frac{\epsilon}{\psi}\right) \cdot |V^*|\,.$$

□

We want to confine our attention to important vertices whose corresponding subgraph $G[S_i]$ locally preserves property (1) of Definition 1.1, i.e. has few inter-cluster edges as compared to its own volume. Formally, we consider the set of good vertices, defined as

$$V_{\mathrm{g}}^* = \left\{s_{i,k} \in V^* : s_{i,k} \in U^*, \left|(E \setminus \mathcal{U}) \cap \binom{S_i}{2}\right| \leq 200\epsilon dm\right\}. \tag{7.1}$$

We next show that the subgraph $G[S_i]$ corresponding to a good vertex is covered almost entirely by a single cluster, except the good vertex itself which is assigned to U^*.

Claim 7.6. Let $s_{i,k} \in V_{\mathrm{g}}^*$. Then, there is a cluster $U \in \mathcal{U}$ such that

$$s_{i,k} \notin U \quad \text{and} \quad |U \cap S_i| \geq (1 - 700\epsilon)\,|S_i|\,.$$

Proof. Since $G[S_i]$ is a $(1 \pm 1/10)2d$-regular $\frac{1}{3}$-expander by assumption, there is at least one component $U \in \mathcal{U}$ containing at least $(1 - 700\epsilon)m/2$ vertices in S_i, i.e.

$$|U \cap S_i| \geq (1 - 700\epsilon)\,\frac{m}{2} = (1 - 700\epsilon)\,|S_i|\,, \tag{7.2}$$

To see why, note that if this was not the case we could construct a cut $R = \cup_{W \in \mathcal{U}'} W \cap S_i$ for some $\mathcal{U}' \subseteq \mathcal{U}$ such that $10/9 \cdot 700\epsilon \cdot |S_i| < |R| \leq 9/11 \cdot |S_i|/2$ and conclude that there would be at least $1/3 \cdot \mathrm{vol}_{G[S_i]}(R) > 230\epsilon dm$ edges crossing $\mathcal{U}$ inside S_i, contradicting the fact that $s_{i,k} \in V_{\mathrm{g}}^*$ (see the definition in (7.1)).

Hereafter, let $U \in \mathcal{U}$ be the cluster that satisfies (7.2) (which is unique, since $\epsilon \leq 10^{-5}$). We know from Claim 7.5 that there is a cluster $U^* \in \mathcal{U}$ that contains all good vertices (by definition of good vertex, see (7.1)) and that covers most of T. To show that $s_{i,k} \notin U$, we leverage the fact that $s_{i,k} \in U^*$ and that $G[U^*]$ must be a ϕ-expander by property (2) of Definition 1.1. Suppose towards a contradiction that $U^* = U$, and let $Q = (U^* \cap S_i) \setminus \{s_{i,k}\}$. Because $G[S_i]$ is $(1 \pm 1/10)2d$-regular, the volume of this set in $G[U^*]$ is at least

$$\begin{aligned}\mathrm{vol}_{G[U^*]}(Q) &\geq -\deg_{G[S_i]}(s_{i,k}) + \sum_{u \in U^* \cap S_i} \deg_{G[U^* \cap S_i]}(u) \\ &\geq -\frac{11}{10}2d + \sum_{u \in U^* \cap S_i \colon \deg_{G[U^* \cap S_i]}(u) \geq (1-600\epsilon)\deg_{G[S_i]}(u)} \deg_{G[U^* \cap S_i]}(u)\,.\end{aligned}$$

We next observe that the sum above has at least $|U^* \cap S_i|/2$ many terms, for if this was not the case we would have at least $300\epsilon |U^* \cap S_i| \cdot 9/10 \cdot 2d \geq 200\epsilon dm$ edges from $U^* \cap S_i$ to $S_i \setminus U^*$, which would contradict $s_{i,k} \in V_{\mathrm{g}}^*$ (see the definition in (7.1)). We then continue from above and hence get

$$\begin{aligned}\mathrm{vol}_{G[U^*]}(Q) &\geq -\frac{11}{10}2d + \frac{1}{2}|U^* \cap S_i| \cdot (1-600\epsilon)\deg_{G[S_i]}(u) \\ &\geq -\frac{11}{10}2d + \frac{1}{2}|U^* \cap S_i| \cdot (1-600\epsilon)\,\frac{9}{10}2d \\ \text{since } |S_i \cap U^*| \geq (1-700\epsilon)|S_i| \quad &\geq -\frac{11}{10}2d + \frac{9}{10}2d\frac{1}{2}\,(1-1400\epsilon)\,\frac{m}{2}\,.\end{aligned}$$

Moreover, we know that $|U^* \cap T| \geq (1 - 3\epsilon/\psi)n/2 \geq (1 - 1/8000)n/2$ by our assumption on ϵ, so the fact that $G[T]$ is d-regular implies $\mathrm{vol}_{G[U^*]}(U^*) \geq \mathrm{vol}_{G[U^* \cap T]}(U^* \cap T) \geq dn/10$. Also, by near-regularity of G and because $m \leq n/100$, we have $\mathrm{vol}_{G[U^*]}(Q) \leq 3d \cdot m/2 < dn/20 \leq \frac{1}{2}\,\mathrm{vol}_{G[U^*]}(U^*)$. This means that Q is the smaller side of the cut. On the other hand, the number of edges crossing the cut Q in $G[U^*]$ is too small, since vertices in Q can only connect to the rest of U^* through edges incident on $s_{i,k}$, i.e.

$$\partial_{U^*} Q = \deg_{G[S_i]}(s_{i,k}) \leq \frac{11}{10}2d\,.$$

Therefore, $U_i = U^*$ implies

$$\Phi_{G[U^*]} \leq \Phi_{G[U^*]}(Q) = \frac{\partial_{U^*} Q}{\mathrm{vol}_{G[U^*]}(Q)} \leq \frac{\frac{11}{10}2d}{\frac{9}{10}d\,(1-1400\epsilon)\,\frac{m}{2} - \frac{11}{10}2d} \leq \frac{10}{m} < \phi\,,$$

a contradiction. Here we used that $m \geq 500$, $\phi \geq 11/m$, and $\epsilon \leq 10^{-5}$. □

By virtue of Claim 7.6, we have the following setting: for every good vertex $s_{i,k} \in V_{\mathrm{g}}^*$, there is one component U covering a $(1 - 700\epsilon)$ fraction of S_i except $s_{i,k}$ itself. Therefore, many important edges incident on each good vertex are crossing edges, i.e.

$$\{\{s, s'\} \in E^* : s = s_{i,k}, s' \notin U^*\} \subseteq E(U^* \cap S_i, S_i \setminus U^*) \subseteq E \setminus \mathcal{U} \quad \text{for all } s_{i,k} \in V_{\mathrm{g}}^*,$$

and

$$|U^* \cap S_i| \leq |U \setminus S_i| \leq 700\epsilon |S_i| = 700\epsilon \frac{m}{2} \quad \text{for all } s_{i,k} \in V_{\mathrm{g}}^*.$$

In particular, the above conditions implies that, among the important edges incident on each good vertex, at most $700\epsilon \cdot m/2$ of them can land in the same cluster U^* as the good vertex itself, i.e.

$$|\{\{s,s'\} \in E^* : s = s_{i,k}, s' \notin U^*\}| \geq \frac{9}{10}2d - 700\epsilon\frac{m}{2} \quad \text{for all } s_{i,k} \in V_{\mathrm{g}}^*. \tag{7.3}$$

If there are many good vertices, we can conclude that a lot of the important edges are crossing.

Claim 7.7. One has $|V_{\mathrm{g}}^*| \geq \frac{49}{50}|V^*|$.

Proof. Since the number of crossing edges is $|E \setminus \mathcal{U}| \leq \epsilon \cdot 1.3dn$, one has

$$\left|\left\{s_{i,k} \in V^* : \left|(E \setminus \mathcal{U}) \cap \binom{S_i}{2}\right| \leq 200\epsilon dm\right\}\right| \geq \left(1 - \frac{1}{100}\right)\frac{n}{m} = \left(1 - \frac{1}{100}\right)|V^*|\,. \tag{7.4}$$

Combining Claim 7.5 and the above bound, we know V_{g}^* to be large:

$$|V_{\mathrm{g}}^*| \geq \left(1 - 12\frac{\epsilon}{\psi} - \frac{1}{100}\right) \cdot |V^*| \geq \left(1 - \frac{1}{50}\right) \cdot |V^*|\,,$$

where we use the assumption that $\epsilon \leq 10^{-5} \cdot \psi \cdot \frac{d}{m}$. □

By Claim 7.7 and the bound in (7.3), we can conclude the proof:

$$|E^* \setminus \mathcal{U}| \geq \sum_{s_{i,k} \in V_{\mathrm{g}}^*} |\{\{s,t\} \in E^* : s = s_{i,k}\}| \geq \left(\frac{9}{10} - 700\epsilon\frac{m}{2}\frac{1}{2d}\right) 2d \cdot |V_{\mathrm{g}}^*|\,,$$

and by Claim 7.7 we get

$$|E^* \setminus \mathcal{U}| \geq \left(\frac{9}{10} - 700\epsilon\frac{m}{2}\frac{1}{2d}\right) 2d \cdot \frac{49}{50} \cdot \frac{n}{m} \geq \frac{4}{5} \cdot |E^*|\,,$$

where we used $\epsilon \leq 10^{-5}\psi\frac{d}{m}$ in the last inequality. □

7.2 Reduction to a communication problem

We introduce the two-party one-way communication problem recover, defined as follows.

Definition 7.8 (Communication problem recover). Let $\xi > 0$ be a real parameter. In the communication problem recover$_\xi$, Alice's input is a graph $G' = (S, E')$, and Bob's output is a set of pairs of vertices $F \subseteq \binom{S}{2}$ such that

1. $|F| \leq 6\xi \cdot |E'|$, and
2. at least a $1/10$ fraction of the pairs in F are edges of G', i.e. $|F \cap E'| \geq 1/10 \cdot |E'|$.

We give a reduction from recover$_{\epsilon m}$ to $(\epsilon, \phi, 2)$-RED, in the setting where Alice's input G' is an $n/2$-vertex graph drawn from $\mathcal{G}'$. We recall that $\mathcal{G}' = \mathcal{G}'_{n,d,m} = \mathrm{ER}(m/2, 4d/m)^{\otimes n/m}$ is the distribution over $n/2$-vertex graphs $G' = (S, E')$ consisting of the disjoint union of n/m disjoint Erdős-Rényi graphs on $m/2$ vertices (see Definition 7.2). We then begin by verifying that, with high probability, an input $G' \sim \mathcal{G}'$ is a collection of n/m nearly regular expanders.

Lemma 7.9. Let $G' \sim \mathcal{G}'$. If $m \geq 500$ and $d \geq 600 \log n$ then, with probability at least $1 - 1/n$, for every $i \in [n/m]$ one has that the subgraph $G'[S_i]$ is a $(1 \pm 1/10)2d$-regular $1/3$-expander

Proof. The statement follows from the following fact about Erdős-Rényi graphs, whose proof is deferred to Section A.2.

Fact 7.10. Let $N \geq 10$ be an integer and $p \in [0,1]$. Then, letting $\bar{d} = p(N-1)$ one has

$$\Pr_{H=(U,F)\sim \mathrm{ER}(N,p)} \left[\left(\Phi_H < \frac{1}{3} \right) \text{ or } \left(\exists u \in U \text{ s.t. } \left| \deg_H(u) - \bar{d} \right| > \frac{1}{11}\bar{d} \right) \right] \leq 4N \cdot \exp\left(-\frac{pN}{600} \right) .$$

In our case, each subgraph $G'[S_i]$ is drawn from $\mathrm{ER}(m/2, 4d/m)$, and note that $m/2 \geq 250 \geq 10$, thus meeting the precondition of Fact 7.10. Hence, for any $i \in [n/m]$ Fact 7.10 gives

$$\Pr \left[\left(\Phi_{G'[S_i]} < \frac{1}{3} \right) \text{ or } \left(\exists u \in S_i \text{ s.t. } \left| \deg_{G'[S_i]}(u) - 2d \right| > \frac{1}{10} 2d \right) \right] \leq 2m \cdot \exp\left(-\frac{d}{300} \right) ,$$

where we used $m \geq 500$ to express the degree bounds in terms of $2d$ instead of $\bar{d}$. Therefore, taking a union bound gives

$$\Pr \left[\forall\, i \in [n/m] \,, \left(\Phi_{G'[S_i]} \geq \frac{1}{3} \text{ and } \forall u \in S_i \,, \left| \deg_{G'[S_i]}(u) - 2d \right| \leq \frac{1}{10} 2d \right) \right] \geq 1 - \frac{1}{n} \,,$$

where we used $d \geq 600 \log n \geq 600 \ln n$. □

By virtue of the above, most of Alice's inputs can be embedded into a sample from $\mathcal{G}$ where a large fraction of important edges are inter-cluster edges of any ED, as per Lemma 7.4. This will allow Alice and Bob to solve $\mathrm{recover}_{\epsilon m}$.

Reduction 7.11. Let $\mathcal{A}$ be a deterministic streaming algorithm for computing a 2-level (ϵ, ϕ)-RED of a graph given in a stream of edges. For a graph $G' = (S, E') \sim \mathcal{G}'$, we reduce $\mathrm{recover}_{\epsilon m}$ to computing a 2-level (ϵ, ϕ)-RED as follows. Alice instantiates $\mathcal{A}$ for the vertex set V, and feeds her edges E' and the fixed edges of $G[T]$ to $\mathcal{A}$. Then, she sends the memory state Π of $\mathcal{A}$ to Bob. Then, Bob instantiates $m/2$ copies of $\mathcal{A}$, call them $\mathcal{A}_1, \ldots, \mathcal{A}_{m/2}$, and sets the memory state of them to Π. Next, for each $k \in [m/2]$ let $G_k = (V, E_k)$ be the graph defined by (G', k), and note that Bob knows the edges between S and T (see Definition 7.2). Then, Bob feeds the edges $E_k(S,T)$ to $\mathcal{A}_k$. For each $k \in [m/2]$, let $\mathcal{U}_1^k, \mathcal{U}_2^k$ be the output of $\mathcal{A}_k$. Bob finally constructs his output set F as follows: for each $k \in [m/2]$, if the number of non-isolated vertices in $\mathcal{U}_2^k$ is at most $3\epsilon dn$, add the pair $\{s, s_{i,k}\}$ to F for every $i \in [n/m]$ and every $s \in S_i$ that is not an isolated vertex in $\mathcal{U}_2^k$. More formally, for a partition $\mathcal{U}$ of V, let $V \setminus \mathcal{U} = \{u \in V : \{u\} \notin \mathcal{U}\}$ be the set of non-isolated vertices in $\mathcal{U}$. With this notation, Bob outputs

$$F = \bigcup_{k \in [m/2] : |V \setminus \mathcal{U}_2^k| \leq 3\epsilon dn} F_k \,, \qquad \text{with } F_k = \left\{ \{s, s_{i,k}\} : i \in [n/m],\, s \in S_i \cap (V \setminus \mathcal{U}_2^k) \right\} . \tag{7.5}$$

We remark that if Bob learns the important edges E_k^* for all $k \in [m/2]$, then he learns all of E'.

Remark 2. In the setting of Reduction 7.11, we have $E' = \bigcup_{k \in [m/2]} E_k^*$ and $|E'| = \frac{1}{2} \sum_{k \in [m/2]} |E_k^*|$.

As suggested by Remark 2, it is enough to ensure that each instance $\mathcal{A}_k$ recovers the important edges of G_k in order for Bob to learn the edges of G'. We in particular restrict ourselves to indices k for which the algorithm $\mathcal{A}_k$, the graph G_k, and the RED $\mathcal{U}_1^k, \mathcal{U}_2^k$ are well behaved, as defined below.

Definition 7.12. In the setting of Reduction 7.11, for every $k \in [m/2]$ let $\mathcal{P}_k$ be the property that

$$\mathcal{U}_1^k, \mathcal{U}_2^k \text{ is an } (\epsilon, \phi, 2)\text{-RED of } G_k \,, \quad \left| E_k^* \setminus \mathcal{U}_1^k \right| \geq \frac{4}{5} \cdot |E_k^*| \,, \quad |E_k^*| \geq \frac{9}{5} \frac{dn}{m} \,, \quad |E_k| \leq 1.3dn \,. \tag{7.6}$$

We first show a preliminary result: if there are many $k \in [m/2]$ that satisfy $\mathcal{P}_k$, then Bob's output set contains many edges of G'.

Lemma 7.13. In the setting of Reduction 7.11, if $\epsilon \in (0,1)$ satisfies $\epsilon \leq \frac{1}{30\sqrt{m}}$, then

$$|E' \cap F| \geq \frac{49}{100}\frac{dn}{m}|\{k \in [m/2] : \mathcal{P}_k\}|\,.$$

Proof. Recall from Reduction 7.11 that F is made by the union of F_k over $k \in [m/2]$. We then first give a guarantee on the F_k's individually.

Claim 7.14. Let $k \in [m/2]$ such that $\mathcal{P}_k$ holds. Then

1. $|V \setminus \mathcal{U}_2^k| \leq 3\epsilon dn$, and
2. $|F_k \cap (E_k^* \setminus \mathcal{U}_1^k)| \geq \frac{dn}{m} - \epsilon^2 \cdot 1.3dn$.

Proof of (1). Since $\mathcal{P}_k$ is verified, $\mathcal{U}_1^k$,$\mathcal{U}_2^k$ is an $(\epsilon, \phi, 2)$-RED of G_k (see Definition 7.12). By definition of RED (see Definition 3.2), we know $|E_k \setminus \mathcal{U}_1^k| \leq \epsilon|E_k|$. Moreover, one can note that the number of non-isolated vertices in $\mathcal{U}_2^k$ is at most twice the number of edges in $E_k \setminus \mathcal{U}_1^k$. This is because every non isolated vertex must have at least one edge incident on it in $E_k \setminus \mathcal{U}_1^k$ (which is the input to the second level ED), as otherwise property (2) of Definition 1.1 would be violated for any $\phi > 0$. Thus,

$$|V \setminus \mathcal{U}_2^k| \leq 2|E_k \setminus \mathcal{U}_1^k| \leq 2\epsilon|E_k| \leq 3\epsilon dn\,,$$

where the last inequality follows because $|E_k| \leq 1.3dn$ (see Definition 7.12). □

Proof of (2). From part (1), together with the assumptions that $\mathcal{U}_1^k$,$\mathcal{U}_2^k$ is an $(\epsilon, \phi, 2)$-RED of G_k and $|E_k| \leq 1.3dn$ (see Definition 7.12), we have

$$\left|V \setminus \mathcal{U}_2^k\right| \leq 3\epsilon dn \quad \text{and} \quad \left|E_k \setminus \mathcal{U}_2^k\right| = \left|\left(E_k \setminus \mathcal{U}_1^k\right) \setminus \mathcal{U}_2^k\right| \leq \epsilon \left|E_k \setminus \mathcal{U}_1^k\right| \leq \epsilon^2 \cdot |E_k| \leq \epsilon^2 1.3dn\,.$$

We want to argue that many of the important edges E_k^* of G_k are present in the set of pairs F_k (see Reduction 7.11). To do this, we make the following observation for any edge $\{s, s'\} \in E_k^* \setminus \mathcal{U}_1^k$: if none of s, s' is an isolated vertex in $\mathcal{U}_2^k$, i.e. $\{s\} \notin \mathcal{U}_2^k$ and $\{s'\} \notin \mathcal{U}_2^k$, then $\{s, s'\} \in F_k$; otherwise, if $\{s\} \in \mathcal{U}_2^k$ or $\{s'\} \in \mathcal{U}_2^k$, then $\{s, s'\} \in (E_k \setminus \mathcal{U}_1^k) \setminus \mathcal{U}_2^k$ (and this is because $\mathcal{U}_2^k$ is an ED of the graph $(V, E_k \setminus \mathcal{U}_1^k)$ and $\{s, s'\} \in E_k \setminus \mathcal{U}_1^k$ by assumption). Therefore,

$$|F_k \cap (E_k^* \setminus \mathcal{U}_1^k)| \geq |E_k^* \setminus \mathcal{U}_1^k| - |(E_k \setminus \mathcal{U}_1^k) \setminus \mathcal{U}_2^k|\,. \tag{7.7}$$

To conclude the claim, we use again the property $\mathcal{P}_k$: since $\mathcal{U}_1^k$,$\mathcal{U}_2^k$ is an $(\epsilon, \phi, 2)$-RED of G_k, we have

$$\left|(E_k \setminus \mathcal{U}_1^k) \setminus \mathcal{U}_2^k\right| \leq \epsilon|E_k \setminus \mathcal{U}_1^k| \leq \epsilon^2 \cdot |E_k|\,.$$

Using again $\mathcal{P}_k$, we know $|E_k^* \setminus \mathcal{U}_1^k| \geq \frac{4}{5} \cdot |E_k^*|$, $|E_k^*| \geq \frac{9}{5}dn/m$, and $|E_k| \leq 1.3dn$, so from (7.7) we conclude

$$|F_k \cap (E_k^* \setminus \mathcal{U}_1^k)| \geq |E_k^* \setminus \mathcal{U}_1^k| - |(E_k \setminus \mathcal{U}_1^k) \setminus \mathcal{U}_2^k| \geq \frac{4}{5} \cdot |E_k^*| - \epsilon^2 1.3dn \geq \frac{dn}{m} - \epsilon^2 1.3dn\,. \tag{7.8}$$

□

From the definition of Bob's output F (see Reduction 7.11) and by Remark 2, one has

$$|E' \cap F| = \left| \left(\bigcup_{k \in [m/2]} E^*_k \right) \cap \left(\bigcup_{k \in [m/2] : |V \setminus \mathcal{U}^k_2| \leq 3\epsilon dn} F_k \right) \right| \geq \left| \bigcup_{k \in [m/2] : |V \setminus \mathcal{U}^k_2| \leq 3\epsilon dn} (E^*_k \cap F_k) \right|$$

$$\text{by Remark 2} \quad \geq \frac{1}{2} \sum_{k \in [m/2] : |V \setminus \mathcal{U}^k_2| \leq 3\epsilon dn} |E^*_k \cap F_k| \ ,$$

and we finish using Claim 7.14:

$$|E' \cap F| \geq \frac{1}{2} \sum_{k \in [m/2] : |V \setminus \mathcal{U}^k_2| \leq 3\epsilon dn} |E^*_k \cap F_k|$$

$$\text{by part (1) of Claim 7.14} \quad \geq \frac{1}{2} \sum_{k \in [m/2] : \mathcal{P}_k \text{ holds}} |E^*_k \cap F_k| \ ,$$

and then

$$|E' \cap F| \geq \frac{1}{2} \sum_{k \in [m/2] : \mathcal{P}_k \text{ holds}} |E^*_k \cap F_k| \geq \frac{1}{2} \sum_{k \in [m/2] : \mathcal{P}_k \text{ holds}} \left| (E^*_k \setminus \mathcal{U}^k_1) \cap F_k \right| \ ,$$

so by part (2) of Claim 7.14 we have

$$|E' \cap F| \geq \frac{1}{2} \sum_{k \in [m/2] : \mathcal{P}_k \text{ holds}} \left(\frac{dn}{m} - \epsilon^2 \cdot 1.3dn \right) \geq \frac{49}{100} \frac{dn}{m} |\{k \in [m/2] : \mathcal{P}_k\}| \, ,$$

where we used $\epsilon \leq \frac{1}{30\sqrt{m}}$ in the last inequality. □

Finally, we prove that an efficient RED algorithm gives an efficient recover protocol, using the lemma we just showed.

Lemma 7.15. Let $\epsilon, \phi \in (0,1)$ such that $\epsilon \leq \min\{10^{-5}\psi \frac{d}{m}, \frac{1}{30\sqrt{m}}\}$ and $\phi \geq 11/m$. If $n \geq 100m$, $m \geq 500$, $d \geq 600 \log n$, and there is a deterministic L-bit space streaming algorithm $\mathcal{A}$ that computes an $(\epsilon, \phi, 2)$-RED with probability at least $9/10$ over inputs $G \sim \mathcal{G}$, then there is a deterministic protocol that solves $\text{recover}_{\epsilon m}$ with probability at least $3/5$ over inputs $G' \sim \mathcal{G}'$ with communication complexity at most L.

Proof. The protocol is given by Reduction 7.11. We observe that the communication complexity bound immediately follows, since $\mathcal{A}$ has space complexity L by assumption, and Alice only sends to Bob the state of the algorithm.

We now focus on proving correctness of the protocol with constant probability. We do this by first proving that F satisfies each of the two properties of Definition 7.8 provided that some conditions hold, and then verify that such conditions occur with constant probability.

Claim 7.16 (Property (1) of Definition 7.8). If $|\{k \in [m/2] : \mathcal{P}_k\}| \geq \frac{m}{3}$, then $|F| \leq 6 \cdot \epsilon m |E'|$.

Claim 7.17 (Property (2) of Definition 7.8). If $|\{k \in [m/2] : \mathcal{P}_k\}| \geq \frac{m}{3}$, then $|F \cap E'| \geq 1/10 \cdot |E'|$.

Claim 7.18. One has $\Pr_{G' \sim \mathcal{G}'} [|\{k \in [m/2] : \mathcal{P}_k\}| \geq m/3] \geq 3/5$.

By Claim 7.16 and Claim 7.17, the protocol given by Reduction 7.11 succeeds provided that the input G' satisfies $|\{k \in [m/2] : \mathcal{P}_k\}| \geq \frac{m}{3}$. By Claim 7.18, the latter condition is verified with probability at least 3/5. The lemma statement is then proven, modulo showing Claim 7.16, Claim 7.16, and Claim 7.18, which we do next.

Proof of Claim 7.16. As per Reduction 7.11, F is the union of $F_k = \{\{s, s_{i,k}\} : i \in [n/m],\ s \in S_i \cap (V \setminus \mathcal{U}_2^k)\}$ over k such that $|V \setminus \mathcal{U}_2^k| \leq 3\epsilon dn$. Note that $|\{\{s, s_{i,k}\} : i \in [n/m],\ s \in S_i \cap (V \setminus \mathcal{U}_2^k)\}|$ is in fact upper-bounded by $|V \setminus \mathcal{U}_2^k|$. One then gets that $|F| \leq 3\epsilon dn\frac{m}{2}$. In order to relate this bound to $|E'|$, we use our assumption: for at least $m/3$ values of $k \in [m/2]$, $|E_k^*| \geq 9/5 \cdot dn/m$. By Remark 2, this means $|E'| \geq 1/2 \cdot m/3 \cdot (9/5 \cdot dn/m) = 3/10 \cdot dn$, so one concludes $|F| \leq 6\epsilon |E'| m$. □

Proof of Claim 7.17. We can use Lemma 7.13, since our bound on ϵ ensures the precondition of the lemma. We then have $|E' \cap F| \geq \frac{49}{100}\frac{dn}{m}|\{k \in [m/2] : \mathcal{P}_k\}|$. Now, we conclude the claim by virtue of our assumption:

$$|E' \cap F| \geq \frac{49}{100}\frac{dn}{m}|\{k \in [m/2] : \mathcal{P}_k\}| \geq \frac{m}{3}\frac{49}{100}\frac{dn}{m} \geq \frac{1}{10}|E'|\,,$$

where in the last inequality we used that our assumption further implies $|E'| \leq |E_k| \leq 1.3dn$ for some $k \in [m/2]$. □

Proof of Claim 7.18. We recall that for $G' \sim \mathcal{G}'$ and $K \sim \mathrm{UNIF}([m/2])$ one has $(G', K) \sim \mathcal{G}$ (see Remark 1). For $k \in [m/2]$, we then interchangeably write $\mathcal{A}(G)$ and $\mathcal{A}(G', k)$ to refer to an execution of $\mathcal{A}$ on the graph G associated with (G', k). Our assumption on $\mathcal{A}$ rewrites as

$$\Pr_{G' \sim \mathcal{G},\, K \sim \mathrm{UNIF}([m/2])}\left[\mathcal{A}(G', K) \text{ fails}\right] = \Pr_{(G', K) \sim \mathcal{G}}\left[\mathcal{A}(G', K) \text{ fails}\right] \leq \frac{1}{10}\,.$$

Now recall Reduction 7.11: for each $k \in [m/2]$, the result of $\mathcal{A}_k$ computed by Bob is exactly $\mathcal{A}(G', k)$, where G' denotes Alice's input. Therefore, using that G' and K are independent and that K is drawn uniformly, we get

$$\mathbb{E}_{G' \sim \mathcal{G}'}\left[|\{k \in [m/2] :\ \mathcal{A}_k \text{ fails }\}|\right] = \mathbb{E}_{G' \sim \mathcal{G}'}\left[\left|\left\{k \in [m/2] :\ \mathcal{A}(G', k) \text{ fails }\right\}\right|\right] \leq \frac{1}{10} \cdot \frac{m}{2}\,.$$

Hence, if we sample $G' \sim \mathcal{G}'$, only a small fraction of the values of k will make (G', k) a failing instance. Specifically, for any $\alpha \in [1/10, 1]$, Markov's inequality gives

$$\Pr_{G' \sim \mathcal{G}'}\left[\left|\left\{k \in [m/2] :\ \mathcal{A}(G', k) \text{ fails }\right\}\right| > \frac{1}{\alpha}\frac{1}{10} \cdot \frac{m}{2}\right] \leq \alpha\,.$$

In other words, at least a $1 - \alpha$ fraction of the probability measure of $\mathcal{G}'$ yields a graph G' such that (G', k) is a succeeding instance for $\mathcal{A}$ with at least a $1 - 0.1/\alpha$ fraction of the possible values for $k \in [m/2]$. By Lemma 7.9 (which applies since we assume $m \geq 500$, $d \geq 600 \log n$), we also have that a $1 - 1/n$ fraction of the measure of $\mathcal{G}'$ yields a graph G' where each component is a $(1 \pm 1/10)2d$-regular $\frac{1}{3}$-expander. This in particular implies that at least a $1 - \alpha - 1/n$ fraction of the probability measure of $\mathcal{G}'$ yields a graph G' such that:

- for every $k \in [m/2]$, the graph $G_k = (V, E_k)$ given by (G', k) has $|E_k| \leq 1.3dn$, by near-regularity and Definition 7.2;
- for every $k \in [m/2]$, the graph $G_k = (V, E_k)$ given by (G', k) has $|E_k^*| \geq \frac{9}{5}\frac{dn}{m}$, by near regularity and Definition 7.2;

- with at least a $1 - 0.1/\alpha$ fraction of the possible values for $k \in [m/2]$ we have the following: $\mathcal{A}_k$ outputs an $(\epsilon, \phi, 2)$-RED $\mathcal{U}_1^k, \mathcal{U}_2^k$ of the graph $G = (V, E_k)$ given by (G', k). Moreover, by Lemma 7.4 (which applies, since our assumptions on n, m, ϵ, ϕ meet its preconditions) one has that $\mathcal{U}_1^k$ verifies the property

$$\left| E_k^* \setminus \mathcal{U}_1^k \right| \geq \frac{4}{5} \cdot |E_k^*| \,.$$

We have then obtained that at least a $1 - \alpha - 1/n$ fraction of the probability measure of $\mathcal{G}'$ yields a graph G' such that $\mathcal{P}_k$ holds for at least a $1 - 0.1/\alpha$ fraction of the possible values for $k \in [m/2]$. Setting $\alpha = 3/10$ gives the claim. □

□

7.3 Hardness of the communication problem

We show that $\mathrm{recover}_\xi$ requires linear communication in the number of edges when $G' \sim \mathcal{G}'$ and $\xi = \epsilon m$. Intuitively, this is because Bob is only allowed to output an $\approx \epsilon m$ factor more pairs than $|E'|$, and for an appropriate parameter regime $\epsilon m |E'| < \frac{1}{1000} \binom{m/2}{2} \frac{n}{m}$ (where $\binom{m/2}{2} \frac{n}{m}$ is the total number of vertex pairs in a graph consisting of n/m disjoint subgraphs on $m/2$ vertices).

Lemma 7.19. Let $\epsilon \in (0, 1)$ such that $\epsilon \leq 10^{-33}/d$. If $m \geq 500$ and $d \geq 600 \log n$, then any deterministic protocol that solves $\mathrm{recover}_{\epsilon m}$ with probability at least $3/5$ over inputs $G' \sim \mathcal{G}'$ requires $\Omega(dn)$ bits of communication.

Proof. Let us fix hereafter $\mathcal{Q}$ to be a deterministic protocol that solves $\mathrm{recover}_{\epsilon m}$ with probability at least $3/5$ over inputs $G' \sim \mathcal{G}'$. We define $\mathcal{I}^{(1)} \subseteq \mathrm{supp}(\mathcal{G}')$ to be the subset of input graphs $G' = (S, E')$ in the support of $\mathcal{G}'$ such that $\mathcal{Q}$ outputs correctly on input G'. We also partition $\mathrm{supp}(\mathcal{G}')$ based on the number of edges, so for every integer t we define $\mathcal{I}_t \subseteq \mathrm{supp}(\mathcal{G}')$ to be the subset of graphs $G' = (S, E') \in \mathrm{supp}(\mathcal{G}')$ with $|E'| = t$. We also let for convenience $\mathcal{I}_t^{(1)} = \mathcal{I}^{(1)} \cap \mathcal{I}_t$ be the subset of correct graphs with t edges. For any $G' = (S, E') \in \mathrm{supp}(\mathcal{G}')$, we denote by $M_{G'} \in \{0,1\}^*$ the message sent by Alice on input G' using the protocol $\mathcal{Q}$.

Claim 7.20. If t is an integer such that $t \in [9/10 \cdot dn/2,\, 11/10 \cdot dn/2]$ and $|\mathcal{I}_t^{(1)}| \geq 1/2 \cdot |\mathcal{I}_t|$, then

$$\mathsf{H}\left(M_{G'} \,\middle|\, G' \in \mathcal{I}_t^{(1)} \right) \geq \frac{dn}{3} \,.$$

Proof. For any $G' \in \mathcal{I}_t^{(1)}$, following the reception of $M_{G'}$, Bob deterministically outputs at most $6 \cdot (\epsilon m) \cdot |E'| = 6 \cdot \epsilon m t$ pairs of vertices $F \subseteq \binom{S}{2}$, such that at least $1/10 \cdot |E'| = t/10$ of them are actually edges of G'. Thus, for any t and any message $M \in \{0,1\}^{\leq nm}$ (where we restrained messages to be at most nm-bit long without loss of generality), the number of $G' \in \mathcal{I}_t^{(1)}$ such that $M_{G'} = M$ is at most

$$\begin{aligned}
\left| \left\{ G' \in \mathcal{I}_t^{(1)} : M_{G'} = M \right\} \right| &\leq \sum_{j=0.1t}^{t} \binom{6 \cdot \epsilon m t}{j} \cdot \binom{\frac{n}{m}\binom{m/2}{2}}{t-j} \\
&\leq \sum_{j=0.1t}^{t} \left(\frac{\mathrm{e} \cdot 6 \cdot \epsilon m t}{j} \right)^j \cdot \left(\frac{\mathrm{e} \cdot nm/8}{t-j} \right)^{t-j} \\
&= \mathrm{e}^t \sum_{j=0.1t}^{t} \left(\frac{60 \cdot \epsilon m t}{t} \right)^j \cdot \left(\frac{nm/8}{t} \right)^{t-j} \cdot \left(\frac{t}{t-j} \right)^{t-j} .
\end{aligned}$$

We employ the following fact to bound the above, whose proof is deferred to Section A.3.

Fact 7.21. For any $0 < x \le y$, $(y/x)^x \le \mathrm{e}^{y/\mathrm{e}}$.

We apply Fact 7.21 to the factor $(t/t-j)^{(t-j)}$ with $x = t-j$ and $y = t$, and hence we get

$$\left|\left\{G' \in \mathcal{I}_t^{(1)} : M_{G'} = M\right\}\right| \le \mathrm{e}^{(1+1/\mathrm{e})t} \sum_{j=0.1t}^{t} (60\epsilon m)^j \cdot \left(\frac{nm/8}{t}\right)^{t-j} \tag{7.9}$$

$$= \mathrm{e}^{(1+1/\mathrm{e})t} \left(\frac{nm}{8t}\right)^t \sum_{j=0.1t}^{t} \left(60 \cdot \frac{\epsilon 8t}{n}\right)^j \tag{7.10}$$

$$\le \mathrm{e}^{(1+1/\mathrm{e})t} \left(\frac{nm}{8t}\right)^t \left(500 \cdot \frac{\epsilon t}{n}\right)^{t/10} t\,, \tag{7.11}$$

where in the last inequality we used the upper bounds on ϵ and t. On the other hand, by virtue of our assumption that $|\mathcal{I}_t^{(1)}| \ge 1/2 \cdot |\mathcal{I}_t|$, one has

$$\left|\mathcal{I}_t^{(1)}\right| \ge \frac{1}{2}\left|\left\{G' = (S, E') \in \mathrm{supp}(\mathcal{G}') : |E'| = t\right\}\right| \tag{7.12}$$

$$\ge \frac{1}{2}\left|\left\{G' = (S, E') \in \mathrm{supp}(\mathcal{G}') : \forall\, i \in [n/m],\, \left|E' \cap \binom{S_i}{2}\right| = \frac{t}{n/m}\right\}\right| \tag{7.13}$$

$$= \frac{1}{2}\binom{\binom{m/2}{2}}{\frac{t}{n/m}}^{n/m} \ge \frac{1}{2}\binom{\frac{m^2}{16}}{\frac{t}{n/m}}^{n/m} \ge \frac{1}{2}\left(\frac{\frac{m^2}{16}}{\frac{t}{n/m}}\right)^{\frac{t}{n/m}\cdot n/m} = \frac{1}{2}\left(\frac{nm}{16t}\right)^t. \tag{7.14}$$

Next, we rewrite

$$\mathsf{H}\left(M_{G'} \,\middle|\, G' \in \mathcal{I}_t^{(1)}\right) \ge \mathsf{I}\left(G'; M_{G'} \,\middle|\, G' \in \mathcal{I}_t^{(1)}\right)$$
$$= \mathsf{H}\left(G' \,\middle|\, G' \in \mathcal{I}_t^{(1)}\right) - \mathsf{H}\left(G' \,\middle|\, M_{G'}, G' \in \mathcal{I}_t^{(1)}\right).$$

Now, observing that all the graphs with same number of edges have the same probability of being sampled, one can employ the upper and lower bounds derived above:

$$\mathsf{H}\left(G' \,\middle|\, G' \in \mathcal{I}_t^{(1)}\right) - \mathsf{H}\left(G' \,\middle|\, M_{G'}, G' \in \mathcal{I}_t^{(1)}\right)$$
$$\ge \log\left(\left|\mathcal{I}_t^{(1)}\right|\right) - \log\left(\max_{M \in \{0,1\}^{\le nm}} \left|\left\{H \in \mathcal{I}_t^{(1)} : M_H = M_{G'}\right\}\right|\right)$$
$$\text{by (7.14) and (7.11)} \ge t \log\left(\frac{\frac{1}{2}\frac{nm}{16t}}{\mathrm{e}^{(1+1/\mathrm{e})} \cdot \frac{nm}{8t} \cdot (t)^{1/t} \cdot \left(500 \cdot \frac{\epsilon t}{n}\right)^{1/10}}\right).$$

Then, we finish using $9/10 \cdot dn/2 \le t \le 11/10 \cdot dn/2$ and $\epsilon \le 10^{-33}/d$:

$$\mathsf{H}\left(M_{G'} \,\middle|\, G' \in \mathcal{I}_t^{(1)}\right) \ge t \log\left(\frac{1/120}{\left(500 \cdot \frac{\epsilon 11 dn}{20n}\right)^{1/10}}\right) \ge t \log 6 \ge \frac{9}{10}\frac{dn}{2} \ge \frac{dn}{3}.$$

□

Claim 7.22. One has

$$\Pr_{G'=(S,E') \sim \mathcal{G}'}\left[|E'| \in \left[\frac{9}{20}dn,\, \frac{11}{20}dn\right] \text{ and } \frac{\left|\mathcal{I}_{|E'|}^{(1)}\right|}{\left|\mathcal{I}_{|E'|}\right|} \ge \frac{1}{2}\right] \ge \frac{1}{10}.$$

Proof. We begin by defining for convenience

$$x = \Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\frac{\left|\mathcal{I}^{(1)}_{|E'|}\right|}{\left|\mathcal{I}_{|E'|}\right|} < \frac{1}{2}\right].$$

Then, by Lemma 7.9 (which applies by our assumption on m and d), one has

$$\Pr_{G'=(S,E')\sim\mathcal{G}'}\left[|E'| \in \left[\frac{9}{20}dn,\ \frac{11}{20}dn\right] \text{ and } \frac{\left|\mathcal{I}^{(1)}_{|E'|}\right|}{\left|\mathcal{I}_{|E'|}\right|} \geq \frac{1}{2}\right] \geq 1 - \frac{1}{n} - x\,. \tag{7.15}$$

We are left with the task of upper-bounding the probability x. Letting $\mathcal{T} = \{0, \ldots, \binom{m/2}{2}\frac{n}{m}\}$ be the set of possible numbers of edges in $G' \sim \mathcal{G}'$, we have

$$\begin{aligned}
&\Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\mathcal{Q} \text{ outputs correctly on input } G'\right] \\
&= \Pr_{G'=(S,E')\sim\mathcal{G}'}\left[G' \in \mathcal{I}^{(1)}\right] \\
&= \sum_{t\in\mathcal{T}} \Pr_{G'=(S,E')\sim\mathcal{G}'}[|E'| = t] \Pr_{G'=(S,E')\sim\mathcal{G}'}[G' \in \mathcal{I}^{(1)} \,|\, |E'| = t] \\
&= \sum_{t\in\mathcal{T}} \Pr_{G'=(S,E')\sim\mathcal{G}'}[|E'| = t] \cdot \frac{\left|\mathcal{I}^{(1)}_t\right|}{|\mathcal{I}_t|}\,,
\end{aligned}$$

where the last equality holds because all the graphs with t edges have the same probability of being sampled. Hence, splitting the sum one has

$$\begin{aligned}
&\Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\mathcal{Q} \text{ outputs correctly on input } G'\right] \\
&\leq \sum_{\substack{t\in\mathcal{T}: \\ |\mathcal{I}^{(}_t 1)| < \frac{1}{2}|\mathcal{I}^{(}_t 1)|}} \Pr_{G'=(S,E')\sim\mathcal{G}'}[|E'| = t] \cdot \frac{1}{2} + \sum_{\substack{t\in\mathcal{T}: \\ |\mathcal{I}^{(}_t 1)| \geq \frac{1}{2}|\mathcal{I}^{(}_t 1)|}} \Pr_{G'=(S,E')\sim\mathcal{G}'}[|E'| = t] \cdot 1 \\
&= \Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\frac{\left|\mathcal{I}^{(1)}_{|E'|}\right|}{\left|\mathcal{I}_{|E'|}\right|} < \frac{1}{2}\right] \cdot \frac{1}{2} + \Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\frac{\left|\mathcal{I}^{(1)}_{|E'|}\right|}{\left|\mathcal{I}_{|E'|}\right|} \geq \frac{1}{2}\right] \cdot 1 \\
&= x \cdot \frac{1}{2} + (1-x) \cdot 1 = 1 - \frac{x}{2}\,.
\end{aligned}$$

If we suppose for the sake of a contradiction that $x \geq 449/500$, then

$$\Pr_{G'=(S,E')\sim\mathcal{G}'}\left[\mathcal{Q} \text{ outputs correctly on input } G'\right] \leq \frac{551}{1000} < \frac{3}{5}\,,$$

which contradicts our assumption on $\mathcal{Q}$. Therefore, we plug $x < 449/500$ into (7.15) together with the assumption $n > m \geq 500$, thus getting the claim. □

Let now $\mathcal{C}$ be the event that $|E'| \in [9/10 \cdot dn/2,\ 11/10 \cdot dn/2]$ and $|\mathcal{I}^{(1)}_{|E'|}| \geq 1/2 \cdot |\mathcal{I}_{|E'|}|$. The size of Alice's message on input $G' = (S, E') \sim \mathcal{G}'$ is lower bounded by

$$\begin{aligned}
\mathsf{H}\left(M_{G'}\right) &\geq \mathsf{H}\left(M_{G'} \,\middle|\, \mathbb{1}_{\mathcal{C}}, |E'|\right) \\
&\geq \Pr\left[\mathcal{C}\right] \sum_{\substack{t\in\left[\frac{9}{20}dn,\ \frac{11}{20}dn\right]\cap\mathbb{N}: \\ \left|\mathcal{I}^{(1)}_t\right| \geq 1/2\cdot|\mathcal{I}_t|}} \Pr\left[|E'| = t \,\middle|\, \mathcal{C}\right] \cdot \mathsf{H}\left(M_{G'} \,\middle|\, \mathcal{C}, |E'| = t\right)\,,
\end{aligned}$$

and because conditioning does not increase entropy we rewrite

$$\begin{aligned}\mathsf{H}\left(M_{G'}\right) &\geq \Pr\left[\mathcal{C}\right] \sum_{\substack{\frac{9}{20}dn\leq t\leq \frac{11}{20}dn:\\ |\mathcal{I}_t^{(1)}|\geq 1/2\cdot|\mathcal{I}_t|}} \Pr\left[|E'|=t \,\middle|\, \mathcal{C}\right]\cdot \mathsf{H}\left(M_{G'} \,\middle|\, \mathbb{1}_{G'\in\mathcal{I}^{(1)}}, \mathcal{C}, |E'|=t\right)\\ &\geq \Pr\left[\mathcal{C}\right] \sum_{\substack{\frac{9}{20}dn\leq t\leq \frac{11}{20}dn:\\ |\mathcal{I}_t^{(1)}|\geq 1/2\cdot|\mathcal{I}_t|}} \Pr\left[|E'|=t \,\middle|\, \mathcal{C}\right]\cdot \Pr\left[G'\in\mathcal{I}^{(1)} \,\middle|\, |E'|=t\right]\cdot \mathsf{H}\left(M_{G'} \,\middle|\, G'\in\mathcal{I}_t^{(1)}\right).\end{aligned}$$

Finally, we use Claim 7.20 and Claim 7.22 and get

$$\begin{aligned}\mathsf{H}\left(M_{G'}\right) &\geq \Pr\left[\mathcal{C}\right] \sum_{\substack{\frac{9}{20}dn\leq t\leq \frac{11}{20}dn:\\ |\mathcal{I}_t^{(1)}|\geq 1/2\cdot|\mathcal{I}_t|}} \Pr\left[|E'|=t \,\middle|\, \mathcal{C}\right]\cdot \frac{1}{2}\cdot\frac{dn}{3}\\ &= \frac{dn}{6}\Pr_{G'=(S,E')\sim\mathcal{G}'}\left[|E'|\in\left[\frac{9}{20}dn,\,\frac{11}{20}dn\right] \text{ and } \frac{\left|\mathcal{I}^{(1)}_{|E'|}\right|}{\left|\mathcal{I}_{|E'|}\right|}\geq\frac{1}{2}\right]\\ &\geq \frac{dn}{60}.\end{aligned}$$

□

7.4 Proving the lower bound

Finally, we set d and m depending on ϵ, ϕ, n so as to combine Lemma 7.15 and Lemma 7.19 and conclude our streaming lower bound.

Theorem 3 (RED lower bound). Let $\ell \geq 2$ and let $\epsilon, \phi \in (0,1)$ such that $\epsilon = 1-\Omega(1)$, $\phi \leq \epsilon$, and $\phi \geq C\cdot\max\{\epsilon^2, 1/n\}$ for a large enough constant $C>0$. Any streaming algorithm that with probability at least $9/10$ computes an ℓ-level (ϵ,ϕ)-RED requires $\Omega(n/\epsilon)$ bits of space.

Proof. We recall that for any $\epsilon = 1-\Omega(1)$, computing an (ϵ,ϕ)-ED with constant probability requires $\Omega(n\log n)$ bits for all $\phi\in(0,\epsilon]$. To see why, consider an input graph that is a matching of size $n/10$, and notice that an ED is a $1-\epsilon$ fraction the edges. Therefore, we now consider the case $\epsilon = O(1/\log n)$ and prove an $\Omega(n/\epsilon)$ lower bound.

Let $\epsilon,\phi\in(0,1)$ be any ED parameters and define $c=\psi\cdot 10^{-40}$. Note that c is a constant since $\psi=\Omega(1)$ is the fixed sparsity from Definition 7.2. In order to apply Lemma 7.15 and Lemma 7.19, we use distributions $\mathcal{G}$ and $\mathcal{G}'$ from Definition 7.2 with d and m set as follows:

$$d = \frac{\epsilon}{2c^3\phi}+\frac{c}{2\epsilon} \quad\text{and}\quad m=\frac{11}{c\phi}. \tag{7.16}$$

We verify that these values meet the preconditions set by Lemma 7.15 and Lemma 7.19, i.e. we show that d and m satisfy

$$\epsilon\leq\min\left\{10^{-5}\psi\frac{d}{m},\,\frac{1}{30\sqrt{m}},\,\frac{10^{-33}}{d}\right\},\quad \phi\geq\frac{11}{m},\quad 500\leq m\leq\frac{n}{100},\quad 600\log n\leq d<m, \tag{7.17}$$

for all ϵ,ϕ such that

$$\epsilon\leq\frac{c^2}{\log n},\quad \phi\geq\frac{1}{c^2 n},\quad\text{and}\quad \frac{\epsilon^2}{c^4}\leq\phi\leq\epsilon. \tag{7.18}$$

Substituting d and m as defined in (7.16) into (7.17) we get

$$10^{-5}\psi\frac{d}{m} = 10^{-5}\psi\left(\frac{\epsilon}{2c^3\phi}+\frac{c}{2\epsilon}\right)\cdot\frac{c\phi}{11} \geq 10^{-5}\psi\frac{\epsilon}{2c^2 11} = 10^{75}\frac{\epsilon}{2\psi 11} \geq \epsilon\,,$$

$$\frac{1}{30\sqrt{m}} = \frac{\sqrt{c}\sqrt{\phi}}{30\sqrt{11}} \geq \frac{\sqrt{c}\epsilon}{30\sqrt{11}c^2} \geq \epsilon\,,$$

$$\frac{10^{-33}}{d} = 10^{-33}\frac{4c^3\epsilon\phi}{2\epsilon^2+2c^4\phi} = 10^{-33}\frac{4c^3\epsilon}{2\epsilon^2/\phi+2c^4} \geq 10^{-33}\frac{4c^3\epsilon}{2c^4+2c^4} = 10^{-33}\frac{\epsilon}{c} \geq \epsilon\,,$$

$$\frac{11}{m} = c\phi \leq \phi\,,$$

$$m = \frac{11}{c\phi} \geq \frac{11}{c\epsilon} \geq \log n \geq 500\,,$$

$$m = \frac{11}{c\phi} \leq 11cn \leq \frac{n}{100}\,,$$

$$d = \frac{\epsilon}{2c^3\phi}+\frac{c}{2\epsilon} \geq \frac{c}{2\epsilon} \geq \frac{\log n}{2c} \geq 600\log n\,,$$

$$d = \frac{\epsilon}{2c^3\phi}+\frac{c}{2\epsilon} \leq \frac{\epsilon}{2c^3\phi}+\frac{c}{2\phi} \leq \frac{1}{2c\log n\phi}+\frac{c}{2\phi} < \frac{11}{c\phi} = m\,,$$

so all the conditions (7.17) are verified.

With any ϵ, ϕ as in (7.18) and for our choice of d, m as in (7.17), Lemma 7.15 gives that if there is an L-bit space deterministic streaming algorithm $\mathcal{A}$ that outputs an $(\epsilon, \phi, 2)$-RED of $G \sim \mathcal{G}_{n,d,m}$ with probability at least $9/10$, then there is a deterministic protocol $\mathcal{Q}$ that solves $\text{recover}_{\epsilon m}$ with probability at least $3/5$ over inputs $G' \sim \mathcal{G}'$ with communication complexity L. On the other hand, by Lemma 7.19 we know that, in the parameter regime of (7.17), the complexity of $\mathcal{Q}$ over $\mathcal{G}'$ is at least $\Omega(dn)$. Hence,

$$L = \Omega(dn) = \Omega\left(\left(\frac{\epsilon}{\phi}+\frac{1}{\epsilon}\right)\cdot n\right) = \Omega\left(\frac{n}{\epsilon}\right)\,.$$

By Yao's minimax principle, the theorem statement is proved with $C = c^{-4}$.

□

References

[AALG18] Vedat Levi Alev, Nima Anari, Lap Chi Lau, and Shayan Oveis Gharan. Graph clustering using effective resistance. In Anna R. Karlin, editor, 9th Innovations in Theoretical Computer Science Conference, ITCS 2018, January 11-14, 2018, Cambridge, MA, USA, volume 94 of LIPIcs, pages 41:1–41:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[AG09] Kook Jin Ahn and Sudipto Guha. Graph sparsification in the semi-streaming model. In Susanne Albers, Alberto Marchetti-Spaccamela, Yossi Matias, Sotiris E. Nikoletseas, and Wolfgang Thomas, editors, Automata, Languages and Programming, 36th Internatilonal Colloquium, ICALP 2009, Rhodes, Greece, July 5-12, 2009, Proceedings, Part II, volume 5556 of Lecture Notes in Computer Science, pages 328–338. Springer, 2009.

[AGM12a] Kook Jin Ahn, Sudipto Guha, and Andrew McGregor. Analyzing graph structure via linear measurements. In Yuval Rabani, editor, Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012, pages 459–467. SIAM, 2012.

[AGM12b] Kook Jin Ahn, Sudipto Guha, and Andrew McGregor. Graph sketches: sparsification, spanners, and subgraphs. In Michael Benedikt, Markus Krötzsch, and Maurizio Lenzerini, editors, Proceedings of the 31st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2012, Scottsdale, AZ, USA, May 20-24, 2012, pages 5–14. ACM, 2012.

[AKLP22] Arpit Agarwal, Sanjeev Khanna, Huan Li, and Prathamesh Patil. Sublinear algorithms for hierarchical clustering. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022.

[APP+23] AmirMahdi Ahmadinejad, John Peebles, Edward Pyne, Aaron Sidford, and Salil P. Vadhan. Singular value approximation and sparsifying random walks on directed graphs. In 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023, pages 846–854. IEEE, 2023.

[BK15] András A. Benczúr and David R. Karger. Randomized approximation schemes for cuts and flows in capacitated graphs. SIAM J. Comput., 44(2):290–319, 2015.

[CGL+20] Julia Chuzhoy, Yu Gao, Jason Li, Danupon Nanongkai, Richard Peng, and Thatchaphol Saranurak. A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond. In Sandy Irani, editor, 61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020, pages 1158–1167. IEEE, 2020.

[CGP+18] Timothy Chu, Yu Gao, Richard Peng, Sushant Sachdeva, Saurabh Sawlani, and Junxing Wang. Graph sparsification, spectral sketches, and faster resistance computation, via short cycle decompositions. In Mikkel Thorup, editor, 59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, Paris, France, October 7-9, 2018, pages 361–372. IEEE Computer Society, 2018.

[CKK+06] Shuchi Chawla, Robert Krauthgamer, Ravi Kumar, Yuval Rabani, and D. Sivakumar. On the hardness of approximating multicut and sparsest-cut. Comput. Complex., 15(2):94–114, 2006.

[CKL+22a] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In 63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022, pages 612–623. IEEE, 2022.

[CKL22b] Yu Chen, Sanjeev Khanna, and Huan Li. On weighted graph sparsification by linear sketching. In 63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022, pages 474–485. IEEE, 2022.

[CPSZ21] Yi-Jun Chang, Seth Pettie, Thatchaphol Saranurak, and Hengjie Zhang. Near-optimal distributed triangle enumeration via expander decompositions. J. ACM, 68(3):21:1–21:36, 2021.

[CS19] Yi-Jun Chang and Thatchaphol Saranurak. Improved distributed expander decomposition and nearly optimal triangle enumeration. In Peter Robinson and Faith Ellen, editors, Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019, pages 66–73. ACM, 2019.

[FHHP11] Wai Shing Fung, Ramesh Hariharan, Nicholas J. A. Harvey, and Debmalya Panigrahi. A general framework for graph sparsification. In Lance Fortnow and Salil P. Vadhan, editors, Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC 2011, San Jose, CA, USA, 6-8 June 2011, pages 71–80. ACM, 2011.

[FKM23] Arnold Filtser, Michael Kapralov, and Mikhail Makarov. Expander decomposition in dynamic streams. In Yael Tauman Kalai, editor, 14th Innovations in Theoretical Computer Science Conference, ITCS 2023, January 10-13, 2023, MIT, Cambridge, Massachusetts, USA, volume 251 of LIPIcs, pages 50:1–50:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[GHKM23] Venkatesan Guruswami, Jun-Ting Hsieh, Pravesh K. Kothari, and Peter Manohar. Efficient algorithms for semirandom planted csps at the refutation threshold. In 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023, pages 307–327. IEEE, 2023.

[GHN+23] Gramoz Goranci, Monika Henzinger, Danupon Nanongkai, Thatchaphol Saranurak, Mikkel Thorup, and Christian Wulff-Nilsen. Fully dynamic exact edge connectivity in sublinear time. In Nikhil Bansal and Viswanath Nagarajan, editors, Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023, pages 70–86. SIAM, 2023.

[GRST21] Gramoz Goranci, Harald Räcke, Thatchaphol Saranurak, and Zihan Tan. The expander hierarchy and its applications to dynamic graph algorithms. In Dániel Marx, editor, Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021, pages 2212–2228. SIAM, 2021.

[HKGW23] Yiding Hua, Rasmus Kyng, Maximilian Probst Gutenberg, and Zihang Wu. Maintaining expander decompositions via sparse cuts. In Nikhil Bansal and Viswanath Nagarajan, editors, Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023, pages 48–69. SIAM, 2023.

[Ind06] Piotr Indyk. Stable distributions, pseudorandom generators, embeddings, and data stream computation. J. ACM, 53(3):307–323, 2006.

[Kar93] David R. Karger. Global min-cuts in rnc, and other ramifications of a simple min-cut algorithm. In Vijaya Ramachandran, editor, Proceedings of the Fourth Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms, 25-27 January 1993, Austin, Texas, USA, pages 21–30. ACM/SIAM, 1993.

[Kar94] David R. Karger. Random sampling in cut, flow, and network design problems. In Frank Thomson Leighton and Michael T. Goodrich, editors, Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing, 23-25 May 1994, Montréal, Québec, Canada, pages 648–657. ACM, 1994.

[KKTY21] Michael Kapralov, Robert Krauthgamer, Jakab Tardos, and Yuichi Yoshida. Towards tight bounds for spectral sparsification of hypergraphs. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 598–611. ACM, 2021.

[KLM+14] Michael Kapralov, Yin Tat Lee, Cameron Musco, Christopher Musco, and Aaron Sidford. Single pass spectral sparsification in dynamic streams. In 55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014, pages 561–570. IEEE Computer Society, 2014.

[KMM+20] Michael Kapralov, Aida Mousavifar, Cameron Musco, Christopher Musco, Navid Nouri, Aaron Sidford, and Jakab Tardos. Fast and space efficient spectral sparsification in dynamic streams. In Shuchi Chawla, editor, Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020, pages 1814–1833. SIAM, 2020.

[KVV00] Ravi Kannan, Santosh S. Vempala, and Adrian Vetta. On clusterings - good, bad and spectral. In 41st Annual Symposium on Foundations of Computer Science, FOCS 2000, 12-14 November 2000, Redondo Beach, California, USA, pages 367–377. IEEE Computer Society, 2000.

[Li21] Jason Li. Deterministic mincut in almost-linear time. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 384–395. ACM, 2021.

[LLR95] Nathan Linial, Eran London, and Yuri Rabinovich. The geometry of graphs and some of its algorithmic applications. Comb., 15(2):215–245, 1995.

[LNPS23] Jason Li, Danupon Nanongkai, Debmalya Panigrahi, and Thatchaphol Saranurak. Near-linear time approximations for cut problems via fair cuts. In Nikhil Bansal and Viswanath Nagarajan, editors, Proceedings of the 2023 ACM-SIAM Symposium on

Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023, pages 240–275. SIAM, 2023.

[LR99] Frank Thomson Leighton and Satish Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. J. ACM, 46(6):787–832, 1999.

[Nis92] Noam Nisan. Pseudorandom generators for space-bounded computation. Comb., 12(4):449–461, 1992.

[ST04] Daniel A. Spielman and Shang-Hua Teng. Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems. In László Babai, editor, Proceedings of the 36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004, pages 81–90. ACM, 2004.

[ST11] Daniel A. Spielman and Shang-Hua Teng. Spectral sparsification of graphs. SIAM J. Comput., 40(4):981–1025, 2011.

[SW19] Thatchaphol Saranurak and Di Wang. Expander decomposition and pruning: Faster, stronger, and simpler. In Timothy M. Chan, editor, Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019, pages 2616–2635. SIAM, 2019.

[Tre08] Luca Trevisan. Approximation algorithms for unique games. Theory Comput., 4(1):111–128, 2008.

[Tro15] Joel A. Tropp. An introduction to matrix concentration inequalities. Found. Trends Mach. Learn., 8(1-2):1–230, 2015.

A Technical facts

A.1 Polynomial time balanced sparse cut algorithm

In this section, we prove the following fast BSCA, used for our polynomial time BLD construction.

Theorem 6.2 ([SW19, LNPS23, LR99]). For $b \in (0,1)$, let $p \leq n$, $q \leq \min\{p^2, n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n\}$ and $W \leq \text{poly}(n)$ be integers. For p-vertex q-edge input graphs with weighted edges and weighted self-loops, and total volume bounded by W, there is a $(c_{\text{bsc}} \log^3 n, C_{\text{bsc}})$-BSCA that runs in $\text{poly}(n)$ time and $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ bits of space, uses $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$ random bits, and works with probability $1 - 1/\text{poly}(n)$ for every input sparsity parameter $0 < \psi \leq \frac{1}{10 c_{\text{bsc}} \log^3 n}$, where $c_{\text{bsc}}, C_{\text{bsc}} \geq 1$ are absolute constants.

Proof. First we show how to reduce the BSCA task on graphs with self-loops to graphs without self-loops. Let $G = (V, E, w)$ be a weighted undirected graph with one weighted self-loop per vertex, where $w : E \cup V \to \mathbb{R}_{\geq 0}$ and $E \subseteq \binom{V}{2}$. The idea is to construct a graph with two copies of every vertex, where the surplus vertices are solely connected to their corresponding vertex with weight equal to half the weight of the self-loop. If a vertex has no self-loops then we make only one copy for it. Formally, let $A = \{u \in V : w(u) > 0\}$ be the set of vertices with self-loops. Then let B be a copy of A representing the self-loops of vertices in A, and let $s : A \to B$ be a bijection mapping $u \in A$ to its corresponding self-loop vertex $s(u) \in B$. We define the self-loop free graph to be $\hat{G} = (\hat{V}, \hat{E}, \hat{w})$, where $\hat{V} = V \cup s(A)$, $\hat{E} = E \cup (\cup_{u \in A} \{\{u, s(u)\}\})$, and $\hat{w}(e) = w(e)$ for every $e \in E$ and $\hat{w}(\{u, s(u)\}) = \frac{1}{2} w(u)$ for all $u \in A$. This is illustrated in Figure 6. We highlight that $\hat{G}$ has the same number of vertices and edges as G, up to constants.

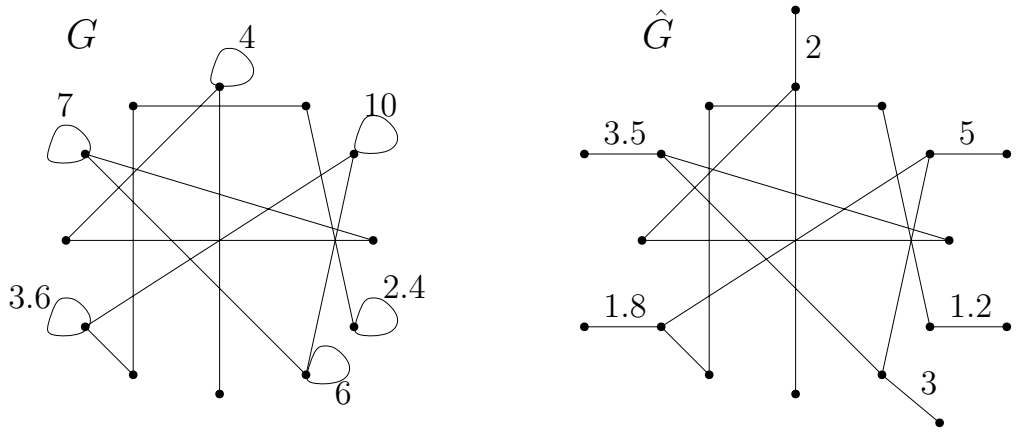


Figure 6: On the left: the original graph with self-loops G. On the right: its self-loop free version $\hat{G}$. The weights of the internal edges of G are copied verbatim (not shown in the figure), while the weights of self-loops are halved.

We run a BSCA for self-loop free graphs on $\hat{G}$, and if it asserts $\hat{G}$ to be an expander we conclude the same for G, whereas if it returns a cut then we tweak it to get a solution for G. This is summarised in Algorithm 5. The following claim gives the correctness of the reduction. The idea is that the subgraphs given by $\{u, s(u)\}$ for $u \in A$ are expanders, so there is no good reason for a sparse cut to cross them. Hence, u and $s(u)$ should be on the same side of the sparse cuts.

Claim A.1. Let $\text{BalSparseCut}_{\alpha,\lambda}$ be an (α, λ)-BSCA. Then SelfLoopBalSparseCut is a $(2\alpha, 4\lambda)$-BSCA, provided that the input sparsity parameter $\phi \in (0,1)$ satisfies $\phi \leq \frac{1}{10\alpha}$.

Proof. For any cut $\emptyset \neq X \subsetneq V$, let us define for convenience $\partial_G X = w(X, V \setminus X)$ to be the total weight of the edges crossing the cut X in G, and for any cut $\emptyset \neq R \subsetneq \hat{V}$ define $\partial_{\hat{G}} R = \hat{w}(R, \hat{V} \setminus R)$ to be the total weight of edges crossing R in $\hat{G}$. A preliminary observation is that for any cut $\emptyset \neq X \subsetneq V$ we have that $\mathrm{vol}_{\hat{G}}(X \cup s(X \cap A)) = \mathrm{vol}_G(X)$, $\partial_{\hat{G}}(X \cup s(X \cap A)) = \partial_G X$, and consequently $\Phi_{\hat{G}}(X \cup s(X \cap A)) = \Phi_G(X)$. By definition of $\mathrm{BalSparseCut}_{\alpha,\lambda}$, we know that $\hat{\omega}$ is a $(\phi, \alpha, \lambda, 0)$-BSCW of $\hat{G}$, i.e.

1. if $\hat{\omega} = \perp$, then $\hat{G}$ is a ϕ-expander;
2. if $\hat{\omega} = (R, \nu)$, then:
   (a) $\Phi_{\hat{G}}(R) < \alpha \cdot \phi$,
   (b) $\mathrm{vol}_{\hat{G}}(R) \leq \mathrm{vol}_{\hat{G}}(\hat{V} \setminus R)$,
   (c) for every other cut $\emptyset \neq T \subsetneq \hat{V}$ with $\Phi_{\hat{G}}(T) < \phi$ and $\mathrm{vol}_{\hat{G}}(T) \leq \mathrm{vol}_{\hat{G}}(\hat{V} \setminus T)$ one has $\mathrm{vol}_{\hat{G}}(T) \leq \lambda \, \mathrm{vol}_{\hat{G}}(R)$:
   (d) $\nu = \mathrm{vol}_{\hat{G}}(R)$.

Our goal is to translate these properties to G by showing that ω, as constructed in SelfLoopBalSparseCut, is a $(\phi, 2\alpha, 4\lambda, 0)$-BSCW of G, i.e.

1. if $\omega = \perp$, then G is a ϕ-expander;
2. if $\omega = (X^*, \nu)$, then:
   (a) $\Phi_G(X^*) < \alpha \cdot \phi$,
   (b) $\mathrm{vol}_G(X^*) \leq \mathrm{vol}_G(V \setminus X^*)$,
   (c) for every other cut $\emptyset \neq T \subsetneq V$ with $\Phi_G(T) < \phi$ and $\mathrm{vol}_G(T) \leq \mathrm{vol}_G(V \setminus T)$ one has $\mathrm{vol}_G(T) \leq \lambda \, \mathrm{vol}_G(X^*)$:
   (d) $\nu = \mathrm{vol}_G(X^*)$.

Correctness of the case $\hat{\omega} = \perp$. This means that for every cut $\emptyset \neq R \subsetneq \hat{V}$ one has $\Phi_{\hat{G}}(R) \geq \phi$. This in particular holds for cuts of the form $X \cup s(X \cap A)$ with $\emptyset \neq X \subsetneq V$. As we observed that $\Phi_{\hat{G}}(X \cup s(X \cap A)) = \Phi_G(X)$, we conclude that G is also a ϕ-expander.

Correctness of the case $\hat{\omega} = (R, \nu)$. The idea is that adding missing self-loops and removing dangling self-loops makes R a solution for G. This is illustrated in Figure 7.
We know from (2a) that

$$\frac{\partial_{\hat{G}} R}{\mathrm{vol}_{\hat{G}}(R)} < \alpha\phi \,.$$

Consider the cut $Q = R \cup s(R \cap A)$, i.e. a version of R where we have added the self-loops of all the vertices in R that have one. Then $Q \subsetneq \hat{V}$, as otherwise $V \subseteq R$, which would imply that R contains more than half the volume of $\mathrm{vol}_{\hat{G}}(\hat{V})$, contradicting (2b). Next, we remark that the volume of Q cannot decrease compared to R, since we added vertices, i.e.

$$\mathrm{vol}_{\hat{G}}(Q) = \mathrm{vol}_{\hat{G}}(R) + \partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B)) \geq \mathrm{vol}_{\hat{G}}(R) \,. \tag{A.1}$$

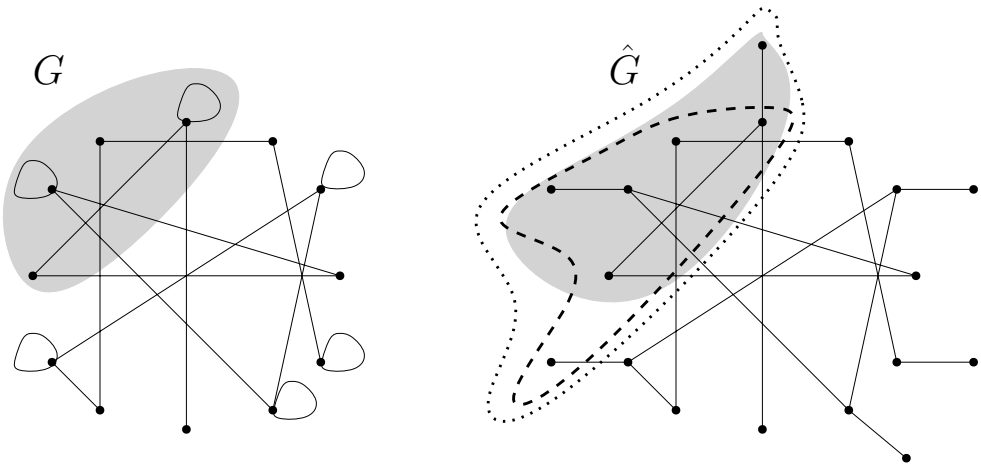


Figure 7: There is a straightforward correspondence between the shaded cuts in G and $\hat{G}$, since they have same volume and same number of crossing edges. The dashed cut has missing self-loops and dangling self-loops (think of it as the cut R). The dotted cut adds the missing self-loops (think of it as the cut Q). The shaded cut in $\hat{G}$ removes the dangling self-loops (think of it as the cut P), and corresponds to the shaded cut in G (think of it as the cut X).

At the same time, we do not increase the number of crossing edges since we move the self-loop edges inside the cut, so

$$\frac{\partial_{\hat{G}} Q}{\mathrm{vol}_{\hat{G}}(Q)} = \frac{\partial_{\hat{G}} R - \partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B))}{\mathrm{vol}_{\hat{G}}(R) + \partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B))} \leq \frac{\partial_{\hat{G}} R}{\mathrm{vol}_{\hat{G}}(R)} \,.$$

Thus, we have added self-loops to R without increasing its sparsity nor decreasing its volume. Now we remove the dangling self-loops from Q, i.e. we remove the set $D = (Q \cap B) \setminus s(Q \cap A)$, and consider the cut $P = Q \setminus D$. We remark that $P \neq \emptyset$, as otherwise R is solely made of vertices from $s(A)$, which contradicts $\Phi_H(R) < \alpha\phi \leq 1/10 < 1$. Since we removed an equal amount of mass from both the crossing edges and the volume, we get

$$\frac{\partial_{\hat{G}} P}{\mathrm{vol}_{\hat{G}}(P)} \leq \min\left\{\frac{\partial_{\hat{G}} P}{\mathrm{vol}_{\hat{G}}(P)}, 1\right\} \leq \frac{\partial_{\hat{G}} P + \partial_{\hat{G}} D}{\mathrm{vol}_{\hat{G}}(P) + \partial_{\hat{G}} D} = \frac{\partial_{\hat{G}} Q}{\mathrm{vol}_{\hat{G}}(Q)} \,.$$

Thus, removing the dangling self-loops does not increase sparsity. However we may have decreased the volume, which threatens the balancedness property of the cut. We then bound the decrease: because we proved before $\partial_{\hat{G}} Q / \mathrm{vol}_{\hat{G}}(Q) < \alpha\phi$, we know

$$\partial_{\hat{G}} D < \frac{\alpha\phi}{1 - \alpha\phi} \mathrm{vol}_{\hat{G}}(P) \,,$$

so

$$\mathrm{vol}_{\hat{G}}(Q) = \mathrm{vol}_{\hat{G}}(P) + \partial_{\hat{G}} D < 2\,\mathrm{vol}_{\hat{G}}(P) \,. \tag{A.2}$$

Now one can see that P is identical to $(R \cap V) \cup s(R \cap A)$, and X as defined in Algorithm 5 equals $R \cap V$. Hence $\partial_G X = \partial_{\hat{G}} P$ and $\mathrm{vol}_G(X) = \mathrm{vol}_{\hat{G}}(P)$. Therefore,

$$\frac{\partial_G X}{\mathrm{vol}_G(X)} < \alpha\phi \,. \tag{A.3}$$

To handle the case where X is the larger volume side of the cut, Algorithm 5 takes X^* to be the side of $(X, V \setminus X)$ with smaller volume. We show that the volume of $V \setminus X$ is within a small constant factor of the volume of X. To do so, recall that $\Phi_{\hat{G}}(R) < \alpha\phi$, so

$$\frac{\partial_{\hat{G}} Q + \partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B))}{\mathrm{vol}_{\hat{G}}(Q) - \partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B))} < \alpha\phi \,,$$

which implies

$$\partial_{\hat{G}}(s(R \cap A) \setminus (R \cap B)) < \alpha\phi \,\mathrm{vol}_{\hat{G}}(Q) \,.$$

Hence, by property (2b) of R, we have

$$\mathrm{vol}_G(X) \leq \mathrm{vol}_{\hat{G}}(Q) < \frac{1}{1-\alpha\phi}\,\mathrm{vol}_{\hat{G}}(R) \leq \frac{1}{2} \cdot \frac{1}{1-\alpha\phi}\,\mathrm{vol}_{\hat{G}}(\hat{V}) = \frac{1}{2} \cdot \frac{1}{1-\alpha\phi}\,\mathrm{vol}_G(V) \,.$$

Therefore, we have that $X^* = \mathrm{argmin}\{\mathrm{vol}_G(X), \mathrm{vol}_G(V \setminus X)\}$ satisfies

$$(1 - 2\alpha\phi)\,\mathrm{vol}_G(X) \leq \mathrm{vol}_G(X^*) \leq \mathrm{vol}_G(V \setminus X^*) \,, \tag{A.4}$$

and

$$\mathrm{vol}_{\hat{G}}(R) < \frac{2}{1-2\alpha\phi}\,\mathrm{vol}_G(X^*) \,, \tag{A.5}$$

by (A.1), (A.2). We now conclude by verifying properties (2a), (2b), (2c), (2d).

- Property (2a). From (A.3) and (A.4), we get $\Phi_G(X^*) < \alpha\phi/(1-2\alpha\phi) \leq 2\alpha\phi$.
- Property (2b). This property follows by definition of X^* in Algorithm 5.
- Property (2c). Consider any cut $\emptyset \neq T \subsetneq V$ such that $\Phi_G(T) < \phi$ and $\mathrm{vol}_G(T) \leq \mathrm{vol}_G(V \setminus T)$. Then we know that

  $$\Phi_G(T) = \Phi_{\hat{G}}(T \cup s(T \cap A)) < \phi \,,$$

  and

  $$\mathrm{vol}_{\hat{G}}(T \cup s(T \cap A)) = \mathrm{vol}_G(T) \leq \mathrm{vol}_G(V \setminus T) = \mathrm{vol}_{\hat{G}}(\hat{V} \setminus (T \cup s(T \cap A)) \,,$$

  as well as

  $$\mathrm{vol}_G(T) = \mathrm{vol}_{\hat{G}}(T \cup s(T \cap A)) \leq \lambda \cdot \mathrm{vol}_{\hat{G}}(R) < \frac{2\lambda}{1-2\alpha\phi}\,\mathrm{vol}_G(X^*) \leq 4\lambda\,\mathrm{vol}_G(X^*) \,,$$

  by an application of (2c) to the cut $T \cup s(T \cap A)$ and (A.5).
- Property (2d). Follows directly by definition of ω in Algorithm 5.

□

Using Claim A.1, we now conclude a proof of Theorem 6.2. To do so, we run two different algorithms, depending on how large p and q are.

The large case. An algorithm was recently given that, for p-vertex q-edge input graphs with edge weights bounded by $\mathrm{poly}(p)$ produces a $(O(\phi \log^3 p), \phi)$-ED[5] with probability $1 - 1/\mathrm{poly}(p)$ in $(p + q) \cdot \mathrm{polylog}(p)$ time, for any $\phi \in (0, 1)$ [LNPS23]. One can then use this ED algorithm to efficiently compute an approximate balanced ψ-sparse cut for all $\psi \in (0, 1)$: as it was observed in [SW19], given access to a graph and its $(O(\psi \log^3 p), \psi)$-ED, one can easily obtain a $(\psi, c' \cdot \log^3 p, C', 0)$-BSCW for some constants $c', C' \geq 1$. This gives a $(c' \cdot \log^3 p, C')$-BSCA that runs in $(p + q) \cdot \mathrm{polylog}(p)$ time for all input sparsity parameters $\psi \in (0, 1)$.

Whenever $p \geq n^{1/c_{\mathrm{time}}}$, for a constant $c_{\mathrm{time}} \geq 1$ to be determined later, we then apply the algorithm of [SW19, LNPS23] together with Claim A.1 setting $\alpha = c' \log^3 n$ and $\lambda = C'$ (where $c', C' \geq 1$ are the constants one gets from the self-loop free BSCA of [SW19, LNPS23]). Then, we get a $(c_{\mathrm{bsc}} \log^3 n, C_{\mathrm{bsc}})$-BSCA where $c_{\mathrm{bsc}} = 2c'$ and $C_{\mathrm{bsc}} = 4C'$, provided that $\phi \leq \frac{1}{10\alpha}$. Using $\phi = \psi$ and since we impose $\psi \leq \frac{1}{10 c_{\mathrm{bsc}} \log^3 n}$, we get $\phi \leq \frac{1}{10\alpha}$.

Using our assumption on p, q, W, the space and random bit complexity trivially follow from the bound on the running time, which is in turn at most $\mathrm{poly}(n)$. The success probability is $1 - 1/\mathrm{poly}(p)$, which gives the claimed success probability of $1 - 1/\mathrm{poly}(n)$.

The small case. When $p < n^{1/c_{\mathrm{time}}}$, we use an algorithm running in $T(p, q, W)$ time. The algorithm consists in using an algorithm that $(c'' \log p)$-approximates the sparsity of the input graph (such as the deterministic polynomial time algorithm of [LR99, LLR95], which allows to have weighted self-loops by incorporating them in the demand graph) as follows (which is essentially the idea of [SW19]): maintain a cut R initially empty and repeatedly compute the approximate sparsest cut S of $G[V \setminus R]^1$; if S has sparsity at least $2\psi c'' \log p$ then terminate, else update R to be $R \cup S$; if R accounts for at least a $1/5$ fraction of the total volume of G then terminate, else repeat.

First note that this algorithm terminates (e.g. when $V \setminus R$ is a single vertex). Also observe that if $R = \emptyset$ then we can conclude that G is a 2ψ-expander. Otherwise, R has always sparsity at most $4\psi c'' \log p$ in G and one of $R, V \setminus R$ hs volume at most half of the total. Moreover, if the smaller of $R, V \setminus R$ accounts for at least a $1/5$ fraction of the total volume of G then every ψ-sparse cut with volume at most half the total is within a $5/3$ fraction of the volume of the smaller of $R, V \setminus R$. Finally, if R accounts for less than a $1/5$ fraction of the total volume of G, then $G[V \setminus R]^1$ is a 2ψ-expander; consider a cut T with sparsity at most ψ and volume at most half the total; then if T had more than half its volume in $V \setminus R$, the weight of edges crossing the cut T in G would be more than a ψ fraction of the volume of T, a contradiction; hence, T is within a factor of 2 of the volume of R. This gives a $(c'' \log n, 2)$-BSCA.

We pick c_{time} so that $T(p, q, W)$ is at most $\tilde{O}(n/b^2) \cdot \log^{O(\frac{\log n}{\log 1/b})} n$, thus giving the claimed bound on the space and random bit complexity. In our case, $T(p, q, W) = \mathrm{poly}(p, q, \log W)$ (which can be achieved by solving the linear programming relaxation of [LR99] with the ellipsoid method and rounding it deterministically with the algorithm of [LLR95]) and recall that $W = \mathrm{poly}(n)$, so there exists a constant $c_{\mathrm{time}} \geq 1$ such that $T(p, q, W)$ is at most $n/b^2 \cdot \log^{O(\frac{\log n}{\log 1/b})} n$. The running time also follows, and the success probability is 1.

Theorem 6.2 is concluded by combining the large and small case.

□

[5] In this case, the definition of ED uses the weight of edges across cuts, and weighted volumes, including self-loops.

A.2 Expansion and regularity of random graphs

Fact 7.10. Let $N \geq 10$ be an integer and $p \in [0,1]$. Then, letting $\bar{d} = p(N-1)$ one has

$$\Pr_{H=(U,F)\sim \mathrm{ER}(N,p)}\left[\left(\Phi_H < \frac{1}{3}\right) \text{ or } \left(\exists u \in U \text{ s.t. } \left|\deg_H(u) - \bar{d}\right| > \frac{1}{11}\bar{d}\right)\right] \leq 4N \cdot \exp\left(-\frac{pN}{600}\right).$$

Proof. Let $H = (U,F)$ be a sample from $\mathrm{ER}(N,p)$, so that $|U| = N$. Also let $L = D - A$ be the Laplacian matrix of H, where D denotes its degree diagonal matrix and A denotes its adjacency matrix. We use the following matrix Bernstein concentration bound on L to lower bound its second eigenvalue, and then lower bound the expansion of H using Cheeger's inequality.

Theorem A.2 (Theorem 1.6.2 in [Tro15]). Let $S_1, \ldots, S_\kappa \in \mathbb{R}^{N\times N}$ be independent symmetric random matrices such that $\mathbb{E}[S_\iota] = 0$ and $\|S_\iota\| \leq \lambda$ for every $\iota \in [\kappa]$, and let $S = \sum_{\iota=1}^{\kappa} S_\iota$. Then, for all $\theta \geq 0$

$$\Pr[\|S\| \geq \theta] \leq 2N \cdot \exp\left(-\frac{\theta^2/2}{\|\sum_{\iota=1}^{\kappa}\mathbb{E}[S_\iota^2]\| + \lambda\theta/3}\right).$$

We start by rewriting L so that we can apply the above result. For each $e = \{u,v\} \in \binom{V}{2}$ let $z_e \sim \mathrm{Ber}(p)$ be an independent Bernoulli random variable with bias p, and also let L_e be the Laplacian matrix of the graph $(U, \{e\})$, i.e. $(L_e)_{ab} = 1$ if $a = b = u$ or $a = b = v$, $(L_e)_{ab} = -1$ if $\{a,b\} = e$, and $(L_e)_{ab} = 0$ otherwise. Define $S_e = z_e L_e - \mathbb{E}[z_e L_e]$ and $S = \sum_{e\in\binom{V}{2}} S_e$. One can now note that

$$\mathbb{E}[S_e] = 0 \quad \text{and} \quad \|S_e\| \leq \max\left\{\|(1-p)L_e\|, \|-pL_e\|\right\} \leq 2 \quad \text{for all } e \in \binom{V}{2}.$$

Letting J be the all-ones $N \times N$ matrix we also have

$$S = \sum_{e\in\binom{V}{2}} z_e L_e - \sum_{e\in\binom{V}{2}} \mathbb{E}[z_e L_e] = L - p\cdot(N\cdot I - J).$$

Hence, Theorem A.2 with $\lambda = 2$ gives

$$\Pr[\|L - p\cdot(N\cdot I - J)\| \geq \theta] \leq 2N \cdot \exp\left(-\frac{\theta^2/2}{\left\|\sum_{e\in\binom{V}{2}}\mathbb{E}[(z_e L_e - \mathbb{E}[z_e L_e])^2]\right\| + 2\theta/3}\right). \tag{A.6}$$

Now we observe that the first term in the denominator can be rewritten as

$$\begin{aligned}
\left\|\sum_{e\in\binom{V}{2}} \mathbb{E}\left[(z_e L_e - \mathbb{E}[z_e L_e])^2\right]\right\| &= \left\|\sum_{e\in\binom{V}{2}} 2L_e\,\mathbb{E}\left[(z_e - z_e\cdot p + p^2)\right]\right\| \\
&= \left\|\sum_{e\in\binom{V}{2}} 2pL_e\right\| \\
&= 2p\,\|N\cdot I - J\| \\
&= 2pN,
\end{aligned}$$

where the last equality holds because $K \coloneqq N \cdot I - J$ is the Laplacian matrix of the complete graph on N vertices. If we now set $\theta = pN/10$ in (A.6), we get

$$\Pr\left[\|L - p \cdot K\| \geq \frac{1}{10}pN\right] \leq 2N \cdot \exp\left(-\frac{pN}{600}\right). \tag{A.7}$$

Now, in order to use a Cheeger type of inequality, it is convenient to view H as a regular graph, so that its normalized Laplacian matrix is simply a scalar multiple of L. If we denote with $\deg(u)$ the degree of $u \in U$ in H, we get by a Chernoff bound that

$$\Pr\left[\exists u \in U,\ |\deg(u) - p \cdot (N-1)| > \frac{1}{14}p \cdot (N-1)\right] \leq 2N \cdot \exp\left(-\frac{p \cdot (N-1)}{600}\right). \tag{A.8}$$

From (A.7) and (A.8), we then have

$$\Pr\left[\left(\|L - pK\| \geq \frac{1}{10}pN\right) \text{ or } \left(\exists u \in U : \left|\deg_H(u) - \bar{d}\right| > \frac{1}{14}\bar{d}\right)\right] \leq 4N \cdot \exp\left(-\frac{pN}{600}\right). \tag{A.9}$$

We conclude by showing that if the event in the above probability does not occur, then the event argument of the probability in the statement of Fact 7.10 does not occur either. To see this, let λ_2 be the second smallest eigenvalue of $1/\bar{d} \cdot L$, and μ_2 be the second smallest eigenvalue of $1/(N-1) \cdot K$. Hence, $\|L - pK\| < \frac{1}{10}pN$ implies $\lambda_2 \geq \mu_2 - 1/9$ by Weyl's inequality. Moreover, we know that $\mu_2 = N/(N-1)$, so $\lambda_2 \geq 9/10$. Finally, let $\emptyset \neq S \subsetneq U$ be a cut with $\mathrm{vol}(S) \leq \mathrm{vol}(U \setminus S)$ such that $\Phi_H(S) = \Phi_H$. Assuming $\left|\deg_H(u) - \bar{d}\right| \leq \frac{1}{14}\bar{d}$ for all $u \in U$, we have that $\mathrm{vol}(S) \in [(1 - 1/14)\bar{d}|S|, (1 + 1/14)\bar{d}|S|]$. Let $\mathrm{y} \in \mathbb{R}^U$ be defined as $y_u = 1/|S|$ if $u \in S$ and $y_u = -1/|U \setminus S|$ otherwise. By Courant-Fischer we then have

$$\lambda_2 = \min_{\substack{\mathrm{x} \in \mathbb{R}^U \setminus \{0\}: \\ \langle \mathrm{x}, \mathbb{1} \rangle = 0}} \frac{1}{\bar{d}} \cdot \frac{\mathrm{x}^\top L \mathrm{x}}{\mathrm{x}^\top \mathrm{x}} \leq \frac{1}{\bar{d}} \cdot \frac{\mathrm{y}^\top L \mathrm{y}}{\mathrm{y}^\top \mathrm{y}} = \frac{\partial_U S}{\bar{d}|S|} \cdot \frac{|U|}{|U \setminus S|} \leq \left(1 + \frac{1}{14}\right) \Phi_H \cdot \frac{|U|}{|U \setminus S|}. \tag{A.10}$$

Again assuming $\left|\deg_H(u) - \bar{d}\right| \leq \frac{1}{14}\bar{d}$ for all $u \in U$, one can check that $\mathrm{vol}(S) \leq \mathrm{vol}(U \setminus S)$ implies $|U \setminus S| \geq 6/7 \cdot |S|$. Thus, (A.10) gives

$$\lambda_2 \leq \left(1 + \frac{1}{14}\right) \cdot \left(1 + \frac{7}{6}\right) \Phi_H \leq \frac{27}{10}\Phi_H,$$

so $\Phi_H \geq 1/3$. □

A.3 A useful inequality

Fact 7.21. For any $0 < x \leq y$, $(y/x)^x \leq \mathrm{e}^{y/\mathrm{e}}$.

Proof. Write $x = y \cdot \mathrm{e}^{\rho - 1}$ for some $\rho \leq 1$. Then $(y/x)^x \leq \mathrm{e}^{y/\mathrm{e}}$ if and only if $(1 - \rho)e^{\rho - 1} \leq 1/\mathrm{e}$. Define $f(\rho) = (1 - \rho)\mathrm{e}^\rho$. We prove the claim by showing that $f(\rho) \leq 1$ for all $\rho \in (-\infty, 1]$.

We begin by noting that f is continuous over $(-\infty, 1]$. Then, we take the derivative of f, and we get $f'(\rho) = -\rho\mathrm{e}^\rho$. We observe that $f'(\rho) > 0$ for $\rho < 0$, and $f'(\rho) > 0$ for $\rho > 0$. We further observe that $f(0) = 1$, $f(1) = 0$, and $\lim_{\rho \to -\infty} f(\rho) = 0$. Hence, $f(\rho)$ is upper-bounded by 1 for all $\rho \in (-\infty, 1]$. □

Algorithm 3 Decompose: space efficient implementation of Algorithm 1 for BLD construction

▷ $b, \phi \in (0,1)$ are global BLD parameters
▷ α, λ are the parameters of $\text{BalSparseCut}_{\alpha,\lambda}$
▷ $C \geq 1$, $c \in (0,1)$ are constant parameters
procedure $\text{Decompose}(U, \ell)$ ▷ cluster $U \subseteq V$, recursion level $\ell \geq 0$
    $H = (V, E', w) \leftarrow \text{Sparsifier}^c_\ell(U, b)$ ▷ sparsifier for U in this level
    $\omega \leftarrow \text{BalSparseCut}_{\alpha,\lambda}(H[U]^\circ, (1 + \frac{1}{2\log n}) \cdot \phi)$

    if $\omega = \perp$ then return $\{U\}$
    end if ▷ if U is an expander, return
    $(R, \nu) \leftarrow \omega$ ▷ otherwise, if R is balanced enough, recurse on both sides
    if $\nu \geq \frac{1}{C\lambda} \text{vol}^{\circ w}_U(U)$ then return $\text{Decompose}(R, \ell+1) \cup \text{Decompose}(U \setminus R, \ell+1)$
    end if

    $(S, \text{exp}) \leftarrow \text{Trim}(U, \ell)$ ▷ find an expander inside U, or settle for a less sparse balanced cut
    if $\text{exp} = \top$ then ▷ if S induces an expander, recurse on the complementary only
        return $\{S\} \cup \text{Decompose}(U \setminus S, \ell+1)$
    else ▷ otherwise, S should be decently sparse and balanced, so recurse on both sides
        return $\text{Decompose}(S, \ell+1) \cup \text{Decompose}(U \setminus S, \ell+1)$
    end if
end procedure

Algorithm 4 Trim: trim some cuts to find an expander, or find a less sparse but balanced cut

▷ $b, \phi \in (0,1)$ are global BLD parameters
▷ α, λ are the parameters of $\text{BalSparseCut}_{\alpha,\lambda}$
▷ $C \geq 1$, $c \in (0,1)$ are constant parameters
▷ k is a positive integer
procedure $\text{Trim}(U, \ell)$ ▷ cluster $U \subseteq V$, recursion level $\ell \geq 0$
    $b_0 \leftarrow b,\ \ \phi_0 \leftarrow \phi,\ \ A \leftarrow U$
    for $j = 1, \ldots, k+1$ do
        $b_j \leftarrow \frac{b_{j-1}}{(1+\frac{1}{\log n})\cdot\alpha},\ \ \phi_j \leftarrow \frac{\phi_{j-1}}{(1+\frac{1}{\log n})\cdot\alpha}$
        for $h = 1, 2, \ldots, \infty$ do
            $H = (V, E', w) \leftarrow \text{Sparsifier}^{c}_{\ell,j,h}(A, b_j)$ ▷ sparsifier for A in this level and iteration
            $\omega \leftarrow \text{BalSparseCut}_{\alpha,\lambda}(H[A]^{\circ}, (1 + \frac{1}{2\log n}) \cdot \phi_j)$
            if $\omega = \bot$ then return $(A, \top)$ ▷ $\top$ means A is an expander
            end if
            $(R, \nu) \leftarrow \omega$
            if $\nu \geq \frac{1}{C\lambda} \text{vol}^{\circ w}_{U}(U)$ then return $(R, \bot)$ ▷ $\bot$ means R is a cut
            end if
            if $\nu \geq \frac{1}{C\lambda} (\text{vol}^{\circ w}_{U}(U))^{1-(j-1)/k}$ then
                $A \leftarrow A \setminus R$ ▷ trim
            else
                break ▷ go to next outer loop iteration, if this happens we should have $j \leq k$
            end if
        end for
    end for
end procedure

Algorithm 5 SelfLoopBalSparseCut: reduction to self-loop free BSCA

▷ $G = (V, E, w)$ is a weighted undirected graph
▷ $\phi \in (0,1)$ is the sparsity parameter
procedure $\text{SelfLoopBalSparseCut}(G, \phi)$
    $\hat{\omega} \leftarrow \text{BalSparseCut}_{\alpha,\lambda}(\hat{G}, \phi)$ ▷ $\hat{G}$ is the self-loop free version of G as described above
    if $\hat{\omega} = \bot$ then
        $\omega \leftarrow \bot$
    else
        $(R, \nu) \leftarrow \hat{\omega}$
        $X \leftarrow R \cap V$
        $X^* \leftarrow \text{argmin}\{\text{vol}_G(X), \text{vol}_G(V \setminus X)\}$
        $\omega \leftarrow (X^*, \text{vol}_G(X^*))$
    end if
    return ω
end procedure